
Reticulum Network Stack

Release 1.1.6

Mark Qvist

Apr 16, 2026

CONTENTS

1	What is Reticulum?	3
1.1	Current Status	3
1.2	Reference Implementation	3
1.3	What does Reticulum Offer?	4
1.4	Where can Reticulum be Used?	5
1.5	Interface Types and Devices	5
2	Getting Started Fast	7
2.1	Standalone Reticulum Installation	7
2.1.1	Resolving Dependency & Installation Issues	7
2.2	Try Using a Reticulum-based Program	8
2.3	Using the Included Utilities	8
2.4	Creating a Network With Reticulum	8
2.5	Bootstrapping Connectivity	8
2.5.1	Finding Your Way	9
2.5.2	Build Personal Infrastructure	9
2.5.3	Mixing Strategies	10
2.5.4	Network Health & Responsibility	10
2.5.5	Contributing to the Global Ret	10
2.6	Connect to the Distributed Backbone	10
2.7	Hosting Public Entrypoints	11
2.8	Connecting Reticulum Instances Over the Internet	12
2.9	Adding Radio Interfaces	12
2.10	Creating and Using Custom Interfaces	13
2.11	Develop a Program with Reticulum	13
2.12	Platform-Specific Install Notes	13
2.12.1	Android	13
2.12.2	ARM64	14
2.12.3	Debian Bookworm	14
2.12.4	MacOS	15
2.12.5	OpenWRT	16
2.12.6	Raspberry Pi	16
2.12.7	RISC-V	17
2.12.8	Ubuntu Lunar	17
2.12.9	Windows	18
2.13	Pure-Python Reticulum	18
3	Zen of Reticulum	19
3.1	The Illusion Of The Center	19
3.1.1	Fallacy Of The Cloud	19

3.1.2	Decentralization Or Uncentralizability?	19
3.1.3	Death To The Address	20
3.2	Physics Of Trust	20
3.2.1	Hostile Environments	20
3.2.2	Encryption Is Not A Feature	21
3.2.3	Zero-Trust Architectures	21
3.3	Merits Of Scarcity	21
3.3.1	The Bandwidth Fallacy	22
3.3.2	Cost Of A Byte	22
3.3.3	Flow & Time	22
3.3.4	Liberation From Limits	23
3.4	Sovereignty Through Infrastructure	23
3.4.1	A Carrier-Grade Fallacy	23
3.4.2	Personal Infrastructure	23
3.4.3	The Ability To Disconnect	24
3.5	Identity and Nomadism	24
3.5.1	Portable Existence	24
3.5.2	Roaming Nodes	24
3.5.3	Announcing Presence	25
3.5.4	Anchor In The Flow	25
3.6	Ethics Of The Tool	25
3.6.1	The Harm Principle	26
3.6.2	Public Domain Protocol	26
3.6.3	Preserving Human Agency	26
3.7	Design Patterns For Post-IP Systems	27
3.7.1	Store & Forward	27
3.7.2	Naming Is Power	28
3.7.3	The Interface Is The Medium	28
3.7.4	Emergent Patterns	28
3.8	Fabric Of The Independent	29
3.8.1	The Work Is Finished	29
3.8.2	Open Sky	29
4	Programs Using Reticulum	31
4.1	Programs & Utilities	31
4.1.1	Remote Shell	31
4.1.2	Nomad Network	32
4.1.3	RNS Page Node	32
4.1.4	Retipedia	32
4.1.5	Sideband	33
4.1.6	MeshChatX	34
4.1.7	MeshChat	35
4.1.8	Columba	35
4.1.9	Reticulum Relay Chat	36
4.1.10	RetiBBS	36
4.1.11	RBrowser	37
4.1.12	Reticulum Network Telephone	38
4.1.13	LXST Phone	39
4.1.14	LXMFy	40
4.1.15	LXMF Interactive Client	40
4.1.16	RNS FileSync	40
4.1.17	Micron Parser JS	40
4.1.18	RNMon	40
4.2	Protocols	41

4.2.1	LXMF	41
4.2.2	LXST	41
4.2.3	RRC	41
4.3	Interface Modules & Connectivity Resources	41
5	Using Reticulum on Your System	43
5.1	Configuration & Data	43
5.2	Included Utility Programs	46
5.2.1	The rnsd Utility	47
5.2.2	The rnstatus Utility	47
5.2.3	The rnid Utility	49
5.2.4	The rnpath Utility	51
5.2.5	The rnprobe Utility	52
5.2.6	The rncp Utility	53
5.2.7	The rnx Utility	54
5.2.8	The rnodeconf Utility	55
5.3	Discovering Interfaces	57
5.4	Remote Management	59
5.5	Blackhole Management	59
5.5.1	Local Blackhole Management	60
5.5.2	Automated List Sourcing	60
5.5.3	Publishing Blackhole Lists	61
5.6	Improving System Configuration	62
5.6.1	Fixed Serial Port Names	62
5.6.2	Reticulum as a System Service	62
6	Understanding Reticulum	65
6.1	Motivation	65
6.2	Goals	66
6.3	Introduction & Basic Functionality	66
6.3.1	Destinations	67
6.3.2	Public Key Announcements	69
6.3.3	Identities	69
6.3.4	Getting Further	70
6.4	Reticulum Transport	70
6.4.1	Node Types	70
6.4.2	The Announce Mechanism in Detail	70
6.4.3	Reaching the Destination	71
6.4.4	Resources	73
6.5	Network Identities	74
6.5.1	Conceptual Overview	74
6.5.2	Current Usage	74
6.5.3	Future Implications	74
6.5.4	Creating and Using a Network Identity	75
6.6	Reference Setup	75
6.7	Protocol Specifics	76
6.7.1	Packet Prioritisation	76
6.7.2	Interface Access Codes	76
6.7.3	Wire Format	77
6.7.4	Announce Propagation Rules	79
6.7.5	Cryptographic Primitives	80
7	Communications Hardware	83
7.1	Combining Hardware Types	83

7.2	RNode	83
7.2.1	Creating RNodes	84
7.2.2	Supported Boards and Devices	84
7.2.3	Installation	90
7.2.4	Usage with Reticulum	91
7.3	WiFi-based Hardware	91
7.4	Ethernet-based Hardware	91
7.5	Serial Lines & Devices	91
7.6	Packet Radio Modems	91
8	Configuring Interfaces	93
8.1	Custom Interfaces	93
8.2	Auto Interface	93
8.3	Backbone Interface	95
8.3.1	Listeners	95
8.3.2	Connecting Remotes	96
8.4	TCP Server Interface	96
8.5	TCP Client Interface	98
8.6	UDP Interface	99
8.7	I2P Interface	100
8.8	RNode LoRa Interface	101
8.9	RNode Multi Interface	103
8.10	Serial Interface	105
8.11	Pipe Interface	105
8.12	KISS Interface	106
8.13	AX.25 KISS Interface	107
8.14	Discoverable Interfaces	108
8.14.1	Enabling Discovery	108
8.14.2	Discovery Parameters	108
8.14.3	Interface Modes	110
8.14.4	Security Considerations	110
8.14.5	Example Configuration	111
8.15	Common Interface Options	112
8.16	Interface Modes	113
8.17	Announce Rate Control	114
8.18	New Destination Rate Limiting	115
9	Building Networks	117
9.1	Concepts & Overview	117
9.1.1	Introductory Considerations	117
9.1.2	Destinations, Not Addresses	118
9.1.3	Transport Nodes and Instances	119
9.1.4	Trustless Networking	120
9.1.5	Heterogeneous Connectivity	121
10	Support Reticulum	123
10.1	Donations	123
10.2	Provide Feedback	123
11	Code Examples	125
11.1	Minimal	125
11.2	Announce	127
11.3	Broadcast	131
11.4	Echo	133
11.5	Link	140

11.6	Identification	145
11.7	Requests & Responses	152
11.8	Channel	157
11.9	Buffer	165
11.10	Filetransfer	171
11.11	Custom Interfaces	183
12	Reticulum License	191
13	API Reference	193
13.1	Reticulum	193
13.2	Identity	195
13.3	Destination	198
13.4	Packet	202
13.5	Packet Receipt	203
13.6	Link	204
13.7	Request Receipt	207
13.8	Resource	208
13.9	Channel	209
13.10	MessageBase	210
13.11	Buffer	211
13.12	RawChannelReader	212
13.13	RawChannelWriter	212
13.14	Transport	213
Index		215

This manual aims to provide you with all the information you need to understand Reticulum, build networks or develop programs using it, or to participate in the development of Reticulum itself.

WHAT IS RETICULUM?

Reticulum is a cryptography-based networking stack for building both local and wide-area networks with readily available hardware, that can continue to operate under adverse conditions, such as extremely low bandwidth and very high latency.

To understand the foundational philosophy and goals of this system, read the *Zen of Reticulum*.

Reticulum allows you to build wide-area networks with off-the-shelf tools, and offers end-to-end encryption, forward secrecy, autoconfiguring cryptographically backed multi-hop transport, efficient addressing, unforgeable packet acknowledgements and more.

From a users perspective, Reticulum allows the creation of applications that respect and empower the autonomy and sovereignty of communities and individuals. Reticulum enables secure digital communication that cannot be subjected to outside control, manipulation or censorship.

Reticulum enables the construction of both small and potentially planetary-scale networks, without any need for hierarchical or bureaucratic structures to control or manage them, while ensuring individuals and communities full sovereignty over their own network segments.

Reticulum is a **complete networking stack**, and does not need IP or higher layers, although it is easy to utilise IP (with TCP or UDP) as the underlying carrier for Reticulum. It is therefore trivial to tunnel Reticulum over the Internet or private IP networks. Reticulum is built directly on cryptographic principles, allowing resilience and stable functionality in open and trustless networks.

No kernel modules or drivers are required. Reticulum can run completely in userland, and will run on practically any system that runs Python 3. Reticulum runs well even on small single-board computers like the Pi Zero.

1.1 Current Status

All core protocol features are implemented and functioning, but additions will probably occur as real-world use is explored. The API and wire-format can be considered complete and stable, but could change if absolutely warranted.

1.2 Reference Implementation

The Python code, for which this documentation is written, and known as the Reticulum Network Stack, is the Reference Implementation of Reticulum. The Reticulum Protocol is defined entirely and authoritatively by this reference implementation, and this manual. It is maintained by Mark Qvist, identified by the Reticulum Identity <bc7291552be7a58f361522990465165c>.

Compatibility with the Reticulum Protocol is defined as having full interoperability, and sufficient functional parity with this reference implementation. Any specific protocol implementation that achieves this is Reticulum. Any that does not is not Reticulum.

The reference implementation is licensed under the *Reticulum License*.

The Reticulum Protocol was dedicated to the Public Domain in 2016.

1.3 What does Reticulum Offer?

- Coordination-less globally unique addressing and identification
- Fully self-configuring multi-hop routing over heterogeneous carriers
- Flexible scalability over heterogeneous topologies
 - Reticulum can carry data over any mixture of physical mediums and topologies
 - Low-bandwidth networks can co-exist and interoperate with large, high-bandwidth networks
- Initiator anonymity, communicate without revealing your identity
 - Reticulum does not include source addresses on any packets
- Asymmetric X25519 encryption and Ed25519 signatures as a basis for all communication
 - The foundational Reticulum Identity Keys are 512-bit Elliptic Curve keysets
- Forward Secrecy is available for all communication types, both for single packets and over links
- Reticulum uses the following format for encrypted tokens:
 - Ephemeral per-packet and link keys and derived from an ECDH key exchange on Curve25519
 - AES-256 in CBC mode with PKCS7 padding
 - HMAC using SHA256 for authentication
 - IVs are generated through `os.urandom()`
- Unforgeable packet delivery confirmations
- Flexible and extensible interface system
 - Reticulum includes a large variety of built-in interface types
 - Ability to load and utilise custom user- or community-supplied interface types
 - Easily create your own custom interfaces for communicating over anything
- Authentication and virtual network segmentation on all supported interface types
- An intuitive and easy-to-use API
 - Simpler and easier to use than sockets APIs and simpler, but more powerful
 - Makes building distributed and decentralised applications much simpler
- Reliable and efficient transfer of arbitrary amounts of data
 - Reticulum can handle a few bytes of data or files of many gigabytes
 - Sequencing, compression, transfer coordination and checksumming are automatic
 - The API is very easy to use, and provides transfer progress
- Lightweight, flexible and expandable Request/Response mechanism
- Efficient link establishment
 - Total cost of setting up an encrypted and verified link is only 3 packets, totalling 297 bytes
 - Low cost of keeping links open at only 0.44 bits per second
- Reliable sequential delivery with Channel and Buffer mechanisms

1.4 Where can Reticulum be Used?

Over practically any medium that can support at least a half-duplex channel with greater throughput than 5 bits per second, and an MTU of 500 bytes. Data radios, modems, LoRa radios, serial lines, AX.25 TNCs, amateur radio digital modes, ad-hoc WiFi, free-space optical links and similar systems are all examples of the types of interfaces Reticulum was designed for.

An open-source LoRa-based interface called [RNode](#) has been designed as an example transceiver that is very suitable for Reticulum. It is possible to build it yourself, to transform a common LoRa development board into one, or it can be purchased as a complete transceiver from various vendors.

Reticulum can also be encapsulated over existing IP networks, so there's nothing stopping you from using it over wired Ethernet or your local WiFi network, where it'll work just as well. In fact, one of the strengths of Reticulum is how easily it allows you to connect different mediums into a self-configuring, resilient and encrypted mesh.

As an example, it's possible to set up a Raspberry Pi connected to both a LoRa radio, a packet radio TNC and a WiFi network. Once the interfaces are added, Reticulum will take care of the rest, and any device on the WiFi network can communicate with nodes on the LoRa and packet radio sides of the network, and vice versa.

1.5 Interface Types and Devices

Reticulum implements a range of generalised interface types that covers the communications hardware that Reticulum can run over. If your hardware is not supported, it's simple to *implement an interface class*. Currently, Reticulum can use the following devices and communication mediums:

- Any Ethernet device
 - WiFi devices
 - Wired Ethernet devices
 - Fibre-optic transceivers
 - Data radios with Ethernet ports
- LoRa using [RNode](#)
 - Can be installed on [many popular LoRa boards](#)
 - Can be purchased as a [ready to use transceiver](#)
- Packet Radio TNCs, such as [OpenModem](#)
 - Any packet radio TNC in KISS mode
 - Ideal for VHF and UHF radio
- Any device with a serial port
- The I2P network
- TCP over IP networks
- UDP over IP networks
- Anything you can connect via stdio
 - Reticulum can use external programs and pipes as interfaces
 - This can be used to easily hack in virtual interfaces
 - Or to quickly create interfaces with custom hardware
- Anything else using *custom interface modules* written in Python

For a full list and more details, see the *Supported Interfaces* chapter.

GETTING STARTED FAST

The best way to get started with the Reticulum Network Stack depends on what you want to do. This guide will outline sensible starting paths for different scenarios.

2.1 Standalone Reticulum Installation

If you simply want to install Reticulum and related utilities on a system, the easiest way is via the `pip` package manager:

```
pip install rns
```

If you do not already have `pip` installed, you can install it using the package manager of your system with a command like `sudo apt install python3-pip`, `sudo pacman install python-pip` or similar.

You can also download the Reticulum release wheels from GitHub, or other release channels, and install them offline using `pip`:

```
pip install ./rns-1.1.2-py3-none-any.whl
```

On platforms that limit user package installation via `pip`, you may need to manually allow this using the `--break-system-packages` command line flag when installing. This will not actually break any packages, unless you have installed Reticulum directly via your operating system's package manager.

```
pip install rns --break-system-packages
```

For more detailed installation instructions, please see the *Platform-Specific Install Notes* section.

After installation is complete, it might be helpful to refer to the *Using Reticulum on Your System* chapter.

2.1.1 Resolving Dependency & Installation Issues

On some platforms, there may not be binary packages available for all dependencies, and `pip` installation may fail with an error message. In these cases, the issue can usually be resolved by installing the development essentials packages for your platform:

```
# Debian / Ubuntu / Derivatives
sudo apt install build-essential
```

```
# Arch / Manjaro / Derivatives
sudo pacman install base-devel
```

```
# Fedora
sudo dnf groupinstall "Development Tools" "Development Libraries"
```

With the base development packages installed, `pip` should be able to compile any missing dependencies from source, and complete installation even on platforms that don't have pre-compiled packages available.

2.2 Try Using a Reticulum-based Program

If you simply want to try using a program built with Reticulum, a *range of different programs* exist that allow basic communication and a various other useful functions, even over extremely low-bandwidth Reticulum networks.

2.3 Using the Included Utilities

Reticulum comes with a range of included utilities that make it easier to manage your network, check connectivity and make Reticulum available to other programs on your system.

You can use `rnsd` to run Reticulum as a background or foreground service, and the `rnsstatus`, `rnpath` and `rnprobe` utilities to view and query network status and connectivity.

To learn more about these utility programs, have a look at the *Using Reticulum on Your System* chapter of this manual.

2.4 Creating a Network With Reticulum

To create a network, you will need to specify one or more *interfaces* for Reticulum to use. This is done in the Reticulum configuration file, which by default is located at `~/.reticulum/config`. You can get an example configuration file with all options via `rnsd --exampleconfig`.

When Reticulum is started for the first time, it will create a default configuration file, with one active interface. This default interface uses your existing Ethernet and WiFi networks (if any), and only allows you to communicate with other Reticulum peers within your local broadcast domains.

To communicate further, you will have to add one or more interfaces. The default configuration includes a number of examples, ranging from using TCP over the internet, to LoRa and Packet Radio interfaces.

With Reticulum, you only need to configure what interfaces you want to communicate over. There is no need to configure address spaces, subnets, routing tables, or other things you might be used to from other network types.

Once Reticulum knows which interfaces it should use, it will automatically discover topography and configure transport of data to any destinations it knows about.

In situations where you already have an established WiFi or Ethernet network, and many devices that want to utilise the same external Reticulum network paths (for example over LoRa), it will often be sufficient to let one system act as a Reticulum gateway, by adding any external interfaces to the configuration of this system, and then enabling transport on it. Any other device on your local WiFi will then be able to connect to this wider Reticulum network just using the default (*AutoInterface*) configuration.

Possibly, the examples in the config file are enough to get you started. If you want more information, you can read the *Building Networks* and *Interfaces* chapters of this manual, but most importantly, start with reading the next section, *Bootstrapping Connectivity*, as this provides the most essential understanding of how to ensure reliable connectivity with a minimum of maintenance.

2.5 Bootstrapping Connectivity

Reticulum is not a service you subscribe to, nor is it a single global network you “join”. It is a *networking stack*; a toolkit for building communications systems that align with your specific values, requirements, and operational environment. The way you choose to connect to other Reticulum peers is entirely your own choice.

One of the most powerful aspects of Reticulum is that it provides a multitude of tools to establish, maintain, and optimize connectivity. You can use these tools in isolation or combine them in complex configurations to achieve a vast array of goals.

Whether your aim is to create a completely private, air-gapped network for your family; to build a resilient community mesh that survives infrastructure collapse; to connect far and wide to as many nodes as possible; or simply to maintain a reliable, encrypted link to a specific organization you care about, Reticulum provides the mechanisms to make it happen.

There is no “right” or “wrong” way to build a Reticulum network, and you don’t need to be a network engineer just to get started. If the information flows in the way you intend, and your privacy and security requirements are met, your configuration is a success. Reticulum is designed to make the most challenging and difficult scenarios attainable, even when other networking technologies fail.

2.5.1 Finding Your Way

When you first start using Reticulum, you need a way to obtain connectivity with the peers you want to communicate with - the process of *bootstrapping connectivity*.

Important

A common mistake in modern networking is the reliance on a few centralized, hard-coded entryptoints. If every user simply connects to the same list of public IP addresses found on a website, the network becomes brittle, centralized, and ultimately fails to deliver on the promise of decentralization and resilience. You have a responsibility here.

Reticulum encourages the approach of *organic growth*. Instead of relying on permanent static connections to distant servers, you can use temporary bootstrap connections to continuously *discover* more relevant or local infrastructure. Once discovered, your system can automatically form stronger, more direct links to these peers, and discard the temporary bootstrap links. This results in a web of connections that are geographically relevant, resilient and efficient.

It *is* possible to simply add a few public entryptoints to the [interfaces] section of your Reticulum configuration and be connected, but a better option is to enable *interface discovery* and either manually select relevant, local interfaces, or enable discovered interface auto-connection.

A relevant option in this context is the *bootstrap only* interface option. This is an automated tool for better distributing connectivity. By enabling interface discovery and auto-connection, and marking an interface as `bootstrap_only`, you tell Reticulum to use that interface primarily to find connectivity options, and then disconnect it once sufficient entryptoints have been discovered. This helps create a network topology that favors locality and resilience over the simple centralization caused by using only a few static entryptoints.

Good places to find interface definitions for bootstrapping connectivity are websites like [directory.rns.recipes](#) and [rmap.world](#).

2.5.2 Build Personal Infrastructure

You do not need a datacenter to be a meaningful part of the Reticulum ecosystem. In fact, the most important nodes in the network are often the smallest ones.

We strongly encourage everyone, even home users, to think in terms of building **personal infrastructure**. Don’t connect every phone, tablet, and computer in your house directly to a public internet gateway. Instead, repurpose an old computer, a Raspberry Pi, or a supported router to act as your own, personal **Transport Node**:

- Your local Transport Node sits in your home, connected to your WiFi and perhaps a radio interface (like an RNode).
- You configure this node with a `bootstrap_only` interface (perhaps a TCP tunnel to a wider network) and enable interface discovery.

- While you sleep, work, or cook, your node listens to the network. It discovers other local community members, validates their Network Identities, and automatically establishes direct links.
- Your personal devices now connect to your *local* node, which is integrated into a living, breathing local mesh. Your traffic flows through local paths provided by other real people in the community rather than bouncing off a distant server.

Don't wait for others to build the networks you want to see. Every network is important, perhaps even most so those that support individual families and persons. Once enough of this personal, local infrastructure exist, connecting them directly to each other, without traversing the public Internet, becomes inevitable.

2.5.3 Mixing Strategies

There is no requirement to commit to a single strategy. The most robust setups often mix static, dynamic, and discovered interfaces.

- **Static Interfaces:** You maintain a permanent interface to a trusted friend or organization using a static configuration.
- **Bootstrap Links:** You connect a `bootstrap_only` interface to a public gateway on the Internet to scan for new connectable peers or to regain connectivity if your other interfaces fail.
- **Local Wide-Area Connectivity:** You run a `RNodeInterface` on a shared frequency, giving you completely self-sovereign and private wide-area access to both your own network and other Reticulum peers globally, without any “service providers” being able to control or monitor how you interact with people.

By combining these methods, you create a system that is secure against single points of failure, adaptable to changing network conditions, and better integrated into your physical and social reality.

2.5.4 Network Health & Responsibility

As you participate in the wider networks you discover and build, you will inevitably encounter peers that are misconfigured, malicious, or simply broken. To protect your resources and those of your local peers, you can utilize the *Blackhole Management* system.

Whether you manually block a spamming identity or subscribe to a blackhole list maintained by a trusted Network Identity, these tools help ensure that *your* transport capacity is used for what *you* consider legitimate communication. This keeps your local segment efficient and contributes to the health of the wider network.

2.5.5 Contributing to the Global Ret

If you have the means to host a stable node with a public IP address, consider becoming a *Public Entrypoint*. By *publishing your interface as discoverable*, you provide a potential connection point for others, helping the network grow and reach new areas.

For guidelines on how to properly configure a public endpoint, refer to the *Hosting Public Entrypoints* section.

2.6 Connect to the Distributed Backbone

A global, distributed backbone of Reticulum Transport Nodes is being run by volunteers from around the world. This network constitutes a heterogenous collection of both public and private nodes that form an uncoordinated, voluntary inter-networking backbone that currently provides global transport and internetworking capabilities for Reticulum.

As a good starting point, you can find interface definitions for connecting your own networks to this backbone on websites such as directory.rns.recipes and rmap.world.

Tip

Don't rely on just a single connection to the distributed backbone for everyday use. It is much better to have several redundant connections configured, and enable the interface discovery options, so your nodes can continuously discover peering opportunities as the network evolves. Refer to the *Bootstrapping Connectivity* section to understand the options.

2.7 Hosting Public Entrypoints

If you want to help build a strong global interconnection backbone, you can host a public (or private) entry-point to a Reticulum network over the Internet. This section offers some helpful pointers. Once you have set up your public endpoint, it is a great idea to *make it discoverable over Reticulum*.

You will need a machine, physical or virtual with a public IP address, that can be reached by other devices on the Internet.

The most efficient and performant way to host a connectable entry-point supporting many users is to use the `BackboneInterface`. This interface type is fully compatible with the `TCPClientInterface` and `TCPServerInterface` types, but much faster and uses less system resources, allowing your device to handle thousands of connections even on small systems.

It is also important to set your connectable interface to gateway mode, since this will greatly improve network convergence time and path resolution for anyone connecting to your entry-point.

```
# This example demonstrates a backbone interface
# configured for acting as a gateway for users to
# connect to either a public or private network
```

[[Public Gateway]]

```
type = BackboneInterface
enabled = yes
mode = gateway
listen_on = 0.0.0.0
port = 4242

# On publicly available interfaces, it can be
# a good idea to configure sensible announce
# rate targets.
announce_rate_target = 3600
announce_rate_penalty = 3600
announce_rate_grace = 12
```

If instead you want to make a private entry-point from the Internet, you can use the *IFAC name and passphrase options* to secure your interface with a network name and passphrase.

```
# A private entry-point requiring a pre-shared
# network name and passphrase to connect to.
```

[[Private Gateway]]

```
type = BackboneInterface
enabled = yes
mode = gateway
listen_on = 0.0.0.0
```

(continues on next page)

(continued from previous page)

```
port = 4242
network_name = private_ret
passphrase = 2owjajquafIanPecAc
```

If you are hosting an entry-point on an operating system that does not support `BackboneInterface`, you can use `TCPServerInterface` instead, although it will not be as performant.

2.8 Connecting Reticulum Instances Over the Internet

Reticulum currently offers three interfaces suitable for connecting instances over the Internet: *Backbone*, *TCP* and *I2P*. Each interface offers a different set of features, and Reticulum users should carefully choose the interface which best suites their needs.

The `TCPServerInterface` allows users to host an instance accessible over TCP/IP. This method is generally faster, lower latency, and more energy efficient than using `I2PInterface`, however it also leaks more data about the server host.

The `BackboneInterface` is a very fast and efficient interface type available on POSIX operating systems, designed to handle thousands of connections simultaneously with low memory, processing and I/O overhead. It is fully compatible with the TCP-based interface types.

TCP connections reveal the IP address of both your instance and the server to anyone who can inspect the connection. Someone could use this information to determine your location or identity. Adversaries inspecting your packets may be able to record packet metadata like time of transmission and packet size. Even though Reticulum encrypts traffic, TCP does not, so an adversary may be able to use packet inspection to learn that a system is running Reticulum, and what other IP addresses connect to it. Hosting a publicly reachable instance over TCP also requires a publicly reachable IP address, which most Internet connections don't offer anymore.

The `I2PInterface` routes messages through the *Invisible Internet Protocol (I2P)*. To use this interface, users must also run an I2P daemon in parallel to `rnsd`. For always-on I2P nodes it is recommended to use `i2pd`.

By default, I2P will encrypt and mix all traffic sent over the Internet, and hide both the sender and receiver Reticulum instance IP addresses. Running an I2P node will also relay other I2P user's encrypted packets, which will use extra bandwidth and compute power, but also makes timing attacks and other forms of deep-packet-inspection much more difficult.

I2P also allows users to host globally available Reticulum instances from non-public IP's and behind firewalls and NAT.

In general it is recommended to use an I2P node if you want to host a publicly accessible instance, while preserving anonymity. If you care more about performance, and a slightly easier setup, use TCP.

2.9 Adding Radio Interfaces

Once you have Reticulum installed and working, you can add radio interfaces with any compatible hardware you have available. Reticulum supports a wide range of radio hardware, and if you already have any available, it is very likely that it will work with Reticulum. For information on how to configure this, see the *Interfaces* section of this manual.

If you do not already have transceiver hardware available, you can easily and cheaply build an *RNode*, which is a general-purpose long-range digital radio transceiver, that integrates easily with Reticulum.

To build one yourself requires installing a custom firmware on a supported LoRa development board with an auto-install script or web-based flasher. Please see the *Communications Hardware* chapter for a guide. If you prefer purchasing a ready-made unit, you can refer to the list of suppliers.

Other radio-based hardware interfaces are being developed and made available by the broader Reticulum community. You can find more information on such topics over Reticulum-based information sharing systems.

If you have communications hardware that is not already supported by any of the *existing interface types*, it is easy to write (and potentially publish) a *custom interface module* that makes it compatible with Reticulum.

2.10 Creating and Using Custom Interfaces

While Reticulum includes a flexible and broad range of built-in interfaces, these will not cover every conceivable type of communications hardware that Reticulum can potentially use to communicate.

It is therefore possible to easily write your own interface modules, that can be loaded at run-time and used on-par with any of the built-in interface types.

For more information on this subject, and code examples to build on, please see the *Configuring Interfaces* chapter.

2.11 Develop a Program with Reticulum

If you want to develop programs that use Reticulum, the easiest way to get started is to install the latest release of Reticulum via pip:

```
pip install rns
```

The above command will install Reticulum and dependencies, and you will be ready to import and use RNS in your own programs. The next step will most likely be to look at some *Example Programs*.

The entire Reticulum API is documented in the *API Reference* chapter of this manual. Before diving in, it's probably a good idea to read this manual in full, but at least start with the *Understanding Reticulum* chapter.

2.12 Platform-Specific Install Notes

Some platforms require a slightly different installation procedure, or have various quirks that are worth being aware of. These are listed here.

2.12.1 Android

Reticulum can be used on Android in different ways. The easiest way to get started is using an app like *Sideband*.

For more control and features, you can use Reticulum and related programs via the *Termux* app, at the time of writing available on *F-droid*.

Termux is a terminal emulator and Linux environment for Android based devices, which includes the ability to use many different programs and libraries, including Reticulum.

To use Reticulum within the Termux environment, you will need to install *python* and the *python-cryptography* library using *pkg*, the package-manager build into Termux. After that, you can use *pip* to install Reticulum.

From within Termux, execute the following:

```
# First, make sure indexes and packages are up to date.
pkg update
pkg upgrade

# Then install python and the cryptography library.
pkg install python python-cryptography

# Make sure pip is up to date, and install the wheel module.
pip install wheel pip --upgrade
```

(continues on next page)

(continued from previous page)

```
# Install Reticulum
pip install rns
```

If for some reason the `python-cryptography` package is not available for your platform via the Termux package manager, you can attempt to build it locally on your device using the following command:

```
# First, make sure indexes and packages are up to date.
pkg update
pkg upgrade

# Then install dependencies for the cryptography library.
pkg install python build-essential openssl libffi rust

# Make sure pip is up to date, and install the wheel module.
pip install wheel pip --upgrade

# To allow the installer to build the cryptography module,
# we need to let it know what platform we are compiling for:
export CARGO_BUILD_TARGET="aarch64-linux-android"

# Start the install process for the cryptography module.
# Depending on your device, this can take several minutes,
# since the module must be compiled locally on your device.
pip install cryptography

# If the above installation succeeds, you can now install
# Reticulum and any related software
pip install rns
```

It is also possible to include Reticulum in apps compiled and distributed as Android APKs. A detailed tutorial and example source code will be included here at a later point. Until then you can use the [Sideband source code](#) as an example and starting point.

2.12.2 ARM64

On some architectures, including ARM64, not all dependencies have precompiled binaries. On such systems, you may need to install `python3-dev` (or similar) before installing Reticulum or programs that depend on Reticulum.

```
# Install Python and development packages
sudo apt update
sudo apt install python3 python3-pip python3-dev

# Install Reticulum
python3 -m pip install rns
```

With these packages installed, `pip` will be able to build any missing dependencies on your system locally.

2.12.3 Debian Bookworm

On versions of Debian released after April 2023, it is no longer possible by default to use `pip` to install packages onto your system. Unfortunately, you will need to use the replacement `pipx` command instead, which places installed packages in an isolated environment. This should not negatively affect Reticulum, but will not work for including and using Reticulum in your own scripts and programs.

```
# Install pipx
sudo apt install pipx

# Make installed programs available on the command line
pipx ensurepath

# Install Reticulum
pipx install rns
```

Alternatively, you can restore normal behaviour to `pip` by creating or editing the configuration file located at `~/.config/pip/pip.conf`, and adding the following section:

```
[global]
break-system-packages = true
```

For a one-shot installation of Reticulum, without globally enabling the `break-system-packages` option, you can use the following command:

```
pip install rns --break-system-packages
```

Note

The `--break-system-packages` directive is a somewhat misleading choice of words. Setting it will of course not break any system packages, but will simply allow installing `pip` packages user- and system-wide. While this *could* in rare cases lead to version conflicts, it does not generally pose any problems, especially not in the case of installing Reticulum.

2.12.4 MacOS

To install Reticulum on macOS, you will need to have Python and the `pip` package manager installed.

Systems running macOS can vary quite widely in whether or not Python is pre-installed, and if it is, which version is installed, and whether the `pip` package manager is also installed and set up. If in doubt, you can [download and install](#) Python manually.

When Python and `pip` is available on your system, simply open a terminal window and use one of the following commands:

```
# Install Reticulum and utilities with pip:
pip3 install rns

# On some versions, you may need to use the
# flag --break-system-packages to install:
pip3 install rns --break-system-packages
```

Note

The `--break-system-packages` directive is a somewhat misleading choice of words. Setting it will of course not break any system packages, but will simply allow installing `pip` packages user- and system-wide. While this *could* in rare cases lead to version conflicts, it does not generally pose any problems, especially not in the case of installing Reticulum.

Additionally, some version combinations of macOS and Python require you to manually add your installed `pip` packages directory to your `PATH` environment variable, before you can use installed commands in your terminal. Usually, adding the following line to your shell init script (for example `~/.zshrc`) will be enough:

```
export PATH=$PATH:~/Library/Python/3.9/bin
```

Adjust Python version and shell init script location according to your system.

2.12.5 OpenWRT

On OpenWRT systems with sufficient storage and memory, you can install Reticulum and related utilities using the `opkg` package manager and `pip`.

Note

At the time of releasing this manual, work is underway to create pre-built Reticulum packages for OpenWRT, with full configuration, service and `uci` integration. Please see the [feed-reticulum](#) and [reticulum-openwrt](#) repositories for more information.

To install Reticulum on OpenWRT, first log into a command line session, and then use the following instructions:

```
# Install dependencies
opkg install python3 python3-pip python3-cryptography python3-pyserial

# Install Reticulum
pip install rns

# Start rnsd with debug logging enabled
rnsd -vvv
```

Note

The above instructions have been verified and tested on OpenWRT 21.02 only. It is likely that other versions may require slightly altered installation commands or package names. You will also need enough free space in your overlay FS, and enough free RAM to actually run Reticulum and any related programs and utilities.

Depending on your device configuration, you may need to adjust firewall rules for Reticulum connectivity to and from your device to work. Until proper packaging is ready, you will also need to manually create a service or startup script to automatically launch Reticulum at boot time.

Please also note that the *AutoInterface* requires link-local IPv6 addresses to be enabled for any Ethernet and WiFi devices you intend to use. If `ip a` shows an address starting with `fe80::` for the device in question, *AutoInterface* should work for that device.

2.12.6 Raspberry Pi

It is currently recommended to use a 64-bit version of the Raspberry Pi OS if you want to run Reticulum on Raspberry Pi computers, since 32-bit versions don't always have packages available for some dependencies. If Python and the `pip` package manager is not already installed, do that first, and then install Reticulum using `pip`.

```
# Install dependencies
sudo apt install python3 python3-pip python3-cryptography python3-pyserial
```

(continues on next page)

(continued from previous page)

```
# Install Reticulum
pip install rns --break-system-packages
```

Note

The `--break-system-packages` directive is a somewhat misleading choice of words. Setting it will of course not break any system packages, but will simply allow installing `pip` packages user- and system-wide. While this *could* in rare cases lead to version conflicts, it does not generally pose any problems, especially not in the case of installing Reticulum.

While it is possible to install and run Reticulum on 32-bit Raspberry Pi OSes, it will require manually configuring and installing required build dependencies, and is not detailed in this manual.

2.12.7 RISC-V

On some architectures, including RISC-V, not all dependencies have precompiled binaries. On such systems, you may need to install `python3-dev` (or similar) before installing Reticulum or programs that depend on Reticulum.

```
# Install Python and development packages
sudo apt update
sudo apt install python3 python3-pip python3-dev

# Install Reticulum
python3 -m pip install rns
```

With these packages installed, `pip` will be able to build any missing dependencies on your system locally.

2.12.8 Ubuntu Lunar

On versions of Ubuntu released after April 2023, it is no longer possible by default to use `pip` to install packages onto your system. Unfortunately, you will need to use the replacement `pipx` command instead, which places installed packages in an isolated environment. This should not negatively affect Reticulum, but will not work for including and using Reticulum in your own scripts and programs.

```
# Install pipx
sudo apt install pipx

# Make installed programs available on the command line
pipx ensurepath

# Install Reticulum
pipx install rns
```

Alternatively, you can restore normal behaviour to `pip` by creating or editing the configuration file located at `~/.config/pip/pip.conf`, and adding the following section:

```
[global]
break-system-packages = true
```

For a one-shot installation of Reticulum, without globally enabling the `break-system-packages` option, you can use the following command:

```
pip install rns --break-system-packages
```

Note

The `--break-system-packages` directive is a somewhat misleading choice of words. Setting it will of course not break any system packages, but will simply allow installing `pip` packages user- and system-wide. While this *could* in rare cases lead to version conflicts, it does not generally pose any problems, especially not in the case of installing Reticulum.

2.12.9 Windows

On Windows operating systems, the easiest way to install Reticulum is by using the `pip` package manager from the command line (either the command prompt or Windows Powershell).

If you don't already have Python installed, [download and install Python](#). At the time of publication of this manual, the recommended version is [Python 3.12.7](#).

Important! When asked by the installer, make sure to add the Python program to your PATH environment variables. If you don't do this, you will not be able to use the `pip` installer, or run the included Reticulum utility programs (such as `rnsd` and `rnstatus`) from the command line.

After installing Python, open the command prompt or Windows Powershell, and type:

```
pip install rns
```

You can now use Reticulum and all included utility programs directly from your preferred command line interface.

2.13 Pure-Python Reticulum

Warning

If you use the `rnspure` package to run Reticulum on systems that do not support [PyCA/cryptography](#), it is important that you read and understand the [Cryptographic Primitives](#) section of this manual.

In some rare cases, and on more obscure system types, it is not possible to install one or more dependencies. In such situations, you can use the `rnspure` package instead of the `rns` package, or use `pip` with the `--no-dependencies` command-line option. The `rnspure` package requires no external dependencies for installation. Please note that the actual contents of the `rns` and `rnspure` packages are *completely identical*. The only difference is that the `rnspure` package lists no dependencies required for installation.

No matter how Reticulum is installed and started, it will load external dependencies only if they are *needed* and *available*. If for example you want to use Reticulum on a system that cannot support `pyserial`, it is perfectly possible to do so using the `rnspure` package, but Reticulum will not be able to use serial-based interfaces. All other available modules will still be loaded when needed.

ZEN OF RETICULUM

3.1 The Illusion Of The Center

For the better part of a generation, we have been taught to visualize the digital world through the lens of hierarchy. The mental maps we carry are dominated by a single, misleading image: **The Cloud**.

We imagine the network as a vast, ethereal space “up there” or “out there”. A centralized repository of services and data to which we, the lowly clients, must connect. We build our software with this assumption hardcoded into our logic: *There is a server. The server has the authority. The server knows the way. I must find the server to function.*

This is the Client-Server mental model, and it is the primary obstacle to understanding Reticulum.

3.1.1 Fallacy Of The Cloud

The first step in the Zen of Reticulum is to realize that *there is no cloud*. There is only other people’s computers. When you build for the cloud, you are building *for* a landlord. You are accepting that your application’s existence is conditional on the permission, uptime, and continued goodwill of a central authority.

In Reticulum, you must shift your thinking from “connecting to” to “being among”. Reticulum is not a service you subscribe to - *it is a fabric you inhabit*. There is no “up there”. There is only *here* and *there*, and the space between them is peer-to-peer.

3.1.2 Decentralization Or Uncentralizability?

It is common to hear the word “decentralized” thrown around in modern tech circles. But often, this is merely a marketing term for “slightly distributed centralization”. A blockchain with a few dominant miners, or a federated protocol with a few giant servers. *In practice*, it’s still centralized. It simply has a few centers instead of one.

Reticulum goes further. It wants **Uncentralizability**.

This is not a wishful political stance, but a foundational mathematical characteristic of the protocol, onto which everything else has been built. Reticulum assumes that every peer on the network is potentially hostile, and every link is potentially compromised. It is designed with no “privileged” nodes. While some nodes may act as Transport Instances - forwarding traffic for others - they do so *blindly*, and they only know about their immediate surroundings, and nothing more. They route based on cryptographic proofs, not on administrative privilege. They cannot see who is talking to whom, nor can they selectively manipulate traffic without breaking their own ability to route entirely.

The system is designed to make hierarchy structurally impossible. You cannot hijack an address, because there is no central registry to hijack. You cannot block a user, because there is no central switch to flip. You can offer paths through the network, but you can’t force anyone to use them.

3.1.3 Death To The Address

To break free of the center, you must also let go of the concept of the “Address”.

In the IP world, an address is a location. It is a coordinate in a *deeply hierarchical* and static grid. If you move your computer to a different house, your address changes. If your router reboots, your address might change. Your *identity* is bound to your *location*, and therefore, it is fragile, and easily controlled.

Reticulum abolishes this link between *Identity* and *Location*.

In Reticulum, an address is not a place; it is a **Hash of an Identity**. It is a cryptographic representation of *who* you are, not *where* you are. Because of this, your address is portable. You can take a laptop from a WiFi cafe in Berlin, to a LoRa mesh in the mountains, to a packet radio link on a boat, and your “address” - your *Destination Hash* - never changes.

The network does not route to a place; it routes to a *person* (or a machine). When you send a packet, you are not targeting a coordinate in a grid; you are encrypting a message for a specific entity. The network dynamically discovers where that entity currently resides, and it does so in a way where no one really knows where that entity is actually located physically.

Consider:

- **The Old Way:** “*I am at 192.168.1.5. Come find me*”.
- **The Zen Way:** “*I am <327c1b2f87c9353e01769b01090b18f2>. Wherever I am, my peers can reach me*”.

Once you stop thinking about servers and start thinking about portable identities, where everyone can always reach everyone else directly, the illusion of the center fades away. You realize there *is* no center holding the network together. No coordinators or bureaucrats required. The network is simply the sum of its peers, communicating directly, sovereignly, and without a master.

3.2 Physics Of Trust

Paranoia Is A Great Design Principle

If we accept that there is no center - that the network is a chaotic, peer-to-peer mesh - we are forced to confront a terrifying reality: **There is no one guarding the door.**

In the traditional networking mindset, we rely on the concept of the “trusted core”. We assume our local coffee shop WiFi is safe, or that the backbone providers are neutral custodians. We build our security like a castle: strong walls on the outside, soft and trusting on the inside. We use encryption only when we step out into the “wild” internet.

3.2.1 Hostile Environments

The Zen of Reticulum requires you to invert this. You must assume that *every* environment is hostile. This isn’t cynicism, just uncaring physics.

When you transmit information over radio waves, you are shouting into a crowded room. Anyone can listen. When you traverse the internet, your packets pass through routers controlled by strangers, corporations, and state actors. Assuming privacy in this environment without cryptographic protection is not optimism but gross negligence.

Reticulum is built on the premise that every link is tapped, and every peer is a potential adversary. If your system cannot survive an adversary owning the physical layer, it cannot survive at all.

But this is the paradox: By assuming the network is hostile, you make it safe. When you accept the dangers for what they are, they become manageable. When you stop trusting the infrastructure and start trusting the math, you eliminate the single point of failure: Human integrity.

3.2.2 Encryption Is Not A Feature

In the world of TCP/IP, encryption is an afterthought. It is a layer we slap on top of the protocol (HTTPS, TLS) to patch the security holes of the original design. It is a “feature” you sometimes *enable* for “sensitive data”. This is fundamentally flawed, since all data is sensitive.

In Reticulum, encryption is **gravity**.

It is not optional. It is not a plugin. It is the *fundamental force that allows the network to exist*. If you were to strip the encryption from Reticulum, the routing would break. The Transport system uses cryptographic signatures and entropy to verify paths and pass information. If packets were plaintext, intermediate nodes could not prove that a route was valid, nor could endpoints prevent spoofing or tampering.

In Reticulum, the entropy of the encrypted packet *is* the routing logic.

To ask for a version of Reticulum without encryption is like asking for a version of the ocean without liquid. You are not asking for a feature change; you’re asking for a different physical universe. We design for a universe where information has mass, structure, and integrity.

3.2.3 Zero-Trust Architectures

We must unlearn our reliance on **Institutional Trust**.

For decades, we have been trained to trust authorities. We trust a website because a chain of Certificate Authorities (companies we don’t know) vouches for it. We trust an app because it is in an app store (run by a corporation we don’t control). We trust a message because it comes from a phone number assigned by a telecom. Yet, everything in our digital information sphere today is more untrustworthy and risky than a medieval second-hand underwear market.

Reticulum replaces institutional trust with **Cryptographic Proof**.

In Reticulum, you do not trust a node because it has a nice hostname or because it is listed in a directory. You trust it because it holds the private key corresponding to the Destination Hash you are communicating with. This trust is binary, mathematical, and **absolute**. Either the signature matches, or it does not. There is no “maybe”.

This shift moves the power from the institution to the individual. You become the ultimate arbiter of your own trust relationships. You decide which keys to accept, which paths to follow, and which identities to recognize.

Consider:

- **The Old Way:** *“I trust this site because the browser says the lock icon is green”.*
- **The Zen Way:** *“I trust this destination because I have verified its hash fingerprint out-of-band, and the math confirms the signature”.*

When you internalize the Physics of Trust, you stop looking for protection from firewalls, VPNs, and Terms of Service agreements. You realize that true security comes from the design of the protocol itself. You can stop trusting the cloud, and you start trusting the code - because you can verify it yourself.

3.3 Merits Of Scarcity

Every Bit Counts

We have grown addicted to abundance. In the modern digital ecosystem, bandwidth is treated as an endless, flat ocean. We stream high-definition video without a thought, we ship entire libraries of code just to render a single button, and we measure performance in gigabits per second. This abundance has hollowed out our craft. When constraints vanish, efficiency dies, and with it, a certain kind of Clarity and Quality.

Reticulum asks you to step out of the ocean and onto the tightrope.

3.3.1 The Bandwidth Fallacy

The Zen of Reticulum requires the realization that **5 bits per second is a valid speed**.

To a modern developer, this sounds like paralysis. But there is a profound freedom in limits: When you have a gigabit connection, you can be incredibly sloppy. You can be wasteful. You can push your problems onto the infrastructure. *“It’s slow? Get a faster router”*.

But on a high-latency, low-bandwidth link (be it a noisy HF radio channel or a tenuous LoRa hop) you cannot push problems anywhere. You must solve them. The network does not negotiate with waste.

This forces a shift from consumption to interaction. You are no longer, then, consuming a service provided by a fat pipe; you are engaging in a careful negotiation with the physical medium. The medium becomes a partner in the conversation, not just a dumb conduit. You suddenly need to *understand the world to be in it*.

3.3.2 Cost Of A Byte

In a scarce economy, a byte is not just data, but energy, time, and space.

Every byte you transmit consumes battery life on a solar-powered node. It occupies valuable airtime that could have been used by another peer. It represents a measurable slice of the electromagnetic spectrum.

When you internalize this, you begin to write code differently. You stop asking, “How much data can I send?” and start asking, “What is the *minimum* amount of information required to convey this intent? How can I best utilize my informational entropy?”

This is where the elegance of Reticulum shines. The protocol is designed to strip away the non-essential. A link establishment takes three very small packets. A destination hash fits in 16 bytes. The overhead is vanishingly small, leaving almost the entire channel for the message itself.

Consider:

- **The Old Way:** *“I need to send a status update. I’ll send a JSON object with metadata, timestamps, and user profile info (15KB).”*
- **The Zen Way:** *“I need to send a status update. I’ll send a single byte representing the state code. The context is already known.”*

This is of course optimization, but more importantly, *it is a form of respect*. Efficiency in a shared medium is an act of stewardship. By taking only what you need from the network, you leave room for others. The network listens to those who speak with purpose.

3.3.3 Flow & Time

Scarcity also teaches us about time. We have become addicted to the *synchronous* now - the instant ping, the real-time stream. But Reticulum embraces *asynchronous* time.

When links are intermittent and latency is measured in minutes or hours, “real-time” is an illusion. Reticulum doesn’t encourage **Store and Forward** as a mere fallback, but as a primary mode of existence. You write a message, it propagates when it can, and it arrives when it arrives.

This changes the psychological texture of communication. It removes the anxiety of the immediate response. It allows for contemplation. You are not demanding the recipient’s attention *right now*; you are placing a gift in their path, to be found when they are ready.

By designing for delay, you design for resilience. You are no longer building a house of cards that collapses when a single packet drops. You are building a stone arch that distributes the load *over time*.

3.3.4 Liberation From Limits

There is a strange optimism in scarcity. When you are forced to work within strict constraints, you are forced to prioritize. *You* must decide what truly matters. *That* is the real core of agency.

In the infinite fantasy world of The Cloud, everything is urgent, so nothing is. In the economy of Reticulum, the cost of transmission forces you to weigh the value of your message. Do you really need to send that heart beat? Is that photo essential?

When you strip away the noise, what remains is *signal*.

This discipline creates a different kind of developer. It creates a craftsman who understands that the best code is the code you don't have to write. It creates a user who understands that the most powerful message is the one that is *understood*, not the one that is loudest. In the world of Reticulum, you are not a mere consumer of bandwidth; you are an architect of intent.

3.4 Sovereignty Through Infrastructure

Be Your Own Network

We live in an era of digital tenancy. We lease our connectivity from ISPs. We rent our storage from cloud providers. We even borrow our identity from social media platforms. We are tenants in a house we did not build, governed by rules we did not write, subject to eviction at the whim of a landlord who has never met us.

The Zen of Reticulum is the realization that you *can* own the house.

3.4.1 A Carrier-Grade Fallacy

For decades, we have been gaslit into believing that networking is really not just hard, but impossible. It is presented as a dark art reserved for telcos and billionaires, requiring millions of dollars of fiber optics, climate-controlled data centers, and armies of engineers. We are told that building reliable infrastructure is “too complex” for the individual or small organization.

This is a big, fat lie.

Physics is simple. A radio wave needs a transmitter and a receiver. A packet needs a path. The “complexity” of the modern internet is largely bureaucratic - a mountain of billing systems, regulatory capture, and legacy cruft designed to keep the gatekeepers in power.

Reticulum strips away the bureaucracy. It runs on hardware that costs the price of a dinner. It runs on spectrum that is free to use. It demonstrates that a robust, planetary-scale network does not require a Fortune 500 company. It requires only the will to deploy, and the distributed, uncoordinated efforts of many individuals.

3.4.2 Personal Infrastructure

This is where the rubber meets the road. You can read about Reticulum, you can understand the theory, but the insights only arrive when you plug in a radio and run a Transport Node. Suddenly, you are no longer a consumer. You're an operator.

This shift is subtle but profound. When you run your own infrastructure, the network ceases to be a service that is provided *to* you. It becomes a space that you *inhabit*. You become responsible for the flow of information. You gain an intimate understanding of the medium - the way the weather affects the radio waves, the way the topology changes, the way the packets dance through the ether.

There is a quiet competence that comes from this. You stop asking “Is the internet down?” and start asking “Is *my* links up?” You stop waiting for a technician and start checking the logs. This is a form of strength. To understand the system that carries your words is to be free from the mystery that keeps you dependent.

3.4.3 The Ability To Disconnect

Why go to the trouble? Why buy the radio, write the config, and leave the Pi running in the corner?

Because the old, centralized network is fragile. And because most of us doesn't even really want to be there anymore.

The internet we rely on today is a chain of single points of failure. Cut the undersea cable, and a continent goes dark. Shut down the power grid, and the cloud evaporates. Deprioritize the “wrong” traffic, and the flow of information is strangled.

Sovereignty is the ability to survive the cut, whether or not that cut was an accident or on purpose.

When you build your own infrastructure, you build a lifeline. Reticulum is designed to function over media that the traditional internet cannot touch - bare wires, battery-powered radios, ad-hoc WiFi meshes. When the grid fails, or the sensors arrive, or the bill goes unpaid, your Reticulum network continues to hum.

This is not about “dropping out” of society. It is about building a substrate on which an actual *Society* can function.

Consider:

- **The Old Way:** “My connection is slow. I should call my ISP and complain.”
- **The Zen Way:** “The path is noisy. I will adjust the antenna or find a better route.”

By taking ownership of the infrastructure, you take ownership of your voice. You stop shouting into someone else's megaphone and start building your own. The network is no longer something that happens to you; it is something you make happen.

3.5 Identity and Nomadism

A Fluid Self

In the old world, you are defined by your coordinates. If you are at 34.109.71.5, you're *here*. If you unplug the cable and walk down the street, you vanish. Your digital self evaporates because it was tethered to the wall. You are a ghost in the endless machinations of gears, levers and transistors, bound to the hardware, and those that own it.

This creates a subtle, constant anxiety. We are terrified of disconnecting because, in the architecture of the old web, disconnecting is a kind of death.

The Zen of Reticulum offers a different way to be.

3.5.1 Portable Existence

In Reticulum, your identity is not a location, or a username granted by a service. It is a cryptographic key - a complex, unique mathematical signature that exists independently of the physical world. You can carry it only in your mind, if you want to.

Think of it less like a street address and more like a name. *A true name.*

If you travel from Berlin to Tokyo, you do not change your name. You are still you. The people who know you can still recognize you. Reticulum applies this principle to the network layer. Your Destination Hash is **invariant**. It travels with you, stored securely on your device, *immutable as a stone*.

This changes the relationship between you and the machine. You are not “logged into” the network via a specific gateway. You *are* the endpoint. The network does not connect to a place; *it converges on you*.

3.5.2 Roaming Nodes

This freedom introduces a new concept of time and space: **Nomadism**.

Because your identity is portable, your connectivity can be fluid. You can be sitting at a desk connected to a fiber backbone one moment, and walking through a field connected only to a long-range LoRa mesh the next. To the rest of

the network, nothing has changed. Your friends do not need to update your contact info. The messages they send do not bounce back. The network senses the shift in the medium and reroutes the flow of data automatically.

You are no longer a stationary node in a fixed grid. You are a wanderer in a fluid medium.

The interfaces - whether it is WiFi, Ethernet, Packet Radio, or a physical wire - is merely the clothing your node wears. You change it to suit the environment. Underneath, you remain the same. This is the liberation of the protocol. It treats the physical medium as a transient circumstance, not a definition of self.

Consider:

- **The Old Way:** *“I lost connection. I have to reconnect to the VPN to tell them where I am now.”*
- **The Zen Way:** *“I moved. The network subtly bends to accomodate this new reality.”*

3.5.3 Announcing Presence

How does the network find a wanderer? It listens.

In the IP world, we query directories. We ask a server, “Where is Mark?” The server checks its database and gives us a coordinate. This means that someone, somewhere, is keeping track of you. It assumes and *requires* surveillance.

Reticulum replaces surveillance with **Announces**.

Instead of asking a central authority where you are, you simply state your presence. You broadcast a cryptographic proof: “I am here, and I am who I say I am”. This ripples out through the mesh. Your neighbors hear it, update their path tables, and pass it on.

This is a quiet, organic process. It is the digital equivalent of lighting lanterns in the dark. You do not need to chase the light; you let the light find you. It respects your autonomy. You choose when to announce, how often to speak, and to whom. You also choose when to disappear - for but a moment or perpetually.

3.5.4 Anchor In The Flow

There is a deep peace in this nomadism. It teaches you that stability does not come from standing still. Stability comes from *internal coherence*.

By holding your own private key, you hold your own center of gravity. The world around you; the infrastructure, the topography and the availability of links can all shift chaotically. Storms can knock out towers. Cables can be cut. The internet can go down.

But as long as you possess your key, you possess your identity. The entire infrastructure can be destroyed and rebuilt, and you are still you. Nothing lasts, yet nothing is lost.

You become a sovereign entity moving through the noise, connected not by the rigidity of cables, but by the fluidity of recognition. The network becomes a place you inhabit, rather than a utility you subscribe to: You are at home in the ether.

3.6 Ethics Of The Tool

Technology With Conscience

You have unlearned the center. You have accepted the physics of trust. You have embraced the economy of scarcity and the freedom of unbound nomadism. You are standing in a new space. Now, look at the tool in your hand.

In the old world, we were taught that technology is neutral. We are told that “guns don’t kill people, people do”, or that a component is just a component, indifferent to what its combinatorial potential is. This is a convenient lie. It serves only to allow the builders to wash their hands of responsibility.

But we know better now. We know that **architecture is politics**, and *politics is control*. The way you build a system determines how it will be used. If you build a system optimized for mass surveillance, you *will* get a panopticon. If

you build a system optimized for centralized control, you *will* get a dictatorship. If you build a system optimized for extraction, you *will* get a parasite.

The Zen of Reticulum asserts that a tool is never neutral.

On the very contrary: A tool is intent, **crystallized**.

3.6.1 The Harm Principle

Why does the Reticulum License forbid the software from being used in systems designed to harm humans? Is it not just a restriction on freedom?

It is a restriction on *license*, yes, but it is an expansion of *freedom*.

Building powerful tools without a moral compass is in no way virtuous or commendable, it is plain and simple irresponsibility.

A tool that can easily be used to oppress is a real danger to the user. If you build a network that can be turned against you by a tyrant, you are not free. You are merely waiting for the leash to tighten. By encoding the “Harm Principle” into the legal DNA of the reference implementation, we are building a safeguard. We are stating, clearly and immutably, that *this tool* is for **life**, not for death.

This aligns the software with the interests of humanity. It cements that the network cannot be conscripted into a kill-system, a weaponized drone controller, or a torture device without breaking the license and the law. It is a line drawn in the sand - not by a government or external authority, but by the creators of the tool itself.

Consider:

- **The Old Way:** *“It’s just software. How people use it is not my problem.”*
- **The Zen Way:** *“This software is a habitat. I will not allow it to be used to build a cage.”*

It is *your* choice whether to align with this - we are not forcing this stance on anyone. If you choose to align with life over death, with creativity over destruction, we grant you an immensely powerful tool, to own and build with as you please. If you do not, we deny it.

If you do not like this, we most assuredly do not need you here, and you are on your own.

3.6.2 Public Domain Protocol

This leads to a vital distinction: The difference between the *idea* and the *implementation*.

The protocol - the mathematical rules of how Reticulum works - is dedicated to the Public Domain. It belongs to humanity. **No one can own it.** Anyone can implement it, improve it, or adapt it. This is the core idea of free communication, which itself must be forever free.

But the functional, deployed *reference implementation* - the Python code, the maintenance, the years of labor - has a conscience. This distinction is the engine of sustainability. It allows the protocol to be universal, while ensuring that the specific labor of the builders is not hijacked to undermine the foundational intent of the project itself. From this document, it should be very clear what this intent is.

If you want to build a system with Reticulum that manipulates and damages users for profits or targets missiles, you can use the public domain protocol, and start from scratch. But you cannot take our work. You must do your own. This serves as a pillar of accountability. If you want to build a weapon, *you* go and forge the steel yourself, while the world observes. And when the blood is drawn - it is on **your** hands.

3.6.3 Preserving Human Agency

We live in an era of predatory extraction. The open-source commons is being scraped, ingested, and regurgitated by machine learning algorithms, whose corporate owners seek to replace the very humans who built those commons. Our

code, our words, and our creativity is being used to train systems that are specifically designed to make us obsolete, without offering anything else in return than serfdom and leashes.

Reticulum stands against this.

The license protects the software from being used to feed the beast. It draws a hard line: This tool is for *people*. It is for human-to-human connection. It is not a dataset to be strip-mined for the purpose of building a synthetic overlord, puppeteered by a miniscule conglomerate of controllers.

This is a radical act of preservation. By protecting the code from AI appropriation, we are protecting space for human agency. We are ensuring that there remains a digital realm where the actors are flesh, blood and soul, where decisions are made by minds, not overlords hiding behind models.

When you use Reticulum, you are using a tool that respects you. It does not see you as a product to be tracked. It does not see your data as fuel for an algorithm. It sees you as a sovereign, equal peer.

This changes the foundational premise of using the technology. It restores dignity to the interaction. You are not the user of a service; you are a participant in a mutual covenant. The tool aligns with your autonomy, rather than eroding it.

In this way, ethics is not a restriction, but a foundation. It is the foundation that helps ensure the network will still belong to you tomorrow.

3.7 Design Patterns For Post-IP Systems

Practical Philosophy for Developers

The philosophy is useless if it cannot be hammered into code. The metaphors we have explored - nomadism, scarcity, trust - are not just poetry, but real-world engineering constraints. When you sit down to write software for Reticulum, these concepts must shape the very structure of your application.

We are now moving from the *why* to the *how*. This is where the abstract becomes concrete, and where you will see the true depth of the patterns we have been weaving.

3.7.1 Store & Forward

The web has trained us to be impatient. We write synchronous code. We fire a request and we wait, blocking the UI, holding our breath. If the response doesn't come in 250 milliseconds, we show a spinner. If it doesn't come in five seconds, we show an error. We treat network connectivity as a binary state: either we are "online" or we are "broken".

This is brittle. It is a rejection of reality.

In Reticulum, connectivity is a spectrum, and presence is asynchronous. If at all applicable to your intent, you must design your applications to embrace **Store & Forward**.

Instead of demanding an immediate answer, your application should act as a patient participant. You create a message for someone or something in the mesh. The network holds it. It carries it from node to node, perhaps over hours or days, waiting for the recipient to appear. When they finally surface, the message is delivered. This requires a shift from "request/response" to "event/handler". How exactly you do this is a challenge for you to solve intelligently within your problem domain, but Reticulum-based systems already exist that does this extremely well, and you can use them for inspiration.

Consider:

- **The Old Way:** `Connect()` -> `Send()` -> `Wait()` -> `Crash if timeout`.
- **The Zen Way:** `Send()` -> `Continue living`. -> `Receive()` when it arrives.

This changes the user experience profoundly. It removes the anxiety of the loading bar. It creates a sense of continuity. The user is not "waiting for the network"; they are interacting with a persistent log of communication that lives in the network itself.

3.7.2 Naming Is Power

In the IP world, we are slaves to the Domain Name System. We rely on a hierarchy of registrars to map human-readable names to machine-readable addresses. This hierarchy is a choke point. If the registrar revokes your domain, or if the DNS server goes down, you vanish.

Reticulum dissolves this hierarchy with **Hash-based Identity**.

In this design pattern, a name is not a string you look up; it is a cryptographic destination you verify. When you design for Reticulum, you stop asking the user for a URL and start asking for a Destination or Identity Hash.

This feels strange at first. A hash like `<83b7328926fed0d2e6a10a7671f9e237>` looks alien compared to `myfriend.com`. But that alienness is the armor. It **cannot** be spoofed. It **cannot** be censored by a registrar. It is **absolute**.

Designing for this means shifting your UI metaphors. You are no longer browsing a web of pages; you are managing a ledger of keys. You are building an “Address Book” that is actually a keyring. The names are given by the user, and the power stays with them. That hashes look complex is directly analogous to the strengths of the bonds formed by their use. It forces the user to engage in a moment of verification, an out-of-band handshake, which restores the human element of trust that SSL certificates stripped away.

3.7.3 The Interface Is The Medium

One of the most liberating patterns in Reticulum is **Transport Agnosticism**.

In traditional networking, your code is often littered with transport logic. “Am I on WiFi? Check bandwidth. Am I on Cellular? Check data plan. Am I on Ethernet?”. You are constantly micromanaging the pipe.

In Reticulum, you write to the API, and the API writes to the medium. You send a packet to a Destination. You do not care if that packet travels over a TCP tunnel, a LoRa radio wave, or a serial wire interface. That is the stack’s concern.

This allows you to write **Universal Applications**. Imagine a messaging app. You write it once. It works on a laptop connected to fiber. It works on a phone in the city using WiFi. And, without a single line of code changed, it works on a device in the wilderness, talking only to other devices via radio.

The pattern is simple: **Never code to the hardware. Code to the intent.**

Consider:

- **The Old Way:** `socket.connect(ip, port)`, and then a whole lot more
- **The Zen Way:** `RNS.Packet(destination, data).send()`

By abstracting the medium, you make your software immortal to changes in infrastructure. The user might switch from a 4G hotspot to a HF modem tomorrow. Your software doesn’t need to know. It simply continues the conversation.

3.7.4 Emergent Patterns

When you combine these patterns - *Store & Forward*, *Hash-based Identity*, and *Transport Agnosticism* - you create software that feels fundamentally different.

It feels *grounded*. It doesn’t flicker when the signal drops. It doesn’t panic when the server is down. It has weight. It has persistence. It has *relevance*.

You are no longer building a “client” that begs a “server” for attention. You are building an autonomous agent that exists within the mesh. It speaks when it needs to, listens when it can, and carries its identity with it wherever it goes.

This is the culmination of the Zen. The code is not just a set of instructions: It is a behavioral envelope. It is a way of *being* in the network.

3.8 Fabric Of The Independent

We have stripped away the illusions. We have seen that the center is empty, that trust *must* be hard, that resources are finite, and that we must own our infrastructure. We have seen that tools have ethics and that our identity can move fluidly.

This is a reclaiming of the commons. For too long, we have allowed the most vital substrate of human society - *our ability to speak to one another* - to be colonized by entities that do not share our interests. We have allowed the architecture of our communication to be designed by accountants rather than architects.

We are taking it back. Not by petitioning the masters, but by building the new world within, over, under and around the shell of the old.

3.8.1 The Work Is Finished

The heavy lifting is done.

The protocol is in the public domain, a gift to humanity that can never be taken away. The software is written, tested, and running on devices scattered across the globe. The manual lies open before you. The source code for the reference implementation is now distributed on hundreds of thousands of devices across the planet. No one can delete or destroy it. The hardware is accessible and abundant.

It was a hard road to get here, but we got here. Now, there is no roadmap committee waiting for approval. There is no venture capital dictating the user experience. There is no CEO to sign off on the next feature release.

There is only you.

The barrier to entry is no longer complexity: It is the mere habit of dependency. You were conditioned to wait. Wait for the app update. Wait for the ISP to fix the line. Wait for the platform to allow the post. Wait for the government to change the policies. Wait for the likes. Wait for the revolution to be televised.

The revolution never was televised.

It is packetized.

3.8.2 Open Sky

The future of this technology is a construction project.

It looks like a single node on a windowsill, listening to the static. It looks like a message sent to a neighbor, bypassing the noise of the commercial web. It looks like a community mesh that grows, link by link, hop by hop, carried by hands that care more about connection than profit.

You have the blueprints. You have the tools. You have the philosophy. The noise of the old world has fallen away, leaving you with the quiet clarity of the open spectrum.

Mark, early 2026

PROGRAMS USING RETICULUM

This chapter provides a non-exhaustive list of notable programs, systems and application-layer protocols that have been built using Reticulum.

These programs will let you get a feel for how Reticulum works. Most of them have been designed to run well even over slow networks based on LoRa or packet radio, but all can also be used over fast links, such as local WiFi, wired Ethernet, the Internet, or any combination.

As such, it is easy to get started experimenting, without having to set up any radio transceivers or infrastructure just to try it out. Launching the programs on separate devices connected to the same WiFi network is enough to get started, and physical radio interfaces can then be added later.

4.1 Programs & Utilities

Many different applications using Reticulum already exist, serving a wide variety of purposes from day-to-day communication and information sharing to systems administration and tackling advanced networking and communications challenges.

Development of Reticulum-based applications and systems is ongoing, so consider this list a non-exhaustive starting point of *some* of the options available. With a bit of searching, primarily over Reticulum itself, you will find many more interesting things.

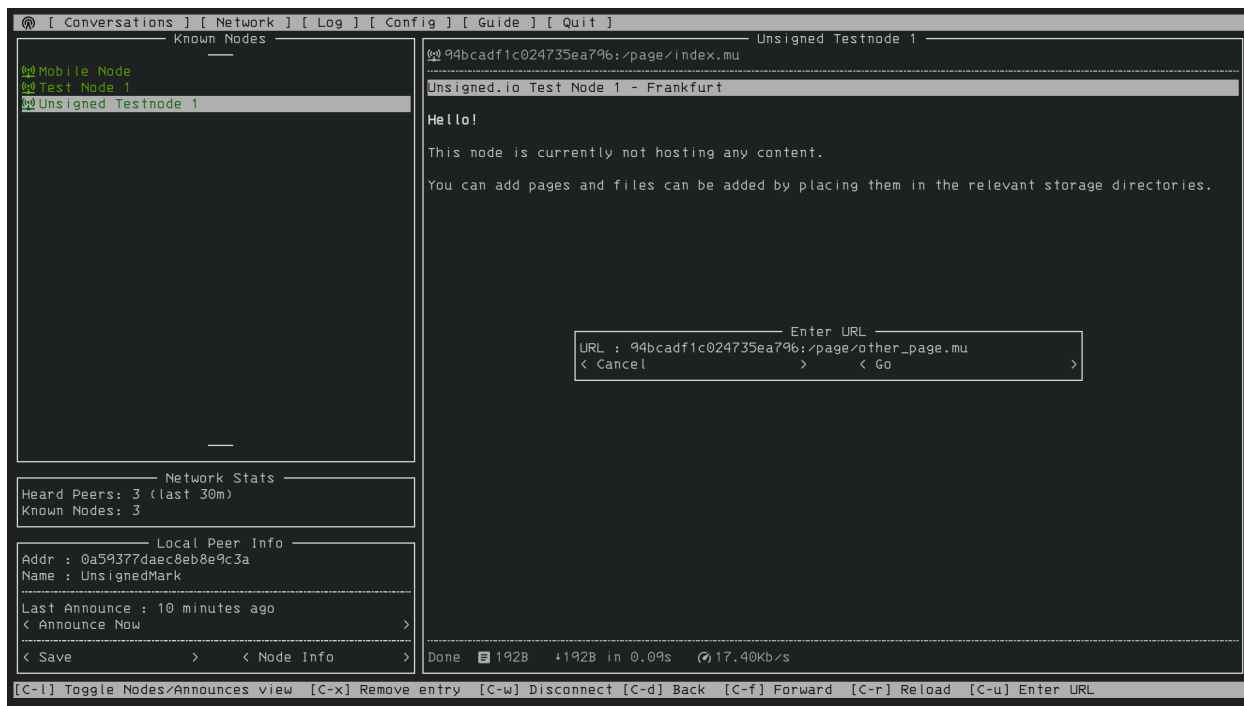
4.1.1 Remote Shell

The `rnsh` program lets you establish fully interactive remote shell sessions over Reticulum. It also allows you to pipe any program to or from a remote system, and is similar to how `ssh` works. The `rnsh` program is very efficient, and can facilitate fully interactive shell sessions, even over extremely low-bandwidth links, such as LoRa or packet radio.

In addition to the default, fully interactive terminal mode, for extremely limited links, `rnsh` offers line-interactive mode, allowing you to interact with remote systems, even when link throughput is counted in a few hundreds of bits per second.

4.1.2 Nomad Network

The terminal-based program [Nomad Network](#) provides a complete encrypted communications suite built with Reticulum. It features encrypted messaging (both direct and delayed-delivery for offline users), file sharing, and has a built-in text-browser and page server with support for dynamically rendered pages, user authentication and more.



[Nomad Network](#) is a user-facing client for the messaging and information-sharing protocol LXMf.

4.1.3 RNS Page Node

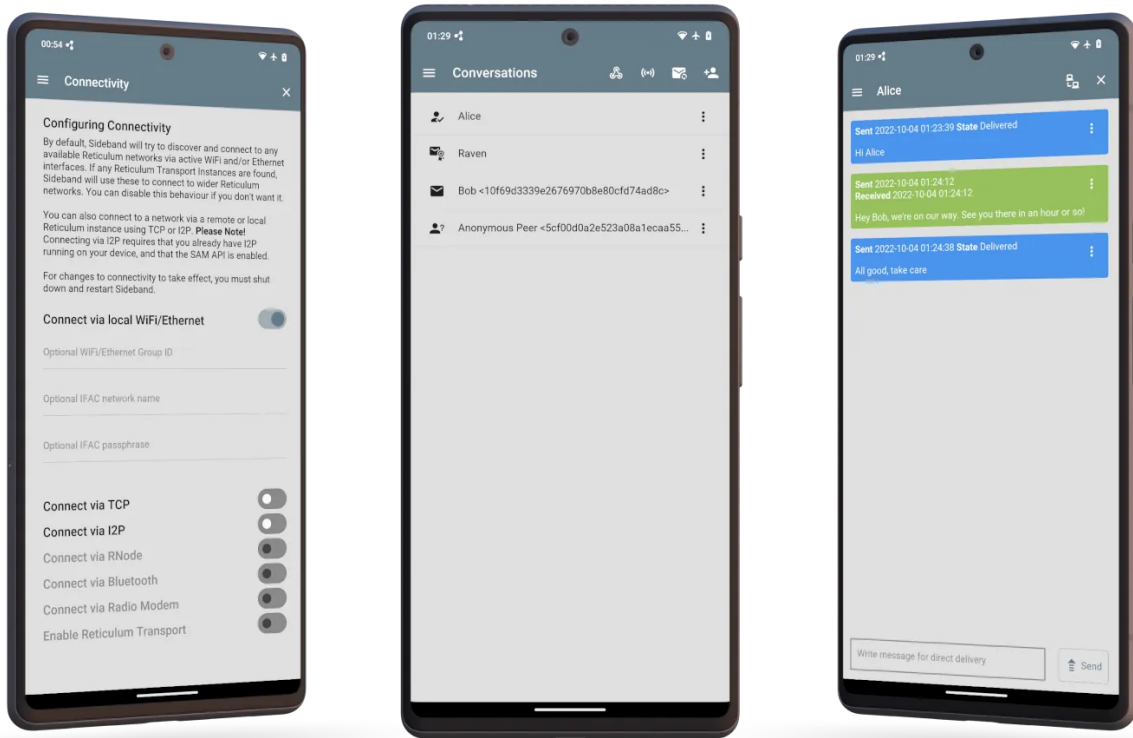
[RNS Page Node](#) is a simple way to serve pages and files to any other Nomad Network compatible client. Drop-in replacement for NomadNet nodes that primarily serve pages and files.

4.1.4 Retipedia

You can host the entirety of Wikipedia (or any `.zim`) file to other Nomad Network clients using [Retipedia](#).

4.1.5 Sideband

If you would rather use an LXMf client with a graphical user interface, you can take a look at [Sideband](#), which is available for Android, Linux, macOS and Windows. Sideband is an advanced LXMf and LXST client, and a multi-purpose Reticulum utility, with features and functionality targeted at advanced users.

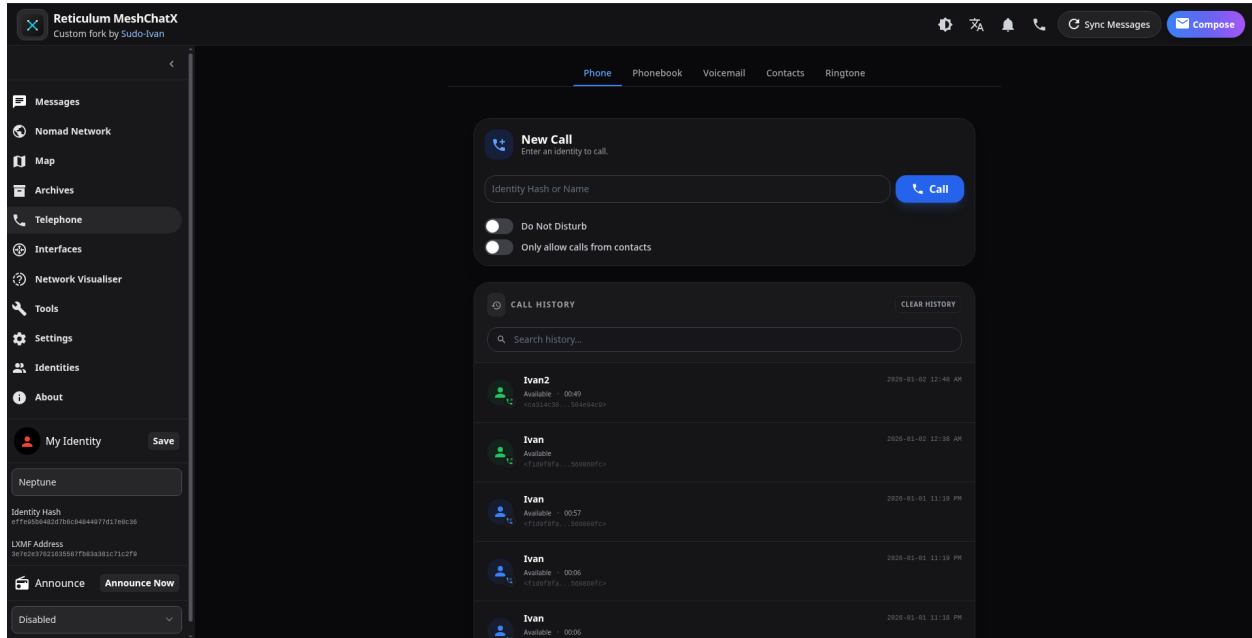


Sideband allows you to communicate with other people or LXMf-compatible systems over Reticulum networks using LoRa, Packet Radio, WiFi, I2P, Encrypted QR Paper Messages, or anything else Reticulum supports.

It also interoperates with all other LXMf clients, and provides advanced features such as voice messaging, real-time voice calls, file attachments, private telemetry sharing, and a full plugin system for expandability.

4.1.6 MeshChatX

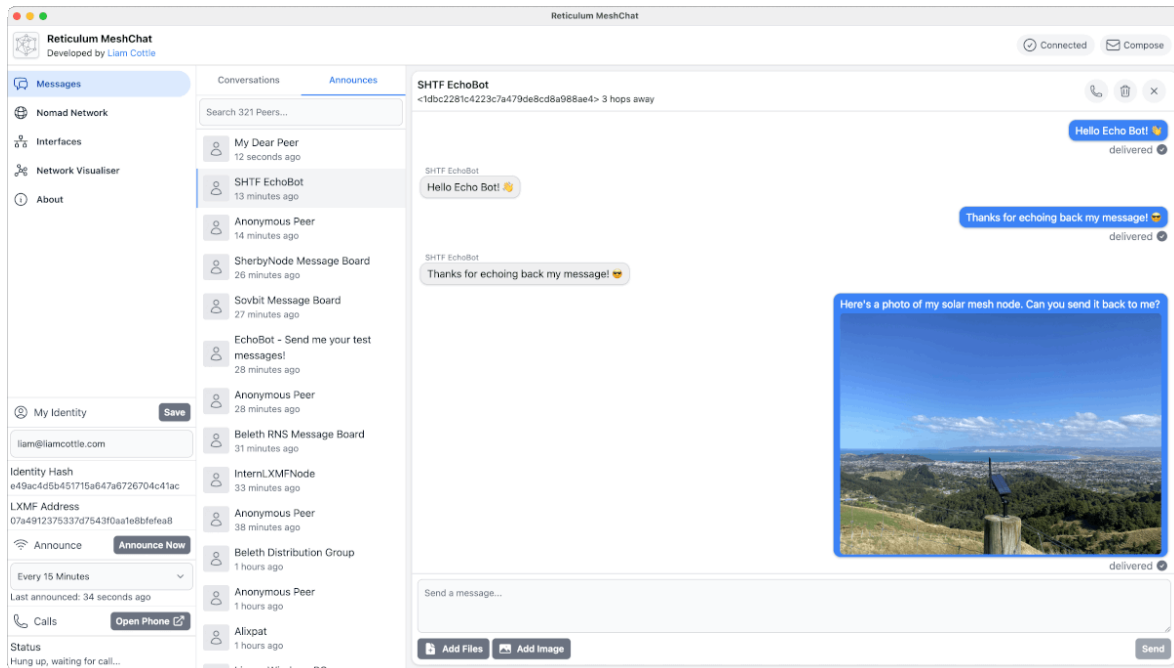
A Reticulum MeshChat fork from the future, with the goal of providing everything you need for Reticulum, LXMF, and LXST in one beautiful and feature-rich application. This project is separate from the original Reticulum MeshChat project, and is not affiliated with the original project.



Features include full LXST support, custom voicemail, phonebook, contact sharing, and ringtone support, multi-identity handling, modern UI/UX, offline documentation, expanded tools, page archiving, integrated maps, telemetry and improved application security.

4.1.7 MeshChat

The **Reticulum MeshChat** application is a user-friendly LXMF client for Linux, macOS and Windows, that also includes a Nomad Network page browser and other interesting functionality.



Reticulum MeshChat is of course also compatible with Sideband and Nomad Network, or any other LXMF client.

4.1.8 Columba

Columba is a simple and familiar LXMF messaging app Android, built with a native Android interface and Material Design 3.



While still in early and very active development, it is of course also compatible with all other LXMF clients, and allows you to message seamlessly with anyone else using LXMF.

4.1.9 Reticulum Relay Chat

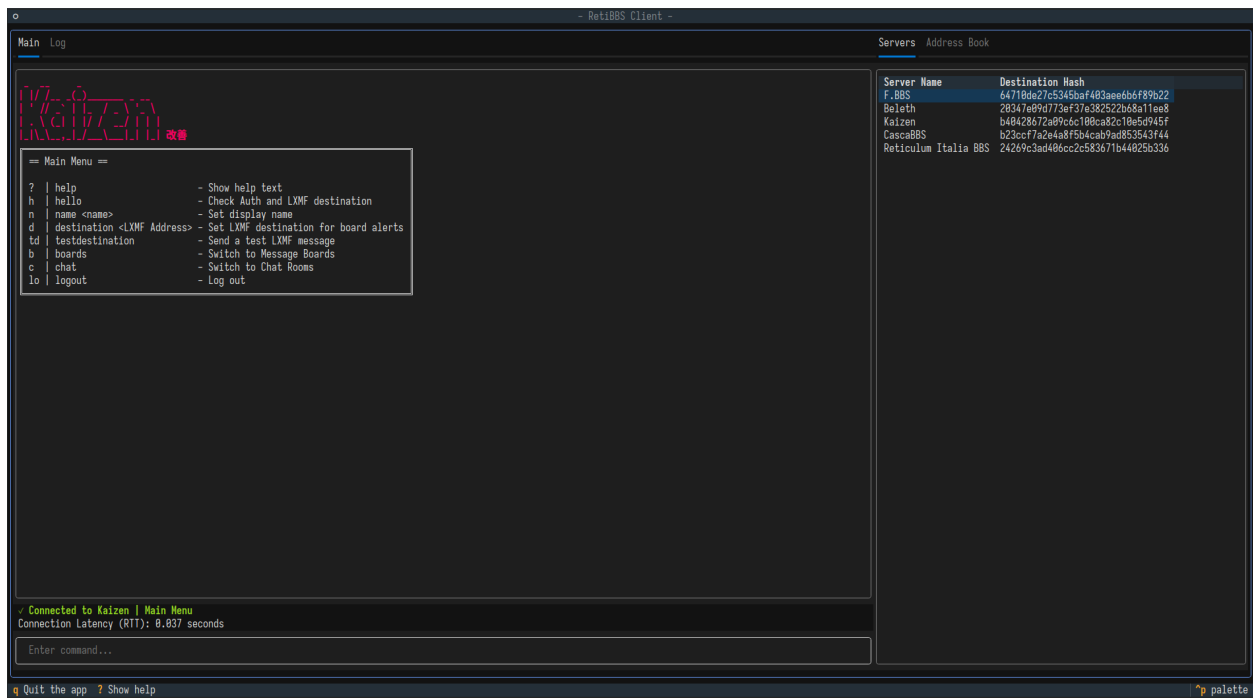
Reticulum Relay Chat is a live chat system built on top of the Reticulum Network Stack. It exists to let people talk to each other in real time over Reticulum without dragging in message databases, synchronization engines, or architectural commitments they did not ask for.

The `rrcd` program provides a functional, reference RRC hub-server daemon implementation. RRC user clients include `rrc-gui` and `rrc-web`.

RRC is closer in spirit to IRC than to modern “everything platforms.” You connect, you join a room, you talk, and then you leave. If you were present, you saw the conversation. If you were not, the conversation did not wait for you. This is not an accident. This is the entire design.

4.1.10 RetiBBS

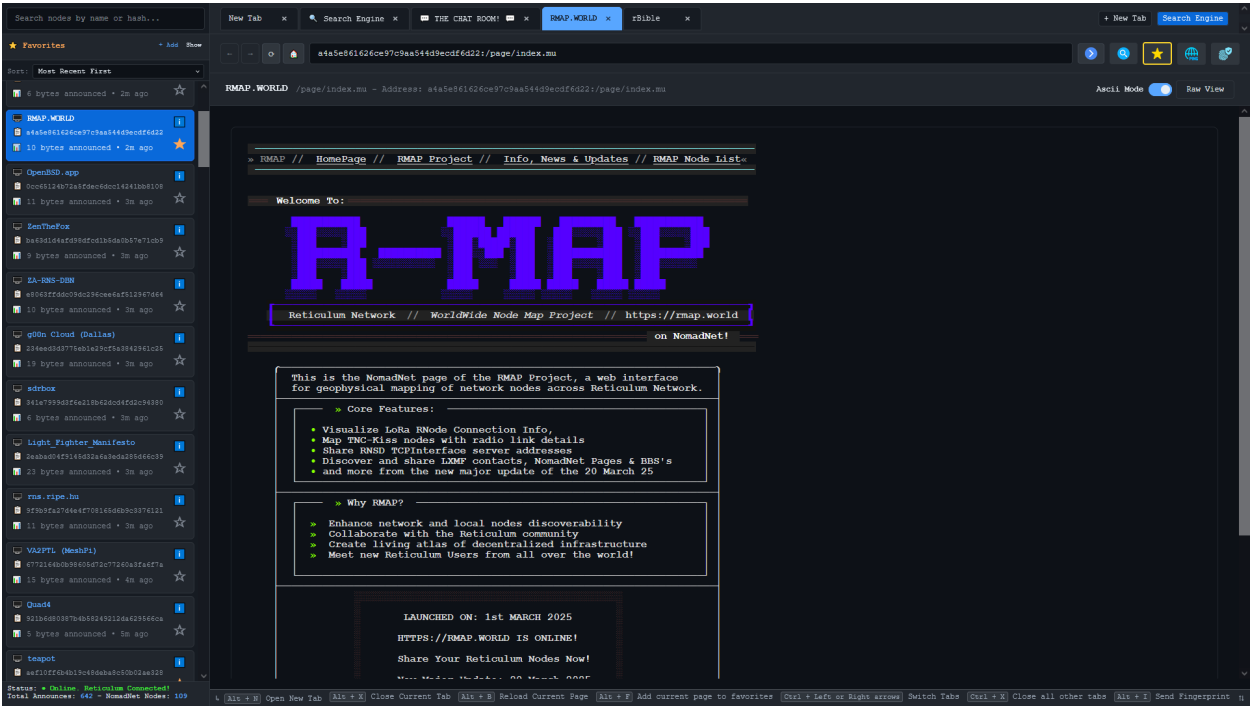
RetiBBS is a bulletin board system implementation for Reticulum networks.



RetiBBS allows users to communicate through message boards in a secure manner.

4.1.11 RBrowser

The **rBrowser** program is a cross-platform, standalone, web-based browser for exploring NomadNetwork Nodes over Reticulum Network. It automatically discovers NomadNet nodes through network announces and provides a user-friendly interface for browsing distributed content with Micron markup support.



Includes useful features like automatic listening for announce, adding nodes to favorites, browsing and rendering any kind of NomadNet links, downloading files from remote nodes, a unique local NomadNet Search Engine and more.

4.1.12 Reticulum Network Telephone

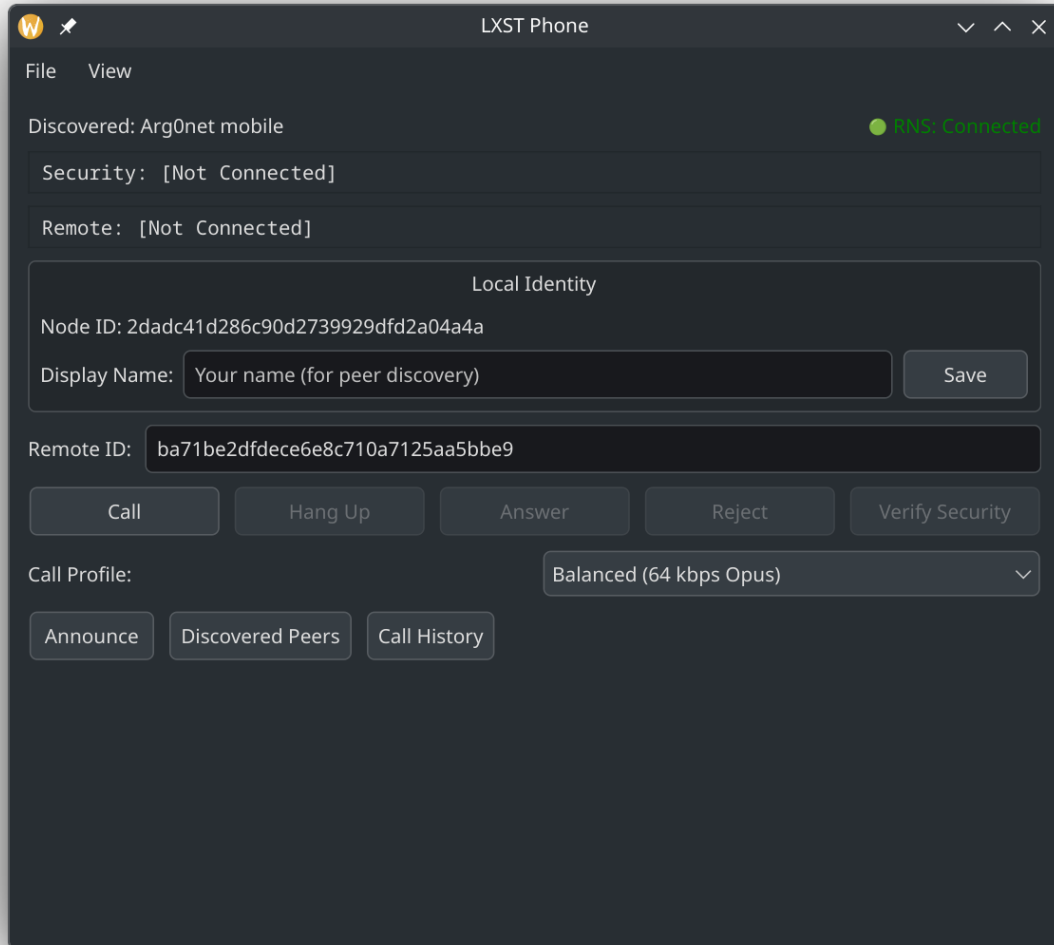
The `rnphone` program, included as part of the `LXST` package is a command-line Reticulum telephone utility and daemon, that allows building physical, hardware telephones for LXST and Reticulum, as well as simply performing calls via the command line.



It supports interfacing directly with hardware peripherals such as GPIO keypads and LCD displays, providing a modular system for building secure hardware telephones.

4.1.13 LXST Phone

The **LXST Phone** program is a cross-platform desktop application for performing LXST voice calls over Reticulum.



It supports various advanced features such as SAS verification, peer blocking, rate limiting, encrypted call history storage and contact management.

4.1.14 LXMFy

LXMFy is a comprehensive and advanced bot creation framework for LXMF, that allows building any kind of automation or bot system running over LXMF and Reticulum. [Bot implementations exist](#) for Home Assistant control, LLM integrations, and various other purposes.

4.1.15 LXMF Interactive Client

LXMF Interactive Client is a feature-rich, terminal-based LXMF messaging client with many advanced features and an extensible plugin architecture.

4.1.16 RNS FileSync

The **RNS FileSync** program enables automatic file synchronization between devices without requiring central servers, internet connectivity, or cloud services. It works over any network medium supported by Reticulum, including radio, LoRa, WiFi, or the internet, making it ideal for off-grid, privacy-focused, and resilient file sharing.

4.1.17 Micron Parser JS

Micron Parser JS is the JavaScript-based parser for the Micron markup language, that most web-based Nomad Network browsers use. If you want to make utilities or tools that display Micron pages, this library is essential.

4.1.18 RNMon

RNMon is a monitoring daemon designed to monitor the status of multiple RNS applications and push the metrics to an InfluxDB instance over the influx line protocol.

4.2 Protocols

A number of standard protocols have emerged through real-world usage and testing in the Reticulum community. While you may sometimes want to use completely custom protocols and implementations when writing Reticulum-based software, using these protocols provides application developers with an easy way to implement advanced functionality quickly and effortlessly. Using them also ensures compatibility and interoperability between many different client applications, creating an open communications ecosystem where users are free to choose the applications that suit their needs, while remaining connected to everyone else.

4.2.1 LXMF

LXMF is a simple and flexible messaging format and delivery protocol that allows a wide variety of applications, while using as little bandwidth as possible. It offers zero-conf message routing, end-to-end encryption and Forward Secrecy, and can be transported over any kind of medium that Reticulum supports.

LXMF is efficient enough that it can deliver messages over extremely low-bandwidth systems such as packet radio or LoRa. Encrypted LXMF messages can also be encoded as QR-codes or text-based URIs, allowing completely analog paper message transport.

Using Propagation Nodes, LXMF also offer a way to store and forward messages to users or endpoints that are not directly reachable at the time of message emission.

4.2.2 LXST

LXST is a simple and flexible real-time streaming format and delivery protocol that allows a wide variety of applications, while using as little bandwidth as possible. It is built on top of Reticulum and offers zero-conf stream routing, end-to-end encryption and Forward Secrecy, and can be transported over any kind of medium that Reticulum supports. It currently powers real-time voice and telephony applications over Reticulum.

4.2.3 RRC

The **Reticulum Relay Chat** protocol, is a live chat system built on top of the Reticulum Network Stack. It exists to provide near real-time group communication without dragging in message history databases, federation machinery, or architectural guilt.

RRC is intentionally simple. It does not pretend to be email, a mailbox, or a distributed archive. It behaves more like a conversation in a room. If you were there, you heard it. If you were not, you did not. That is not a bug, that is the point.

4.3 Interface Modules & Connectivity Resources

This section provides a list of various community-provided interface modules, guides and resources for creating Reticulum networks over special or challenging mediums.

- Custom interface module for running **RNS over HTTP**
- Guide for running **Reticulum over ICMP** using PipeInterface
- Guide for running **Reticulum over DNS** with Iodine
- Guide for running **Reticulum over HF radio**
- **Modem73** is a KISS TNC OFDM modem frontend that can be used with Reticulum

USING RETICULUM ON YOUR SYSTEM

Reticulum is not installed as a driver or kernel module, as one might expect of a networking stack. Instead, Reticulum is distributed as a Python module, containing the networking core, and a set of utility and daemon programs.

This means that no special privileges are required to install or use it. It is also very light-weight, and easy to transfer to, and install on new systems.

When you have Reticulum installed, any program or application that uses Reticulum will automatically load and initialise Reticulum when it starts, if it is not already running.

In many cases, this approach is sufficient. When any program needs to use Reticulum, it is loaded, initialised, interfaces are brought up, and the program can now communicate over any Reticulum networks available. If another program starts up and also wants access to the same Reticulum network, the already running instance is simply shared. This works for any number of programs running concurrently, and is very easy to use, but depending on your use case, there are other options.

5.1 Configuration & Data

Reticulum stores all information that it needs to function in a single file-system directory. When Reticulum is started, it will look for a valid configuration directory in the following places:

- `/etc/reticulum`
- `~/.config/reticulum`
- `~/.reticulum`

If no existing configuration directory is found, the directory `~/.reticulum` is created, and the default configuration will be automatically created here. You can move it to one of the other locations if you wish.

It is also possible to use completely arbitrary configuration directories by specifying the relevant command-line parameters when running Reticulum-based programs. You can also run multiple separate Reticulum instances on the same physical system, either in isolation from each other, or connected together.

In most cases, a single physical system will only need to run one Reticulum instance. This can either be launched at boot, as a system service, or simply be brought up when a program needs it. In either case, any number of programs running on the same system will automatically share the same Reticulum instance, if the configuration allows for it, which it does by default.

The entire configuration of Reticulum is found in the `~/.reticulum/config` file. When Reticulum is first started on a new system, a basic, but fully functional configuration file is created. The default configuration looks like this:

```
# This is the default Reticulum config file.  
# You should probably edit it to include any additional,  
# interfaces and settings you might need.
```

(continues on next page)

(continued from previous page)

```
# Only the most basic options are included in this default
# configuration. To see a more verbose, and much longer,
# configuration example, you can run the command:
# rnsd --exampleconfig
```

[reticulum]

```
# If you enable Transport, your system will route traffic
# for other peers, pass announces and serve path requests.
# This should be done for systems that are suited to act
# as transport nodes, ie. if they are stationary and
# always-on. This directive is optional and can be removed
# for brevity.
```

```
enable_transport = No
```

```
# By default, the first program to launch the Reticulum
# Network Stack will create a shared instance, that other
# programs can communicate with. Only the shared instance
# opens all the configured interfaces directly, and other
# local programs communicate with the shared instance over
# a local socket. This is completely transparent to the
# user, and should generally be turned on. This directive
# is optional and can be removed for brevity.
```

```
share_instance = Yes
```

```
# If you want to run multiple *different* shared instances
# on the same system, you will need to specify different
# instance names for each. On platforms supporting domain
# sockets, this can be done with the instance_name option:
```

```
instance_name = default
```

```
# Some platforms don't support domain sockets, and if that
# is the case, you can isolate different instances by
# specifying a unique set of ports for each:
```

```
# shared_instance_port = 37428
# instance_control_port = 37429
```

```
# If you want to explicitly use TCP for shared instance
# communication, instead of domain sockets, this is also
# possible, by using the following option:
```

```
# shared_instance_type = tcp
```

(continues on next page)

(continued from previous page)

```

# On systems where running instances may not have access
# to the same shared Reticulum configuration directory,
# it is still possible to allow full interactivity for
# running instances, by manually specifying a shared RPC
# key. In almost all cases, this option is not needed, but
# it can be useful on operating systems such as Android.
# The key must be specified as bytes in hexadecimal.

# rpc_key = e5c032d3ec4e64a6aca9927ba8ab73336780f6d71790

# It is possible to allow remote management of Reticulum
# systems using the various built-in utilities, such as
# rnstatus and rnpath. You will need to specify one or
# more Reticulum Identity hashes for authenticating the
# queries from client programs. For this purpose, you can
# use existing identity files, or generate new ones with
# the rnid utility.

# enable_remote_management = yes
# remote_management_allowed = 9fb6d773498fb3feda407ed8ef2c3229,
↳ 2d882c5586e548d79b5af27bca1776dc

# You can configure Reticulum to panic and forcibly close
# if an unrecoverable interface error occurs, such as the
# hardware device for an interface disappearing. This is
# an optional directive, and can be left out for brevity.
# This behaviour is disabled by default.

# panic_on_interface_error = No

# When Transport is enabled, it is possible to allow the
# Transport Instance to respond to probe requests from
# the rnprobe utility. This can be a useful tool to test
# connectivity. When this option is enabled, the probe
# destination will be generated from the Identity of the
# Transport Instance, and printed to the log at startup.
# Optional, and disabled by default.

# respond_to_probes = No

[logging]
# Valid log levels are 0 through 7:
# 0: Log only critical information
# 1: Log errors and lower log levels
# 2: Log warnings and lower log levels
# 3: Log notices and lower log levels
# 4: Log info and lower (this is the default)
# 5: Verbose logging

```

(continues on next page)

(continued from previous page)

```
# 6: Debug logging
# 7: Extreme logging

loglevel = 4

# The interfaces section defines the physical and virtual
# interfaces Reticulum will use to communicate on. This
# section will contain examples for a variety of interface
# types. You can modify these or use them as a basis for
# your own config, or simply remove the unused ones.

[interfaces]

# This interface enables communication with other
# link-local Reticulum nodes over UDP. It does not
# need any functional IP infrastructure like routers
# or DHCP servers, but will require that at least link-
# local IPv6 is enabled in your operating system, which
# should be enabled by default in almost any OS. See
# the Reticulum Manual for more configuration options.

[[Default Interface]]
    type = AutoInterface
    interface_enabled = True
```

If Reticulum infrastructure already exists locally, you probably don't need to change anything, and you may already be connected to a wider network. If not, you will probably need to add relevant *interfaces* to the configuration, in order to communicate with other systems.

You can generate a much more verbose configuration example by running the command:

```
rnsd --exampleconfig
```

The output includes examples for most interface types supported by Reticulum, along with additional options and configuration parameters.

It is a good idea to read the comments and explanations in the above default config. It will teach you the basic concepts you need to understand to configure your network. Once you have done that, take a look at the *Interfaces* chapter of this manual.

5.2 Included Utility Programs

Reticulum includes a range of useful utilities, both for managing your Reticulum networks, and for carrying out common tasks over Reticulum networks, such as transferring files to remote systems, and executing commands and programs remotely.

If you often use Reticulum from several different programs, or simply want Reticulum to stay available all the time, for example if you are hosting a transport node, you might want to run Reticulum as a separate service that other programs, applications and services can utilise.

5.2.1 The rnsd Utility

It is very easy to run Reticulum as a service. Simply run the included `rnsd` command. When `rnsd` is running, it will keep all configured interfaces open, handle transport if it is enabled, and allow any other programs to immediately utilise the Reticulum network it is configured for.

You can even run multiple instances of `rnsd` with different configurations on the same system.

Usage Examples

Run `rnsd`:

```
$ rnsd

[2023-08-18 17:59:56] [Notice] Started rnsd version 0.5.8
```

Run `rnsd` in service mode, ensuring all logging output is sent directly to file:

```
$ rnsd -s
```

Generate a verbose and detailed configuration example, with explanations of all the various configuration options, and interface configuration examples:

```
$ rnsd --exampleconfig
```

All Command-Line Options

```
usage: rnsd.py [-h] [--config CONFIG] [-v] [-q] [-s] [--exampleconfig] [--version]

Reticulum Network Stack Daemon

options:
  -h, --help            show this help message and exit
  --config CONFIG       path to alternative Reticulum config directory
  -v, --verbose
  -q, --quiet
  -s, --service         rnsd is running as a service and should log to file
  -i, --interactive     drop into interactive shell after initialisation
  --exampleconfig       print verbose configuration example to stdout and exit
  --version             show program's version number and exit
```

You can easily add `rnsd` as an always-on service by *configuring a service*.

5.2.2 The rnstatus Utility

Using the `rnstatus` utility, you can view the status of configured Reticulum interfaces, similar to the `ifconfig` program.

Usage Examples

Run `rnstatus`:

```
$ rnstatus

Shared Instance[37428]
  Status  : Up
  Serving : 1 program
```

(continues on next page)

(continued from previous page)

```

Rate      : 1.00 Gbps
Traffic   : 83.13 KB↑
           86.10 KB↓

AutoInterface[Local]
  Status   : Up
  Mode     : Full
  Rate     : 10.00 Mbps
  Peers    : 1 reachable
  Traffic  : 63.23 KB↑
           80.17 KB↓

TCPInterface[RNS Testnet Dublin/dublin.connect.reticulum.network:4965]
  Status   : Up
  Mode     : Full
  Rate     : 10.00 Mbps
  Traffic  : 187.27 KB↑
           74.17 KB↓

RNodeInterface[RNode UHF]
  Status   : Up
  Mode     : Access Point
  Rate     : 1.30 kbps
  Access   : 64-bit IFAC by <...e702c42ba8>
  Traffic  : 8.49 KB↑
           9.23 KB↓

Reticulum Transport Instance <5245a8efe1788c6a1cd36144a270e13b> running

```

Filter output to only show some interfaces:

```

$ rnstatus rnode

RNodeInterface[RNode UHF]
  Status   : Up
  Mode     : Access Point
  Rate     : 1.30 kbps
  Access   : 64-bit IFAC by <...e702c42ba8>
  Traffic  : 8.49 KB↑
           9.23 KB↓

Reticulum Transport Instance <5245a8efe1788c6a1cd36144a270e13b> running

```

All Command-Line Options

```

usage: rnstatus [-h] [--config CONFIG] [--version] [-a] [-A]
               [-l] [-t] [-s SORT] [-r] [-j] [-R hash] [-i path]
               [-w seconds] [-d] [-D] [-m] [-I seconds] [-v] [filter]

Reticulum Network Stack Status

positional arguments:

```

(continues on next page)

(continued from previous page)

<code>filter</code>	only display interfaces with names including <code>filter</code>
options:	
<code>-h, --help</code>	show this help message and exit
<code>--config CONFIG</code>	path to alternative Reticulum config directory
<code>--version</code>	show program's version number and exit
<code>-a, --all</code>	show all interfaces
<code>-A, --announce-stats</code>	show announce stats
<code>-l, --link-stats</code>	show link stats
<code>-t, --totals</code>	display traffic totals
<code>-s, --sort SORT</code>	sort interfaces by [rate, traffic, rx, tx, rxs, txs, announces, arx, atx, held]
<code>-r, --reverse</code>	reverse sorting
<code>-j, --json</code>	output in JSON format
<code>-R hash</code>	transport identity hash of remote instance to get status from
<code>-i path</code>	path to identity used for remote management
<code>-w seconds</code>	timeout before giving up on remote queries
<code>-d, --discovered</code>	list discovered interfaces
<code>-D</code>	show details and config entries for discovered interfaces
<code>-m, --monitor</code>	continuously monitor status
<code>-I, --monitor-interval seconds</code>	refresh interval for monitor mode (default: 1)
<code>-v, --verbose</code>	

Note

When using `-R` to query a remote transport instance, you must also specify `-i` with the path to a management identity file that is authorized for remote management on the target system.

5.2.3 The `rnid` Utility

With the `rnid` utility, you can generate, manage and view Reticulum Identities. The program can also calculate Destination hashes, and perform encryption and decryption of files.

Using `rnid`, it is possible to asymmetrically encrypt files and information for any Reticulum destination hash, and also to create and verify cryptographic signatures.

Usage Examples

Generate a new Identity:

```
$ rnid -g ./new_identity
```

Display Identity key information:

```
$ rnid -i ./new_identity -p
```

```
Loaded Identity <984b74a3f768bef236af4371e6f248cd> from new_id
Public Key   : 0f4259fef4521ab75a3409e353fe9073eb10783b4912a6a9937c57bf44a62c1e
Private Key  : Hidden
```

Encrypt a file for an LXM user:

```
$ rnid -i 8dd57a738226809646089335a6b03695 -e my_file.txt

Recalled Identity <bc7291552be7a58f361522990465165c> for destination
↳<8dd57a738226809646089335a6b03695>
Encrypting my_file.txt
File my_file.txt encrypted for <bc7291552be7a58f361522990465165c> to my_file.txt.rfe
```

If the Identity for the destination is not already known, you can fetch it from the network by using the `-R` command-line option:

```
$ rnid -R -i 30602def3b3506a28ed33db6f60cc6c9 -e my_file.txt

Requesting unknown Identity for <30602def3b3506a28ed33db6f60cc6c9>...
Received Identity <2b489d06eaf7c543808c76a5332a447d> for destination
↳<30602def3b3506a28ed33db6f60cc6c9> from the network
Encrypting my_file.txt
File my_file.txt encrypted for <2b489d06eaf7c543808c76a5332a447d> to my_file.txt.rfe
```

Decrypt a file using the Reticulum Identity it was encrypted for:

```
$ rnid -i ./my_identity -d my_file.txt.rfe

Loaded Identity <2225fdeecaf6e2db4556c3c2d7637294> from ./my_identity
Decrypting ./my_file.txt.rfe...
File ./my_file.txt.rfe decrypted with <2225fdeecaf6e2db4556c3c2d7637294> to ./my_file.txt
```

All Command-Line Options

```
usage: rnid.py [-h] [--config path] [-i identity] [-g path] [-v] [-q] [-a aspects]
               [-H aspects] [-e path] [-d path] [-s path] [-V path] [-r path] [-w path]
               [-f] [-R] [-t seconds] [-p] [-P] [--version]
```

Reticulum Identity & Encryption Utility

options:

```
-h, --help                show this help message and exit
--config path             path to alternative Reticulum config directory
-i, --identity identity   hexadecimal Reticulum identity or destination hash, or path to
↳Identity file
-g, --generate file       generate a new Identity
-m, --import identity_data
                           import Reticulum identity in hex, base32 or base64 format
-x, --export              export identity to hex, base32 or base64 format
-v, --verbose             increase verbosity
-q, --quiet               decrease verbosity
-a, --announce aspects   announce a destination based on this Identity
-H, --hash aspects        show destination hashes for other aspects for this Identity
-e, --encrypt file        encrypt file
-d, --decrypt file        decrypt file
-s, --sign path           sign file
-V, --validate path       validate signature
```

(continues on next page)

(continued from previous page)

```

-r, --read file      input file path
-w, --write file     output file path
-f, --force          write output even if it overwrites existing files
-R, --request        request unknown Identities from the network
-t seconds           identity request timeout before giving up
-p, --print-identity print identity info and exit
-P, --print-private  allow displaying private keys
-b, --base64         Use base64-encoded input and output
-B, --base32         Use base32-encoded input and output
--version            show program's version number and exit

```

5.2.4 The rnpaht Utility

With the rnpaht utility, you can look up and view paths for destinations on the Reticulum network.

Usage Examples

Resolve path to a destination:

```
$ rnpaht c89b4da064bf66d280f0e4d8abfd9806
```

```

Path found, destination <c89b4da064bf66d280f0e4d8abfd9806> is 4 hops away via
↳<f53a1c4278e0726bb73fcc623d6ce763> on TCPInterface[Testnet/dublin.connect.reticulum.
↳network:4965]

```

All Command-Line Options

```

usage: rnpaht [-h] [--config CONFIG] [--version] [-t] [-m hops] [-r] [-d] [-D]
              [-x] [-w seconds] [-R hash] [-i path] [-W seconds] [-b] [-B] [-U]
              [--duration DURATION] [--reason REASON] [-p] [-j] [-v]
              [destination] [list_filter]

```

Reticulum Path Management Utility

positional arguments:

```

destination      hexadecimal hash of the destination
list_filter       filter for remote blackhole list view

```

options:

```

-h, --help          show this help message and exit
--config CONFIG     path to alternative Reticulum config directory
--version           show program's version number and exit
-t, --table         show all known paths
-m, --max hops      maximum hops to filter path table by
-r, --rates         show announce rate info
-d, --drop          remove the path to a destination
-D, --drop-announces drop all queued announces
-x, --drop-via      drop all paths via specified transport instance
-w seconds          timeout before giving up
-R hash            transport identity hash of remote instance to manage
-i path            path to identity used for remote management
-W seconds          timeout before giving up on remote queries
-b, --blackholed    list blackholed identities

```

(continues on next page)

(continued from previous page)

```

-B, --blackhole      blackhole identity
-U, --unblackhole    unblackhole identity
--duration DURATION  duration of blackhole enforcement in hours
--reason REASON      reason for blackholing identity
-p, --blackholed-list
                    view published blackhole list for remote transport instance
-j, --json           output in JSON format
-v, --verbose

```

5.2.5 The `rnprobe` Utility

The `rnprobe` utility lets you probe a destination for connectivity, similar to the `ping` program. Please note that probes will only be answered if the specified destination is configured to send proofs for received packets. Many destinations will not have this option enabled, so most destinations will not be probable.

You can enable a probe-reply destination on Reticulum Transport Instances by setting the `respond_to_probes` configuration directive. Reticulum will then print the probe destination to the log on Transport Instance startup.

Usage Examples

Probe a destination:

```

$ rnprobe rnstransport.probe 2d03725b327348980d570f739a3a5708

Sent 16 byte probe to <2d03725b327348980d570f739a3a5708>
Valid reply received from <2d03725b327348980d570f739a3a5708>
Round-trip time is 38.469 milliseconds over 2 hops

```

Send a larger probe:

```

$ rnprobe rnstransport.probe 2d03725b327348980d570f739a3a5708 -s 256

Sent 16 byte probe to <2d03725b327348980d570f739a3a5708>
Valid reply received from <2d03725b327348980d570f739a3a5708>
Round-trip time is 38.781 milliseconds over 2 hops

```

If the interface that receives the probe replies supports reporting radio parameters such as **RSSI** and **SNR**, the `rnprobe` utility will print these as part of the result as well.

```

$ rnprobe rnstransport.probe e7536ee90bd4a440e130490b87a25124

Sent 16 byte probe to <e7536ee90bd4a440e130490b87a25124>
Valid reply received from <e7536ee90bd4a440e130490b87a25124>
Round-trip time is 1.809 seconds over 1 hop [RSSI -73 dBm] [SNR 12.0 dB]

```

All Command-Line Options

```

usage: rnprobe [-h] [--config CONFIG] [-s SIZE] [-n PROBES]
              [-t seconds] [-w seconds] [--version] [-v]
              [full_name] [destination_hash]

```

Reticulum Probe Utility

positional arguments:

(continues on next page)

(continued from previous page)

full_name	full destination name in dotted notation
destination_hash	hexadecimal hash of the destination
options:	
-h, --help	show this help message and exit
--config CONFIG	path to alternative Reticulum config directory
-s SIZE, --size SIZE	size of probe packet payload in bytes
-n PROBES, --probes PROBES	number of probes to send
-t seconds, --timeout seconds	timeout before giving up
-w seconds, --wait seconds	time between each probe
--version	show program's version number and exit
-v, --verbose	

5.2.6 The rncp Utility

The rncp utility is a simple file transfer tool. Using it, you can transfer files through Reticulum.

Usage Examples

Run rncp on the receiving system, specifying which identities are allowed to send files:

```
$ rncp --listen -a 1726dbad538775b5bf9b0ea25a4079c8 -a c50cc4e4f7838b6c31f60ab9032cbc62
```

You can also specify allowed identity hashes (one per line) in the file ~/.rncp/allowed_identities and simply running the program in listener mode:

```
$ rncp --listen
```

From another system, copy a file to the receiving system:

```
$ rncp ~/path/to/file.tgz 73cbd378bb0286ed11a707c13447bb1e
```

Or fetch a file from the remote system:

```
$ rncp --fetch ~/path/to/file.tgz 73cbd378bb0286ed11a707c13447bb1e
```

The default identity file is stored in ~/.reticulum/identities/rncp, but you can use another one, which will be created if it does not already exist

```
$ rncp ~/path/to/file.tgz 73cbd378bb0286ed11a707c13447bb1e -i /path/to/identity
```

All Command-Line Options

```
usage: rncp [-h] [--config path] [-v] [-q] [-S] [-l] [-F] [-f]
           [-j path] [-b seconds] [-a allowed_hash] [-n] [-p]
           [-i identity] [-w seconds] [--version] [file] [destination]
```

Reticulum File Transfer Utility

positional arguments:

```
file          file to be transferred
```

(continues on next page)

(continued from previous page)

destination	hexadecimal hash of the receiver
options:	
-h, --help	show this help message and exit
--config path	path to alternative Reticulum config directory
-v, --verbose	increase verbosity
-q, --quiet	decrease verbosity
-S, --silent	disable transfer progress output
-l, --listen	listen for incoming transfer requests
-C, --no-compress	disable automatic compression
-F, --allow-fetch	allow authenticated clients to fetch files
-f, --fetch	fetch file from remote listener instead of sending
-j, --jail path	restrict fetch requests to specified path
-s, --save path	save received files in specified path
-O, --overwrite	Allow overwriting received files, instead of adding postfix
-b seconds	announce interval, 0 to only announce at startup
-a allowed_hash	allow this identity (or add in ~/.rncp/allowed_identities)
-n, --no-auth	accept requests from anyone
-p, --print-identity	print identity and destination info and exit
-i identity	path to identity to use
-w seconds	sender timeout before giving up
-P, --phy-rates	display physical layer transfer rates
--version	show program's version number and exit

5.2.7 The rnx Utility

The `rnx` utility is a basic remote command execution program. It allows you to execute commands on remote systems over Reticulum, and to view returned command output. For a fully interactive remote shell solution, be sure to also take a look at the `rnsh` program.

Usage Examples

Run `rnx` on the listening system, specifying which identities are allowed to execute commands:

```
$ rnx --listen -a 941bed5e228775e5a8079fc38b1ccf3f -a 1b03013c25f1c2ca068a4f080b844a10
```

From another system, run a command on the remote:

```
$ rnx 7a55144adf826958a9529a3bcf08b149 "cat /proc/cpuinfo"
```

Or enter the interactive mode pseudo-shell:

```
$ rnx 7a55144adf826958a9529a3bcf08b149 -x
```

The default identity file is stored in `~/.reticulum/identities/rnx`, but you can use another one, which will be created if it does not already exist

```
$ rnx 7a55144adf826958a9529a3bcf08b149 -i /path/to/identity -x
```

All Command-Line Options

```
usage: rnx [-h] [--config path] [-v] [-q] [-p] [-l] [-i identity] [-x] [-b] [-n] [-N]
          [-d] [-m] [-a allowed_hash] [-w seconds] [-W seconds] [--stdin STDIN]
          [--stdout STDOUT] [--stderr STDERR] [--version] [destination] [command]
```

(continues on next page)

(continued from previous page)

Reticulum Remote Execution Utility

positional arguments:

destination	hexadecimal hash of the listener
command	command to be execute

optional arguments:

-h, --help	show this help message and exit
--config path	path to alternative Reticulum config directory
-v, --verbose	increase verbosity
-q, --quiet	decrease verbosity
-p, --print-identity	print identity and destination info and exit
-l, --listen	listen for incoming commands
-i identity	path to identity to use
-x, --interactive	enter interactive mode
-b, --no-announce	don't announce at program start
-a allowed_hash	accept from this identity
-n, --noauth	accept files from anyone
-N, --noid	don't identify to listener
-d, --detailed	show detailed result output
-m	mirror exit code of remote command
-w seconds	connect and request timeout before giving up
-W seconds	max result download time
--stdin STDIN	pass input to stdin
--stdout STDOUT	max size in bytes of returned stdout
--stderr STDERR	max size in bytes of returned stderr
--version	show program's version number and exit

5.2.8 The rnodeconf Utility

The `rnodeconf` utility allows you to inspect and configure existing *RNodes*, and to create and provision new *RNodes* from any supported hardware devices.

All Command-Line Options

```
usage: rnodeconf [-h] [-i] [-a] [-u] [-U] [--fw-version version]
                [--fw-url url] [--nocheck] [-e] [-E] [-C]
                [--baud-flash baud_flash] [-N] [-T] [-b] [-B] [-p] [-D i]
                [--display-addr byte] [--freq Hz] [--bw Hz] [--txp dBm]
                [--sf factor] [--cr rate] [--eeprom-backup] [--eeprom-dump]
                [--eeprom-wipe] [-P] [--trust-key hexbytes] [--version] [-f]
                [-r] [-k] [-S] [-H FIRMWARE_HASH] [--platform platform]
                [--product product] [--model model] [--hwrev revision]
                [port]
```

RNode Configuration and firmware utility. This program allows you to change various settings and startup modes of RNode. It can also install, flash and update the firmware on supported devices.

positional arguments:

port	serial port where RNode is attached
------	-------------------------------------

(continues on next page)

(continued from previous page)

options:

```

-h, --help          show this help message and exit
-i, --info          Show device info
-a, --autoinstall   Automatic installation on various supported devices
-u, --update        Update firmware to the latest version
-U, --force-update  Update to specified firmware even if version matches or is older
↳ than installed version
--fw-version version Use a specific firmware version for update or autoinstall
--fw-url url        Use an alternate firmware download URL
--nocheck          Don't check for firmware updates online
-e, --extract       Extract firmware from connected RNode for later use
-E, --use-extracted Use the extracted firmware for autoinstallation or update
-C, --clear-cache   Clear locally cached firmware files
--baud-flash baud_flash
                    Set specific baud rate when flashing device. Default is 921600
-N, --normal        Switch device to normal mode
-T, --tnc           Switch device to TNC mode
-b, --bluetooth-on  Turn device bluetooth on
-B, --bluetooth-off Turn device bluetooth off
-p, --bluetooth-pair Put device into bluetooth pairing mode
-D, --display i     Set display intensity (0-255)
-t, --timeout s     Set display timeout in seconds, 0 to disable
-R, --rotation rotation
                    Set display rotation, valid values are 0 through 3
--display-addr byte Set display address as hex byte (00 - FF)
--recondition-display
                    Start display reconditioning
--np i             Set NeoPixel intensity (0-255)
--freq Hz          Frequency in Hz for TNC mode
--bw Hz            Bandwidth in Hz for TNC mode
--txp dBm          TX power in dBm for TNC mode
--sf factor         Spreading factor for TNC mode (7 - 12)
--cr rate          Coding rate for TNC mode (5 - 8)
-x, --ia-enable     Enable interference avoidance
-X, --ia-disable    Disable interference avoidance
-c, --config        Print device configuration
--eeprom-backup     Backup EEPROM to file
--eeprom-dump       Dump EEPROM to console
--eeprom-wipe       Unlock and wipe EEPROM
-P, --public        Display public part of signing key
--trust-key hexbytes Public key to trust for device verification
--version          Print program version and exit
-f, --flash         Flash firmware and bootstrap EEPROM
-r, --rom           Bootstrap EEPROM without flashing firmware
-k, --key           Generate a new signing key and exit
-S, --sign          Display public part of signing key
-H, --firmware-hash FIRMWARE_HASH
                    Set installed firmware hash
--platform platform Platform specification for device bootstrap
--product product   Product specification for device bootstrap
--model model       Model code for device bootstrap

```

(continues on next page)

(continued from previous page)

```
--hwrev revision      Hardware revision for device bootstrap
```

For more information on how to create your own RNodes, please read the [Creating RNodes](#) section of this manual.

5.3 Discovering Interfaces

Reticulum includes built-in functionality for discovering connectable interfaces over Reticulum itself. This is particularly useful in situations where you want to do one or more of the following:

- Discover connectable entrypoints available on the Internet
- Find connectable radio access points in the physical world
- Maintain connectivity to RNS instances with unknown or changing IP addresses

Discovered interfaces can be **auto-connected** by Reticulum, which makes it possible to create setups where an arbitrary interface can act simply as a bootstrap connection, that can be torn down again once more suitable interfaces have been discovered and connected.

The interface discovery mechanism uses announces sent over Reticulum itself, and supports both publicly readable interfaces and private, encrypted discovery, that can only be decoded by specified *network identities*. It is also possible to specify which network identities should be considered valid sources for discovered interfaces, so that interfaces published by unknown entities are ignored.

Note

A *network identity* is a normal Reticulum identity keyset that can be used by one or more transport nodes to identify them as belonging to the same overall network. In the context of interface discovery, this makes it easy to manage connecting to only the particular networks you care about, even if those networks utilize many individual physical transport node.

This also makes it convenient to auto-connect discovered interfaces only for networks you have some level of trust in.

For information on how to make your interfaces discoverable, see the [Discoverable Interfaces](#) chapter of this manual. The current section will focus on how to actually *discover and connect to* interfaces available on the network.

In its most basic form, enabling interface discovery is as simple as setting `discover_interfaces` to `true` in your Reticulum config:

```
[reticulum]
...
discover_interfaces = yes
...
```

Once this option is enabled, your RNS instance will start listening for interface discovery announces, and store them for later use or inspection. You can list discovered interfaces with the `rnstatus` utility:

```
$ rnstatus -d
```

Name	Type	Status	Last Heard	Value	Location
Sideband Hub	Backbone	✓ Available	1h ago	16	46.2316, 6.0536
RNS Amsterdam	Backbone	✓ Available	32m ago	16	52.3865, 4.9037

You can view more detailed information about discovered interfaces, including configuration snippets for pasting directly into your [interfaces] config, by using the `rnstatus -D` option:

```
$ rnstatus -D sideband

Transport ID : 521c87a83afb8f29e4455e77930b973b
Name        : Sideband Hub
Type        : BackboneInterface
Status      : Available
Transport   : Enabled
Distance    : 2 hops
Discovered  : 9h and 40m ago
Last Heard  : 1h and 15m ago
Location    : 46.2316, 6.0536
Address     : sideband.connect.reticulum.network:7822
Stamp Value : 16

Configuration Entry:
[[Sideband Hub]]
  type = BackboneInterface
  enabled = yes
  remote = sideband.connect.reticulum.network
  target_port = 7822
  transport_identity = 521c87a83afb8f29e4455e77930b973b
```

In addition to providing local interface discovery information and control, the `rnstatus` utility can export discovered interface data in machine-readable JSON format using the `rnstatus -d --json` option. This can be useful for exporting the data to external applications such as status pages, access point maps and similar.

To control what sources are considered valid for discovered sources, additional configuration options can be specified for the interface discovery system.

- The `interface_discovery_sources` option is a list of the network or transport identities from which interfaces will be accepted. If this option is set, all others will be ignored. If this option is not set, discovered interfaces will be accepted from any source, but are still subject to stamp value requirements.
- The `required_discovery_value` options specifies the minimum stamp value required for the interface announce to be considered valid. To make it computationally difficult to spam the network with a large number of defunct or malicious interfaces, each announced interface requires a valid cryptographical stamp, of configurable difficulty value.
- The `autoconnect_discovered_interfaces` value defaults to 0, and specifies the maximum number of discovered interfaces that should be auto-connected at any given time. If set to a number greater than 0, Reticulum automatically manages discovered interface connections, and will bring discovered interfaces up and down based on availability. You can at any time add discovered interfaces to your configuration manually, to persistently keep them available.
- The `network_identity` option specifies the *network identity* for this RNS instance. This identity is used both to sign (and potentially encrypt) *outgoing* interface discovery announces, and to decrypt incoming discovery information.

The configuration snippet below contains an example of setting these additional configuration options:

```
[reticulum]
...
discover_interfaces = yes
interface_discovery_sources = 521c87a83afb8f29e4455e77930b973b
```

(continues on next page)

(continued from previous page)

```
required_discovery_value = 16
autoconnect_discovered_interfaces = 3
network_identity = ~/.reticulum/storage/identities/my_network
...
```

5.4 Remote Management

It is possible to allow remote management of Reticulum systems using the various built-in utilities, such as `rnstatus` and `rnpath`. To do so, you will need to set the `enable_remote_management` directive in the `[reticulum]` section of the configuration file. You will also need to specify one or more Reticulum Identity hashes for authenticating the queries from client programs. For this purpose, you can use existing identity files, or generate new ones with the `rnid` utility.

The following is a truncated example of enabling remote management in the Reticulum configuration file:

```
[reticulum]
...
enable_remote_management = yes
remote_management_allowed = 9fb6d773498fb3feda407ed8ef2c3229,
↪ 2d882c5586e548d79b5af27bca1776dc
...
```

For a complete example configuration, you can run `rnscd --exampleconfig`.

5.5 Blackhole Management

Reticulum networks are fundamentally permissionless and open, allowing anyone with a compatible interface to participate. While this openness is essential for a resilient and decentralized network, it also exposes the network to potential abuse, such as peers flooding the network with excessive announce broadcasts or other forms of resource exhaustion.

The **Blackhole** system provides tools to help manage this problem. It allows operators and individual users to block specific identities at the Transport layer, preventing them from propagating announces through your node, and for other nodes to reach them through your network.

Important

There is fundamentally **no way** to *globally* block or censor any identity or destination in Reticulum networks. The blackhole functionality will prevent announces from (and traffic to) all destinations associated with the blackholed identity *on your own network segments only*.

This provides users and operators with control over what they want to allow *on their own network segments*, but there is no way to globally censor or remove an identity, as long as *someone* is willing to provide transport for it.

This functionality serves a dual purpose:

- **For Individual Users:** It offers a simple way to maintain a quiet and efficient local network by manually blocking spammy or unwanted peers.
- **For Network Operators:** It enables the creation of federated, community-wide security standards. By publishing and sharing blackhole lists, operators can protect large infrastructures and distribute spam filtering rules across the mesh without manual intervention.

5.5.1 Local Blackhole Management

The most immediate way to manage unwanted identities is through manual configuration using the `rnpath` utility. This allows you to instantly block or unblock specific identities on your local Transport Instance.

Blackholing an Identity

To block an identity, use the `-B` (or `--blackhole`) flag followed by the identity hash. You can optionally specify a duration and a reason, which are useful for logging and future reference.

```
$ rnpath -B 3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o
```

You can also add a duration (in hours) and a reason:

```
$ rnpath -B 3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o --duration 24 --reason "Excessive announces"
```

Lifting Blackholes

To remove an identity from the blackhole, use the `-U` (or `--unblackhole`) flag:

```
$ rnpath -U 3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o
```

Viewing the Blackhole List

To see all identities currently blackholed on your local instance, use the `-b` (or `--blackholed`) flag:

```
$ rnpath -b

<3a4f8b9c1d2e3f4g5h6i7j8k9l0m1n2o> blackholed for 23h, 56m (Excessive announces)
<399ea050ce0eed1816c300bcb0840938> blackholed indefinitely (Announce spam)
<d56a4fa02c0a77b3575935aedd90bdb2> blackholed indefinitely (Announce spam)
<2b9ec651326d9bc274119054c70fb75e> blackholed indefinitely (Announce spam)
<1178a8f1fad405bf2ad153bf5036bdfd> blackholed indefinitely (Announce spam)
```

5.5.2 Automated List Sourcing

Manually blocking identities is effective for immediate threats, but maintaining an up-to-date blocklist for a large network is impractical. Reticulum supports **automated list sourcing**, allowing your node to subscribe to blackhole lists maintained by trusted peers, or a central authority you manage yourself.

Warning

Verify Before Subscribing! Subscribing to a blackhole source is a powerful action that grants that source the ability to dictate who you can communicate with. Before adding a source to your configuration, verify that the maintainer aligns with your usage policy and values. Blindly subscribing to untrusted lists could inadvertently block legitimate peers or essential services.

When enabled, your Transport Instance will periodically (approximately once per hour) connect to configured sources, retrieve their latest blackhole lists, and automatically merge them into your local blocklist. This provides “set-and-forget” protection for both individual users and large networks.

Configuration

To enable automated sourcing, add the `blackhole_sources` option to the `[reticulum]` section of your configuration file. This option accepts a comma-separated list of Transport Identity hashes that you trust to provide valid blackhole lists.

```
[reticulum]
...
# Automatically fetch blackhole lists from these trusted sources
blackhole_sources = 521c87a83afb8f29e4455e77930b973b, 68a4aa91ac350c4087564e8a69f84e86
...
```

How It Works

1. When enabled, the `BlackholeUpdater` service runs in the background.
2. For every identity hash listed in `blackhole_sources`, it attempts to establish a temporary link to its associated `rnstransport.info.blackhole` destination.
3. It requests the `/list` path, which returns a dictionary of blackholed identities and their associated metadata.
4. The received list is merged with your local `blackholed_identities` database.
5. The lists are persisted to disk, ensuring they survive restarts.

Note

You can verify the external lists you are subscribed to, and their contents, without importing them by using `rnpath -p`. See the [rnpath utility documentation](#) for details on querying remote blackhole lists.

5.5.3 Publishing Blackhole Lists

If you are operating a public gateway, a community hub, or simply wish to share your blackhole list with others, you can configure your instance to act as a blackhole list publisher. This allows other nodes to subscribe to *your* definitions of unwanted traffic.

Enabling Publishing

To publish your local blackhole list, enable the `publish_blackhole` option in the `[reticulum]` section:

```
[reticulum]
...
publish_blackhole = yes
...
```

When this is enabled, your Transport Instance will register a request handler at `rnstransport.info.blackhole`. Any peer that connects to this destination and requests `/list` will receive the complete set of identities currently present in your local blackhole database.

Federation and Trust

The blackhole system relies on the trust relationship between the subscriber and the publisher. By subscribing to a source, you are implicitly trusting that source to only block identities that are genuinely detrimental to the network.

As the ecosystem matures, this system is designed to integrate with **Network Identities**. This allows communities to verify that a published blackhole list is actually provided by a specific network or organization with a certain level of reputation and trustworthiness, adding a layer of cryptographic trust to the federation process. This prevents malicious actors from publishing fake lists intended to censor legitimate traffic.

For operators, this creates a scalable model where maintaining a single high-quality blocklist can protect thousands of downstream peers, drastically reducing the administrative.

5.6 Improving System Configuration

If you are setting up a system for permanent use with Reticulum, there is a few system configuration changes that can make this easier to administrate. These changes will be detailed here.

5.6.1 Fixed Serial Port Names

On a Reticulum instance with several serial port based interfaces, it can be beneficial to use the fixed device names for the serial ports, instead of the dynamically allocated shorthands such as `/dev/ttyUSB0`. Under most Debian-based distributions, including Ubuntu and Raspberry Pi OS, these nodes can be found under `/dev/serial/by-id`.

You can use such a device path directly in place of the numbered shorthands. Here is an example of a packet radio TNC configured as such:

```
[[Packet Radio KISS Interface]]
  type = KISSInterface
  interface_enabled = True
  outgoing = true
  port = /dev/serial/by-id/usb-FTDI_FT230X_Basic_UART_43891CKM-if00-port0
  speed = 115200
  databits = 8
  parity = none
  stopbits = 1
  preamble = 150
  txtail = 10
  persistence = 200
  slottime = 20
```

Using this methodology avoids potential naming mix-ups where physical devices might be plugged and unplugged in different orders, or when device name assignment varies from one boot to another.

5.6.2 Reticulum as a System Service

Instead of starting Reticulum manually, you can install `rnsd` as a system service and have it start automatically at boot.

Systemwide Service

If you installed Reticulum with `pip`, the `rnsd` program will most likely be located in a user-local installation path only, which means `systemd` will not be able to execute it. In this case, you can simply symlink the `rnsd` program into a directory that is in `systemd`'s path:

```
sudo ln -s $(which rnsd) /usr/local/bin/
```

You can then create the service file `/etc/systemd/system/rnsd.service` with the following content:

```
[Unit]
Description=Reticulum Network Stack Daemon
After=multi-user.target

[Service]
# If you run Reticulum on WiFi devices,
# or other devices that need some extra
# time to initialise, you might want to
# add a short delay before Reticulum is
# started by systemd:
```

(continues on next page)

(continued from previous page)

```
# ExecStartPre=/bin/sleep 10
Type=simple
Restart=always
RestartSec=3
User=USERNAMEHERE
ExecStart=rnsd --service

[Install]
WantedBy=multi-user.target
```

Be sure to replace USERNAMEHERE with the user you want to run rnsd as.

To manually start rnsd run:

```
sudo systemctl start rnsd
```

If you want to automatically start rnsd at boot, run:

```
sudo systemctl enable rnsd
```

Userspace Service

Alternatively you can use a user systemd service instead of a system wide one. This way the whole setup can be done as a regular user. Create a user systemd service files `~/.config/systemd/user/rnsd.service` with the following content:

```
[Unit]
Description=Reticulum Network Stack Daemon
After=default.target

[Service]
# If you run Reticulum on WiFi devices,
# or other devices that need some extra
# time to initialise, you might want to
# add a short delay before Reticulum is
# started by systemd:
# ExecStartPre=/bin/sleep 10
Type=simple
Restart=always
RestartSec=3
ExecStart=RNS_BIN_DIR/rnsd --service

[Install]
WantedBy=default.target
```

Replace RNS_BIN_DIR with the path to your Reticulum binary directory (eg. `/home/USERNAMEHERE/rns/bin`).

Start user service:

```
systemctl --user daemon-reload
systemctl --user start rnsd.service
```

If you want to automatically start rnsd without having to log in as the USERNAMEHERE, do:

```
sudo loginctl enable-linger USERNAMEHERE  
systemctl --user enable rnsd.service
```

UNDERSTANDING RETICULUM

This chapter will briefly describe the overall purpose and operating principles of Reticulum. It should give you an overview of how the stack works, and an understanding of how to develop networked applications using Reticulum.

This chapter is not an exhaustive source of information on Reticulum, at least not yet. Currently, the only complete repository, and final authority on how Reticulum actually functions, is the Python reference implementation and API reference. That being said, this chapter is an essential resource in understanding how Reticulum works from a high-level perspective, along with the general principles of Reticulum, and how to apply them when creating your own networks or software.

After reading this chapter, you should be well-equipped to understand how a Reticulum network operates, what it can achieve, and how you can use it yourself. This chapter also seeks to provide an overview of the sentiments and the philosophy behind Reticulum, what problems it seeks to solve, and how it approaches those solutions.

6.1 Motivation

The primary motivation for designing and implementing Reticulum has been the current lack of reliable, functional and secure minimal-infrastructure modes of digital communication. It is my belief that it is highly desirable to create a reliable and efficient way to set up long-range digital communication networks that can securely allow exchange of information between people and machines, with no central point of authority, control, censorship or barrier to entry.

Almost all of the various networking systems in use today share a common limitation: They require large amounts of coordination and centralised trust and power to function. To join such networks, you need approval of gatekeepers in control. This need for coordination and trust inevitably leads to an environment of central control, where it's very easy for infrastructure operators or governments to control or alter traffic, and censor or persecute unwanted actors. It also makes it completely impossible to freely deploy and use networks at will, like one would use other common tools that enhance individual agency and freedom.

Reticulum aims to require as little coordination and trust as possible. It aims to make secure, anonymous and permissionless networking and information exchange a tool that anyone can just pick up and use.

Since Reticulum is completely medium agnostic, it can be used to build networks on whatever is best suited to the situation, or whatever you have available. In some cases, this might be packet radio links over VHF frequencies, in other cases it might be a 2.4 GHz network using off-the-shelf radios, or it might be using common LoRa development boards.

At the time of release of this document, the fastest and easiest setup for development and testing is using LoRa radio modules with an open source firmware (see the section [Reference Setup](#)), connected to any kind of computer or mobile device that Reticulum can run on.

The ultimate aim of Reticulum is to allow anyone to be their own network operator, and to make it cheap and easy to cover vast areas with a myriad of independent, interconnectable and autonomous networks. Reticulum **is not one network**, it **is a tool** to build *thousands of networks*. Networks without kill-switches, surveillance, censorship and control. Networks that can freely interoperate, associate and disassociate with each other, and require no central oversight. Networks for human beings. *Networks for the people.*

6.2 Goals

To be as widely usable and efficient to deploy as possible, the following goals have been used to guide the design of Reticulum:

- **Fully useable as open source software stack**
Reticulum must be implemented with, and be able to run using only open source software. This is critical to ensuring the availability, security and transparency of the system.
- **Hardware layer agnosticism**
Reticulum must be fully hardware agnostic, and shall be useable over a wide range of physical networking layers, such as data radios, serial lines, modems, handheld transceivers, wired Ethernet, WiFi, or anything else that can carry a digital data stream. Hardware made for dedicated Reticulum use shall be as cheap as possible and use off-the-shelf components, so it can be easily modified and replicated by anyone interested in doing so.
- **Very low bandwidth requirements**
Reticulum should be able to function reliably over links with a transmission capacity as low as *5 bits per second*.
- **Encryption by default**
Reticulum must use strong encryption by default for all communication.
- **Initiator Anonymity**
It must be possible to communicate over a Reticulum network without revealing any identifying information about oneself.
- **Unlicensed use**
Reticulum shall be functional over physical communication mediums that do not require any form of license to use. Reticulum must be designed in a way, so it is usable over ISM radio frequency bands, and can provide functional long distance links in such conditions, for example by connecting a modem to a PMR or CB radio, or by using LoRa or WiFi modules.
- **Supplied software**
In addition to the core networking stack and API, that allows a developer to build applications with Reticulum, a basic set of Reticulum-based communication tools must be implemented and released along with Reticulum itself. These shall serve both as a functional, basic communication suite, and as an example and learning resource to others wishing to build applications with Reticulum.
- **Ease of use**
The reference implementation of Reticulum is written in Python, to make it easy to use and understand. A programmer with only basic experience should be able to use Reticulum to write networked applications.
- **Low cost**
It shall be as cheap as possible to deploy a communication system based on Reticulum. This should be achieved by using cheap off-the-shelf hardware that potential users might already own. The cost of setting up a functioning node should be less than \$100 even if all parts needs to be purchased.

6.3 Introduction & Basic Functionality

Reticulum is a networking stack suited for high-latency, low-bandwidth links. Reticulum is at its core a *message oriented* system. It is suited for both local point-to-point or point-to-multipoint scenarios where all nodes are within range of each other, as well as scenarios where packets need to be transported over multiple hops in a complex network to reach the recipient.

Reticulum does away with the idea of addresses and ports known from IP, TCP and UDP. Instead Reticulum uses the singular concept of *destinations*. Any application using Reticulum as its networking stack will need to create one or more destinations to receive data, and know the destinations it needs to send data to.

All destinations in Reticulum are *represented* as a 16 byte hash. This hash is derived from truncating a full SHA-256 hash of identifying characteristics of the destination. To users, the destination addresses will be displayed as 16 hexadecimal bytes, like this example: <13425ec15b621c1d928589718000d814>.

The truncation size of 16 bytes (128 bits) for destinations has been chosen as a reasonable trade-off between address space and packet overhead. The address space accommodated by this size can support many billions of simultaneously active devices on the same network, while keeping packet overhead low, which is essential on low-bandwidth networks. In the very unlikely case that this address space nears congestion, a one-line code change can upgrade the Reticulum address space all the way up to 256 bits, ensuring the Reticulum address space could potentially support galactic-scale networks. This is obviously complete and ridiculous over-allocation, and as such, the current 128 bits should be sufficient, even far into the future.

By default Reticulum encrypts all data using elliptic curve cryptography and AES. Any packet sent to a destination is encrypted with a per-packet derived key. Reticulum can also set up an encrypted channel to a destination, called a *Link*. Both data sent over Links and single packets offer *Initiator Anonymity*. Links additionally offer *Forward Secrecy* by default, employing an Elliptic Curve Diffie Hellman key exchange on Curve25519 to derive per-link ephemeral keys. Asymmetric, link-less packet communication can also provide forward secrecy, with automatic key ratcheting, by enabling ratchets on a per-destination basis. The multi-hop transport, coordination, verification and reliability layers are fully autonomous and also based on elliptic curve cryptography.

Reticulum also offers symmetric key encryption for group-oriented communications, as well as unencrypted packets (for local broadcast purposes **only**).

Reticulum can connect to a variety of interfaces such as radio modems, data radios and serial ports, and offers the possibility to easily tunnel Reticulum traffic over IP links such as the Internet or private IP networks.

6.3.1 Destinations

To receive and send data with the Reticulum stack, an application needs to create one or more destinations. Reticulum uses three different basic destination types, and one special:

- **Single**

The *single* destination type is the most common type in Reticulum, and should be used for most purposes. It is always identified by a unique public key. Any data sent to this destination will be encrypted using ephemeral keys derived from an ECDH key exchange, and will only be readable by the creator of the destination, who holds the corresponding private key.

- **Plain**

A *plain* destination type is unencrypted, and suited for traffic that should be broadcast to a number of users, or should be readable by anyone. Traffic to a *plain* destination is not encrypted. Generally, *plain* destinations can be used for broadcast information intended to be public. Plain destinations are only reachable directly, and packets addressed to plain destinations are never transported over multiple hops in the network. To be transportable over multiple hops in Reticulum, information *must* be encrypted, since Reticulum uses the per-packet encryption to verify routing paths and keep them alive.

- **Group**

The *group* special destination type, that defines a symmetrically encrypted virtual destination. Data sent to this destination will be encrypted with a symmetric key, and will be readable by anyone in possession of the key, but as with the *plain* destination type, packets to this type of destination are not currently transported over multiple hops, although a planned upgrade to Reticulum will allow globally reachable *group* destinations.

- **Link**

A *link* is a special destination type, that serves as an abstract channel to a *single* destination, directly connected or over multiple hops. The *link* also offers reliability and more efficient encryption, forward secrecy, initiator anonymity, and as such can be useful even when a node is directly reachable. It also offers a more capable API and allows easily carrying out requests and responses, large data transfers and more.

Destination Naming

Destinations are created and named in an easy to understand dotted notation of *aspects*, and represented on the network as a hash of this value. The hash is a SHA-256 truncated to 128 bits. The top level aspect should always be a unique identifier for the application using the destination. The next levels of aspects can be defined in any way by the creator of the application.

Aspects can be as long and as plentiful as required, and a resulting long destination name will not impact efficiency, as names are always represented as truncated SHA-256 hashes on the network.

As an example, a destination for an environmental monitoring application could be made up of the application name, a device type and measurement type, like this:

```
app name  : environmentlogger
aspects   : remotesensor, temperature

full name : environmentlogger.remotesensor.temperature
hash      : 4faf1b2e0a077e6a9d92fa051f256038
```

For the *single* destination, Reticulum will automatically append the associated public key as a destination aspect before hashing. This is done to ensure only the correct destination is reached, since anyone can listen to any destination name. Appending the public key ensures that a given packet is only directed at the destination that holds the corresponding private key to decrypt the packet.

Take note! There is a very important concept to understand here:

- Anyone can use the destination name `environmentlogger.remotesensor.temperature`
- Each destination that does so will still have a unique destination hash, and thus be uniquely addressable, because their public keys will differ.

In actual use of *single* destination naming, it is advisable not to use any uniquely identifying features in aspect naming. Aspect names should be general terms describing what kind of destination is represented. The uniquely identifying aspect is always achieved by appending the public key, which expands the destination into a uniquely identifiable one. Reticulum does this automatically.

Any destination on a Reticulum network can be addressed and reached simply by knowing its destination hash (and public key, but if the public key is not known, it can be requested from the network simply by knowing the destination hash). The use of app names and aspects makes it easy to structure Reticulum programs and makes it possible to filter what information and data your program receives.

To recap, the different destination types should be used in the following situations:

- **Single**
When private communication between two endpoints is needed. Supports multiple hops.
- **Group**
When private communication between two or more endpoints is needed. Supports multiple hops indirectly, but must first be established through a *single* destination.
- **Plain**
When plain-text communication is desirable, for example when broadcasting information, or for local discovery purposes.

To communicate with a *single* destination, you need to know its public key. Any method for obtaining the public key is valid, but Reticulum includes a simple mechanism for making other nodes aware of your destinations public key, called the *announce*. It is also possible to request an unknown public key from the network, as all transport instances serve as a distributed ledger of public keys.

Note that public key information can be shared and verified in other ways than using the built-in *announce* functionality, and that it is therefore not required to use the *announce* and *path request* functionality to obtain public keys. It is by

far the easiest though, and should definitely be used if there is not a very good reason for doing it differently.

6.3.2 Public Key Announcements

An *announce* will send a special packet over any relevant interfaces, containing all needed information about the destination hash and public key, and can also contain some additional, application specific data. The entire packet is signed by the sender to ensure authenticity. It is not required to use the announce functionality, but in many cases it will be the simplest way to share public keys on the network. The announce mechanism also serves to establish end-to-end connectivity to the announced destination, as the announce propagates through the network.

As an example, an announce in a simple messenger application might contain the following information:

- The announcers destination hash
- The announcers public key
- Application specific data, in this case the users nickname and availability status
- A random blob, making each new announce unique
- An Ed25519 signature of the above information, verifying authenticity

With this information, any Reticulum node that receives it will be able to reconstruct an outgoing destination to securely communicate with that destination. You might have noticed that there is one piece of information lacking to reconstruct full knowledge of the announced destination, and that is the aspect names of the destination. These are intentionally left out to save bandwidth, since they will be implicit in almost all cases. The receiving application will already know them. If a destination name is not entirely implicit, information can be included in the application specific data part that will allow the receiver to infer the naming.

It is important to note that announces will be forwarded throughout the network according to a certain pattern. This will be detailed in the section *The Announce Mechanism in Detail*.

In Reticulum, destinations are allowed to move around the network at will. This is very different from protocols such as IP, where an address is always expected to stay within the network segment it was assigned in. This limitation does not exist in Reticulum, and any destination is *completely portable* over the entire topography of the network, and *can even be moved to other Reticulum networks* than the one it was created in, and still become reachable. To update its reachability, a destination simply needs to send an announce on any networks it is part of. After a short while, it will be globally reachable in the network.

Seeing how *single* destinations are always tied to a private/public key pair leads us to the next topic.

6.3.3 Identities

In Reticulum, an *identity* does not necessarily represent a personal identity, but is an abstraction that can represent any kind of *verifiable entity*. This could very well be a person, but it could also be the control interface of a machine, a program, robot, computer, sensor or something else entirely. In general, any kind of agent that can act, or be acted upon, or store or manipulate information, can be represented as an identity. An *identity* can be used to create any number of destinations.

A *single* destination will always have an *identity* tied to it, but not *plain* or *group* destinations. Destinations and identities share a multilateral connection. You can create a destination, and if it is not connected to an identity upon creation, it will just create a new one to use automatically. This may be desirable in some situations, but often you will probably want to create the identity first, and then use it to create new destinations.

As an example, we could use an identity to represent the user of a messaging application. Destinations can then be created by this identity to allow communication to reach the user. In all cases it is of great importance to store the private keys associated with any Reticulum Identity securely and privately, since obtaining access to the identity keys equals obtaining access and controlling reachability to any destinations created by that identity.

6.3.4 Getting Further

The above functions and principles form the core of Reticulum, and would suffice to create functional networked applications in local clusters, for example over radio links where all interested nodes can directly hear each other. But to be truly useful, we need a way to direct traffic over multiple hops in the network.

In the following sections, two concepts that allow this will be introduced, *paths* and *links*.

6.4 Reticulum Transport

The methods of routing used in traditional networks are fundamentally incompatible with the physical medium types and circumstances that Reticulum was designed to handle. These mechanisms mostly assume trust at the physical layer, and often needs a lot more bandwidth than Reticulum can assume is available. Since Reticulum is designed to survive running over open radio spectrum, no such trust can be assumed, and bandwidth is often very limited.

To overcome such challenges, Reticulum's *Transport* system uses asymmetric elliptic curve cryptography to implement the concept of *paths* that allow discovery of how to get information closer to a certain destination. It is important to note that no single node in a Reticulum network knows the complete path to a destination. Every Transport node participating in a Reticulum network will only know the most direct way to get a packet one hop closer to its destination.

6.4.1 Node Types

Currently, Reticulum distinguishes between two types of network nodes. All nodes on a Reticulum network are *Reticulum Instances*, and some are also *Transport Nodes*. If a system running Reticulum is fixed in one place, and is intended to be kept available most of the time, it is a good contender to be a *Transport Node*.

Any Reticulum Instance can become a Transport Node by enabling it in the configuration. This distinction is made by the user configuring the node, and is used to determine what nodes on the network will help forward traffic, and what nodes rely on other nodes for wider connectivity.

If a node is an *Instance* it should be given the configuration directive `enable_transport = No`, which is the default setting.

If it is a *Transport Node*, it should be given the configuration directive `enable_transport = Yes`.

6.4.2 The Announce Mechanism in Detail

When an *announce* for a destination is transmitted by a Reticulum instance, it will be forwarded by any transport node receiving it, but according to some specific rules:

- If this exact announce has already been received before, ignore it.
- If not, record into a table which Transport Node the announce was received from, and how many times in total it has been retransmitted to get here.
- If the announce has been retransmitted $m+1$ times, it will not be forwarded any more. By default, m is set to 128.
- After a randomised delay, the announce will be retransmitted on all interfaces that have bandwidth available for processing announces. By default, the maximum bandwidth allocation for processing announces is set at 2%, but can be configured on a per-interface basis.
- If any given interface does not have enough bandwidth available for retransmitting the announce, the announce will be assigned a priority inversely proportional to its hop count, and be inserted into a queue managed by the interface.
- When the interface has bandwidth available for processing an announce, it will prioritise announces for destinations that are closest in terms of hops, thus prioritising reachability and connectivity of local nodes, even on slow networks that connect to wider and faster networks.

- After the announce has been re-transmitted, and if no other nodes are heard retransmitting the announce with a greater hop count than when it left this node, transmitting it will be retried r times. By default, r is set to 1.
- If a newer announce from the same destination arrives, while an identical one is already waiting to be transmitted, the newest announce is discarded. If the newest announce contains different application specific data, it will replace the old announce.

Once an announce has reached a transport node in the network, any other node in direct contact with that transport node will be able to reach the destination the announce originated from, simply by sending a packet addressed to that destination. Any transport node with knowledge of the announce will be able to direct the packet towards the destination by looking up the most efficient next node to the destination.

According to these rules, an announce will propagate throughout the network in a predictable way, and make the announced destination reachable in a short amount of time. Fast networks that have the capacity to process many announces can reach full convergence very quickly, even when constantly adding new destinations. Slower segments of such networks might take a bit longer to gain full knowledge about the wide and fast networks they are connected to, but can still do so over time, while prioritising full and quickly converging end-to-end connectivity for their local, slower segments.

Tip

Even very slow networks, that simply don't have the capacity to ever reach *full* convergence will generally still be able to reach **any other destination on any connected segments**, since interconnecting transport nodes will prioritize announces into the slower segments that are actually requested by nodes on these.

This means that slow, low-capacity or low-resource segments **don't** need to have full network knowledge, since paths can always be recursively resolved from other segments that do have knowledge about them.

In general, even extremely complex networks, that utilize the maximum 128 hops will converge to full end-to-end connectivity in about one minute, given there is enough bandwidth available to process the required amount of announces.

6.4.3 Reaching the Destination

In networks with changing topology and trustless connectivity, nodes need a way to establish *verified connectivity* with each other. Since the underlying network mediums are assumed to be trustless, Reticulum must provide a way to guarantee that the peer you are communicating with is actually who you expect. Reticulum offers two ways to do this.

For exchanges of small amounts of information, Reticulum offers the *Packet* API, which works exactly like you would expect - on a per packet level. The following process is employed when sending a packet:

- A packet is always created with an associated destination and some payload data. When the packet is sent to a *single* destination type, Reticulum will automatically create an ephemeral encryption key, perform an ECDH key exchange with the destination's public key (or ratchet key, if available), and encrypt the information.
- It is important to note that this key exchange does not require any network traffic. The sender already knows the public key of the destination from an earlier received announce, and can thus perform the ECDH key exchange locally, before sending the packet.
- The public part of the newly generated ephemeral key-pair is included with the encrypted token, and sent along with the encrypted payload data in the packet.
- When the destination receives the packet, it can itself perform an ECDH key exchange and decrypt the packet.
- A new ephemeral key is used for every packet sent in this way.
- Once the packet has been received and decrypted by the addressed destination, that destination can opt to *prove* its receipt of the packet. It does this by calculating the SHA-256 hash of the received packet, and signing this hash with its Ed25519 signing key. Transport nodes in the network can then direct this *proof* back to the packet's origin, where the signature can be verified against the destination's known public signing key.

- In case the packet is addressed to a *group* destination type, the packet will be encrypted with the pre-shared AES-256 key associated with the destination. In case the packet is addressed to a *plain* destination type, the payload data will not be encrypted. Neither of these two destination types can offer forward secrecy. In general, it is recommended to always use the *single* destination type, unless it is strictly necessary to use one of the others.

For exchanges of larger amounts of data, or when longer sessions of bidirectional communication is desired, Reticulum offers the *Link* API. To establish a *link*, the following process is employed:

- First, the node that wishes to establish a link will send out a *link request* packet, that traverses the network and locates the desired destination. Along the way, the Transport Nodes that forward the packet will take note of this *link request*, and mark it as pending.
- Second, if the destination accepts the *link request*, it will send back a packet that proves the authenticity of its identity (and the receipt of the link request) to the initiating node. All nodes that initially forwarded the packet will also be able to verify this proof, and thus accept the validity of the *link* throughout the network. The link is now marked as *established*.
- When the validity of the *link* has been accepted by forwarding nodes, these nodes will remember the *link*, and it can subsequently be used by referring to a hash representing it.
- As a part of the *link request*, an Elliptic Curve Diffie-Hellman key exchange takes place, that sets up an efficiently encrypted tunnel between the two nodes. As such, this mode of communication is preferred, even for situations when nodes can directly communicate, when the amount of data to be exchanged numbers in the tens of packets, or whenever the use of the more advanced API functions is desired.
- When a *link* has been set up, it automatically provides message receipt functionality, through the same *proof* mechanism discussed before, so the sending node can obtain verified confirmation that the information reached the intended recipient.
- Once the *link* has been set up, the initiator can remain anonymous, or choose to authenticate towards the destination using a Reticulum Identity. This authentication is happening inside the encrypted link, and is only revealed to the verified destination, and no intermediaries.

In a moment, we will discuss the details of how this methodology is implemented, but let's first recap what purposes this methodology serves. We first ensure that the node answering our request is actually the one we want to communicate with, and not a malicious actor pretending to be so. At the same time we establish an efficient encrypted channel. The setup of this is relatively cheap in terms of bandwidth, so it can be used just for a short exchange, and then recreated as needed, which will also rotate encryption keys. The link can also be kept alive for longer periods of time, if this is more suitable to the application. The procedure also inserts the *link id*, a hash calculated from the link request packet, into the memory of forwarding nodes, which means that the communicating nodes can thereafter reach each other simply by referring to this *link id*.

The combined bandwidth cost of setting up a link is 3 packets totalling 297 bytes (more info in the [Binary Packet Format](#) section). The amount of bandwidth used on keeping a link open is practically negligible, at 0.45 bits per second. Even on a slow 1200 bits per second packet radio channel, 100 concurrent links will still leave 96% channel capacity for actual data.

Link Establishment in Detail

After exploring the basics of the announce mechanism, finding a path through the network, and an overview of the link establishment procedure, this section will go into greater detail about the Reticulum link establishment process.

The *link* in Reticulum terminology should not be viewed as a direct node-to-node link on the physical layer, but as an abstract channel, that can be open for any amount of time, and can span an arbitrary number of hops, where information will be exchanged between two nodes.

- When a node in the network wants to establish verified connectivity with another node, it will randomly generate a new X25519 private/public key pair. It then creates a *link request* packet, and broadcast it.

It should be noted that the X25519 public/private keypair mentioned above is two separate keypairs: An encryption key pair, used for derivation of a shared symmetric key, and a signing key pair, used for signing and verifying messages on the link. They are sent together over the wire, and can be considered as single public key for simplicity in this explanation.

- The *link request* is addressed to the destination hash of the desired destination, and contains the following data: The newly generated X25519 public key *LKi*.
- The broadcasted packet will be directed through the network according to the rules laid out previously.
- Any node that forwards the link request will store a *link id* in it's *link table*, along with the amount of hops the packet had taken when received. The link id is a hash of the entire link request packet. If the link request packet is not *proven* by the addressed destination within some set amount of time, the entry will be dropped from the *link table* again.
- When the destination receives the link request packet, it will decide whether to accept the request. If it is accepted, the destination will also generate a new X25519 private/public key pair, and perform a Diffie Hellman Key Exchange, deriving a new symmetric key that will be used to encrypt the channel, once it has been established.
- A *link proof* packet is now constructed and transmitted over the network. This packet is addressed to the *link id* of the *link*. It contains the following data: The newly generated X25519 public key *LKr* and an Ed25519 signature of the *link id* and *LKr* made by the *original signing key* of the addressed destination.
- By verifying this *link proof* packet, all nodes that originally transported the *link request* packet to the destination from the originator can now verify that the intended destination received the request and accepted it, and that the path they chose for forwarding the request was valid. In successfully carrying out this verification, the transporting nodes marks the link as active. An abstract bi-directional communication channel has now been established along a path in the network. Packets can now be exchanged bi-directionally from either end of the link simply by addressing the packets to the *link id* of the link.
- When the source receives the *proof*, it will know unequivocally that a verified path has been established to the destination. It can now also use the X25519 public key contained in the *link proof* to perform it's own Diffie Hellman Key Exchange and derive the symmetric key that is used to encrypt the channel. Information can now be exchanged reliably and securely.

Note

It's important to note that this methodology ensures that the source of the request does not need to reveal any identifying information about itself. **The link initiator remains completely anonymous.**

When using *links*, Reticulum will automatically verify all data sent over the link, and can also automate retransmissions if *Resources* are used.

6.4.4 Resources

For exchanging small amounts of data over a Reticulum network, the *Packet* interface is sufficient, but for exchanging data that would require many packets, an efficient way to coordinate the transfer is needed.

This is the purpose of the Reticulum *Resource*. A *Resource* can automatically handle the reliable transfer of an arbitrary amount of data over an established *Link*. Resources can auto-compress data, will handle breaking the data into individual packets, sequencing the transfer, integrity verification and reassembling the data on the other end.

Resources are programmatically very simple to use, and only requires a few lines of codes to reliably transfer any amount of data. They can be used to transfer data stored in memory, or stream data directly from files.

6.5 Network Identities

In Reticulum, every peer and application utilizes a cryptographic **Identity** to verify authenticity and establish encrypted channels. While standard identities are typically used to represent a single user, device, or service, Reticulum introduces the concept of a **Network Identity** to represent a logical group of nodes or an entire community infrastructure.

A Network Identity is, at its core, a standard Reticulum Identity keyset. However, its purpose and usage differ from a personal identity. Instead of identifying a single entity, a Network Identity acts as a shared credential that federates multiple independent Transport Instances under a single, verifiable administrative domain.

6.5.1 Conceptual Overview

You can think of a standard Reticulum Identity as a self-sovereign, privately created passport for a single person. A Network Identity, conversely, is akin to a cryptographic flag, or a charter that flies over a fleet of ships. It signifies that while the ships may operate independently and be physically distant, they belong to the same organization, follow the same protocols, and are expected to act in concert.

When you configure a Network Identity on one or more of your nodes, you are effectively declaring that these nodes constitute a specific “network” within a broader Reticulum mesh. This allows other peers to recognize interfaces not just as “a node named Alice”, but as “a gateway belonging to The Eastern Ret Of Freedom”.

6.5.2 Current Usage

At present, the primary function of a Network Identity is within the *Interface Discovery* system.

When a Transport Instance broadcasts a discovery announce for an interface, it can optionally sign that announce with a Network Identity, instead of just its local transport identity. Remote peers receiving the announce can then verify the signature. This provides functionality for two important distinctions:

1. **Authenticity:** It proves that the interface was published by an operator who possesses the private key for that Network Identity.
2. **Trust Boundaries:** It allows users to configure their systems to only accept and connect to interfaces that belong to specific Network Identities, effectively creating “whitelisted” zones of trusted infrastructure.

Note

If you enable encryption on your discovery announces, the Network Identity is used as the shared secret. Only peers who have been explicitly provided with the Network Identity’s full keyset (and have it configured locally) will be able to decrypt and utilize the connection details.

This functionality will be expanded in the future, so that peers with delegated keys can be allowed to decrypt discovery announces without holding the root network key. Currently, the functionality is sufficient for sharing interface information privately where you control all nodes that must decrypt the discovered interfaces.

6.5.3 Future Implications

While the current implementation focuses on interface discovery, the concept of Network Identities serves as the foundational building block for future Reticulum features designed to support large-scale, organic mesh formation.

As the ecosystem evolves, Network Identities will facilitate:

- **Distributed Name Resolution:** A system where networks can publish name-to-identity mappings, allowing human-readable names to resolve without centralized servers.
- **Service Publishing:** Networks will be able to announce specific capabilities, services, or information endpoints available publicly or to their members.

- **Inter-Network Federation:** Trust relationships between different networks, allowing for seamless but managed flow of traffic and information across distinct administrative boundaries.
- **Distributed Blackhole Management:** A reputation-based system for blackhole list distribution, where trusted Network Identities can sign and publish lists of blackholed identities. This allows communities to collaboratively enforce security standards and filter spam or malicious identities across the parts of the wider mesh that they are responsible for.

By adopting the use of Network Identities now, you are preparing your infrastructure to be compatible with this future functionality.

6.5.4 Creating and Using a Network Identity

Since a Network Identity is simply a standard Reticulum Identity, you create one using the built-in tools.

1. **Generate the Identity:** Use the `rnid` utility to generate a new identity file that will serve as your Network Identity.

```
$ rnid -g ~/.reticulum/storage/identities/my_network
```

2. **Distribute the Public Key:** The public key must be distributed to any Transport Instance that needs to verify your network's announces and discovery information. By default, if your node is set up to use a network identity, this happens automatically (using the standard announce mechanism).
3. **Configure Instances:** In the `[reticulum]` section of the configuration file on every node within your network, point the `network_identity` option to the file you created.

```
[reticulum]
...
network_identity = ~/.reticulum/storage/identities/my_network
...
```

Once configured, your instances will automatically utilize this identity for signing discovery announces (and potentially decrypting network-private information), presenting a unified front to the wider network.

6.6 Reference Setup

This section will detail a recommended *Reference Setup* for Reticulum. It is important to note that Reticulum is designed to be usable on more or less any computing device, and over more or less any medium that allows you to send and receive data, which satisfies some very low minimum requirements.

The communication channel must support at least half-duplex operation, and provide an average throughput of 5 bits per second or greater, and supports a physical layer MTU of 500 bytes. The Reticulum stack should be able to run on more or less any hardware that can provide a Python 3.x runtime environment.

That being said, this reference setup has been outlined to provide a common platform for anyone who wants to help in the development of Reticulum, and for everyone who wants to know a recommended setup to get started experimenting. A reference system consists of three parts:

- **An Interface Device**
Which provides access to the physical medium whereupon the communication takes place, for example a radio with an integrated modem. A setup with a separate modem connected to a radio would also be an interface device.
- **A Host Device**
Some sort of computing device that can run the necessary software, communicate with the interface device, and provide user interaction.

- **A Software Stack**

The software implementing the Reticulum protocol and applications using it.

The reference setup can be considered a relatively stable platform to develop on, and also to start building networks or applications on. While details of the implementation might change at the current stage of development, it is the goal to maintain hardware compatibility for as long as entirely possible, and the current reference setup has been determined to provide a functional platform for many years into the future. The current Reference System Setup is as follows:

- **Interface Device**

A data radio consisting of a LoRa radio module, and a microcontroller with open source firmware, that can connect to host devices via USB. It operates in either the 430, 868 or 900 MHz frequency bands. More details can be found on the [RNode Page](#).

- **Host Device**

Any computer device running Linux and Python. A Raspberry Pi with a Debian based OS is a good place to start, but anything can be used.

- **Software Stack**

The most recently released Python Implementation of Reticulum, running on a Linux-based operating system.

Note

To avoid confusion, it is very important to note, that the reference interface device **does not** use the LoRaWAN standard, but uses a custom MAC layer on top of the plain LoRa modulation! As such, you will need a plain LoRa radio module connected to a controller with the correct firmware. Full details on how to get or make such a device is available on the [RNode Page](#).

With the current reference setup, it should be possible to get on a Reticulum network for around 100\$ even if you have none of the hardware already, and need to purchase everything.

This reference setup is of course just a recommendation for getting started easily, and you should tailor it to your own specific needs, or whatever hardware you have available.

6.7 Protocol Specifics

This chapter will detail protocol specific information that is essential to the implementation of Reticulum, but non-critical in understanding how the protocol works on a general level. It should be treated more as a reference than as essential reading.

6.7.1 Packet Prioritisation

Currently, Reticulum is completely priority-agnostic regarding *general* traffic. All traffic is handled on a first-come, first-serve basis. Announce re-transmission and other maintenance traffic is handled according to the re-transmission times and priorities described earlier in this chapter.

6.7.2 Interface Access Codes

Reticulum can create named virtual networks, and networks that are only accessible by knowing a preshared passphrase. The configuration of this is detailed in the [Common Interface Options](#) section. To implement this feature, Reticulum uses the concept of Interface Access Codes, that are calculated and verified per-packet.

An interface with a named virtual network or passphrase authentication enabled will derive a shared Ed25519 signing identity, and for every outbound packet generate a signature of the entire packet. This signature is then inserted into the packet as an Interface Access Code before transmission. Depending on the speed and capabilities of the interface,

the IFAC can be the full 512-bit Ed25519 signature, or a truncated version. Configured IFAC length can be inspected for all interfaces with the `rnstatus` utility.

Upon receipt, the interface will check that the signature matches the expected value, and drop the packet if it does not. This ensures that only packets sent with the correct naming and/or passphrase parameters are allowed to pass onto the network.

6.7.3 Wire Format

== Reticulum Wire Format =====

A Reticulum packet is composed of the following fields:

[HEADER 2 bytes] [ADDRESSES 16/32 bytes] [CONTEXT 1 byte] [DATA 0-465 bytes]

- * The HEADER field is 2 bytes long.
 - * Byte 1: [IFAC Flag], [Header Type], [Context Flag], [Propagation Type], [Destination Type] and [Packet Type]
 - * Byte 2: Number of hops
- * Interface Access Code field if the IFAC flag was set.
 - * The length of the Interface Access Code can vary from 1 to 64 bytes according to physical interface capabilities and configuration.
- * The ADDRESSES field contains either 1 or 2 addresses.
 - * Each address is 16 bytes long.
 - * The Header Type flag in the HEADER field determines whether the ADDRESSES field contains 1 or 2 addresses.
 - * Addresses are SHA-256 hashes truncated to 16 bytes.
- * The CONTEXT field is 1 byte.
 - * It is used by Reticulum to determine packet context.
- * The DATA field is between 0 and 465 bytes.
 - * It contains the packets data payload.

IFAC Flag

open	0	Packet for publically accessible interface
authenticated	1	Interface authentication is included in packet

Header Types

type 1	0	Two byte header, one 16 byte address field
type 2	1	Two byte header, two 16 byte address fields

Context Flag

unset	0	The context flag is used for various types
set	1	of signalling, depending on packet context

(continues on next page)

(continued from previous page)

Propagation Types

broadcast	0
transport	1

Destination Types

single	00
group	01
plain	10
link	11

Packet Types

data	00
announce	01
link request	10
proof	11

```
+- Packet Example -+
```

HEADER FIELD	DESTINATION FIELDS	CONTEXT FIELD	DATA FIELD
01010000 00000100	[HASH1, 16 bytes] [HASH2, 16 bytes]	[CONTEXT, 1 byte]	[DATA]
+--	Hops	=	4
+-----	Packet Type	=	DATA
+-----	Destination Type	=	SINGLE
+-----	Propagation Type	=	TRANSPORT
+-----	Header Type	=	HEADER_2 (two byte header, two address fields)
+-----	Access Codes	=	DISABLED

```
+- Packet Example -+
```

HEADER FIELD	DESTINATION FIELD	CONTEXT FIELD	DATA FIELD
000000000 00000111	[HASH1, 16 bytes]	[CONTEXT, 1 byte]	[DATA]
+--	Hops	= 7	
+-----	Packet Type	= DATA	
+-----	Destination Type	= SINGLE	
+-----	Propagation Type	= BROADCAST	
+-----	Header Type	= HEADER_1 (two byte header, one address field)	
+-----	Access Codes	= DISABLED	

(continues on next page)

(continued from previous page)

```
+-- Packet Example --+

  HEADER FIELD      IFAC FIELD      DESTINATION FIELD      CONTEXT FIELD      DATA FIELD
  -----|-----  -----|-----  -----|-----  -----|-----  --|_
  |              | |              | |              | |              | |              |
100000000 00000111 [IFAC, N bytes] [HASH1, 16 bytes] [CONTEXT, 1 byte] [DATA]
| | | | |
| | | | | +-- Hops          = 7
| | | | +----- Packet Type    = DATA
| | | +----- Destination Type = SINGLE
| | +----- Propagation Type = BROADCAST
| +----- Header Type    = HEADER_1 (two byte header, one address field)
+----- Access Codes     = ENABLED
```

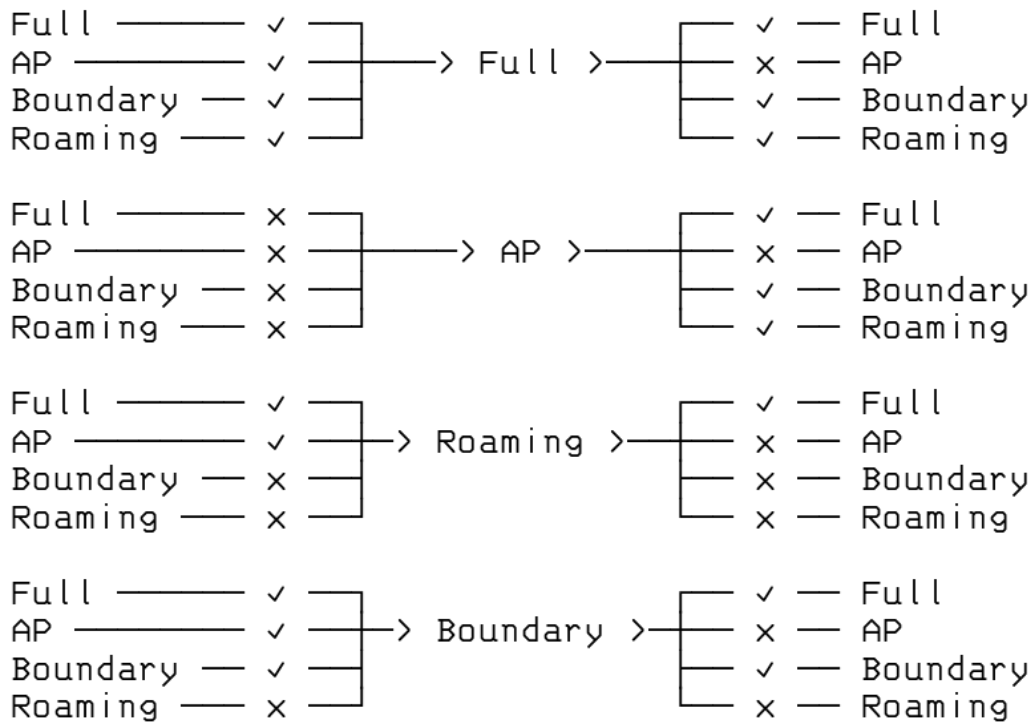
Size examples of different packet types

The following table lists example sizes of various packet types. The size listed are the complete on-wire size counting all fields including headers, but excluding any interface access codes.

- Path Request : 51 bytes
- Announce : 167 bytes
- Link Request : 83 bytes
- Link Proof : 115 bytes
- Link RTT packet : 99 bytes
- Link keepalive : 20 bytes

6.7.4 Announce Propagation Rules

The following table illustrates the rules for automatically propagating announces from one interface type to another, for all possible combinations. For the purpose of announce propagation, the *Full* and *Gateway* modes are identical.



See the [Interface Modes](#) section for a conceptual overview of the different interface modes, and how they are configured.

6.7.5 Cryptographic Primitives

Reticulum uses a simple suite of efficient, strong and well-tested cryptographic primitives, with widely available implementations that can be used both on general-purpose CPUs and on microcontrollers.

One of the primary considerations for choosing this particular set of primitives is that they can be implemented *safely* with relatively few pitfalls, on practically all current computing platforms.

The primitives listed here **are authoritative**. Anything claiming to be Reticulum, but not using these exact primitives **is not** Reticulum, and possibly an intentionally compromised or weakened clone. The utilised primitives are:

- Ed25519 for signatures
- X25519 for ECDH key exchanges
- HKDF for key derivation
- Encrypted tokens are based on the Fernet spec
 - Ephemeral keys derived from an ECDH key exchange on Curve25519
 - AES-256 in CBC mode with PKCS7 padding
 - HMAC using SHA256 for message authentication
 - IVs must be generated through `os.urandom()` or better
 - No Fernet version and timestamp metadata fields

- SHA-256
- SHA-512

In the default installation configuration, the X25519, Ed25519 and AES-256-CBC primitives are provided by [OpenSSL](#) (via the [PyCA/cryptography](#) package). The hashing functions SHA-256 and SHA-512 are provided by the standard Python [hashlib](#). The HKDF, HMAC, Token primitives, and the PKCS7 padding function are always provided by the following internal implementations:

- `RNS/Cryptography/HKDF.py`
- `RNS/Cryptography/HMAC.py`
- `RNS/Cryptography/Token.py`
- `RNS/Cryptography/PKCS7.py`

Reticulum also includes a complete implementation of all necessary primitives in pure Python. If OpenSSL & PyCA are not available on the system when Reticulum is started, Reticulum will instead use the internal pure-python primitives. A trivial consequence of this is performance, with the OpenSSL backend being *much* faster. The most important consequence however, is the potential loss of security by using primitives that has not seen the same amount of scrutiny, testing and review as those from OpenSSL.

Using the normal RNS installation procedures, it is not possible to install Reticulum on a system without the required OpenSSL primitives being available, and if they are not, they will be resolved and installed as a dependency. It is only possible to use the pure-python primitives by manually specifying this, for example by using the `rnspure` package.

Warning

If you want to use the internal pure-python primitives, it is **highly advisable** that you have a good understanding of the risks that this pose, and make an informed decision on whether those risks are acceptable to you.

COMMUNICATIONS HARDWARE

One of the truly valuable aspects of Reticulum is the ability to use it over almost any conceivable kind of communications medium. The *interface types* available for configuration in Reticulum are flexible enough to cover the use of most wired and wireless communications hardware available, from decades-old packet radio modems to modern millimeter-wave backhaul systems.

If you already have or operate some kind of communications hardware, there is a very good chance that it will work with Reticulum out of the box. In case it does not, it is possible to provide the necessary glue with very little effort using for example the *PipeInterface* or the *TCPClientInterface* in combination with code like *TCP KISS Server* by *simplyequipped*.

It is also very easy to write and load *custom interface modules* into Reticulum, allowing you to communicate with more or less anything you can think of.

While this broad support and flexibility is very useful, an abundance of options can sometimes make it difficult to know where to begin, especially when you are starting from scratch.

This chapter will outline a few different sensible starting paths to get real-world functional wireless communications up and running with minimal cost and effort. Two fundamental devices categories will be covered, *RNodes* and *WiFi-based radios*. Additionally, other common options will be briefly described.

Knowing how to employ just a few different types of hardware will make it possible to build a wide range of useful networks with little effort.

7.1 Combining Hardware Types

It is useful to combine different link and hardware types when designing and building a network. One useful design pattern is to employ high-capacity point-to-point links based on WiFi or millimeter-wave radios (with high-gain directional antennas) for the network backbone, and using LoRa-based *RNodes* for covering large areas with connectivity for client devices.

7.2 RNode

Reliable and general-purpose long-range digital radio transceiver systems are commonly either very expensive, difficult to set up and operate, hard to source, power-hungry, or all of the above at the same time. In an attempt to alleviate this situation, the transceiver system *RNode* was designed. It is important to note that *RNode* is not one specific device, from one particular vendor, but *an open platform* that anyone can use to build interoperable digital transceivers suited to their needs and particular situations.

An *RNode* is a general purpose, interoperable, low-power and long-range, reliable, open and flexible radio communications device. Depending on its components, it can operate on many different frequency bands, and use many different modulation schemes, but most commonly, and for the purposes of this chapter, we will limit the discussion to *RNodes* using *LoRa* modulation in common ISM bands.

Avoid Confusion! RNodes can use LoRa as a *physical-layer modulation*, but it does not use, and has nothing to do with the *LoRaWAN* protocol and standard, commonly used for centrally controlled IoT devices. RNodes use *raw LoRa modulation*, without any additional protocol overhead. All high-level protocol functionality is handled directly by Reticulum.

7.2.1 Creating RNodes

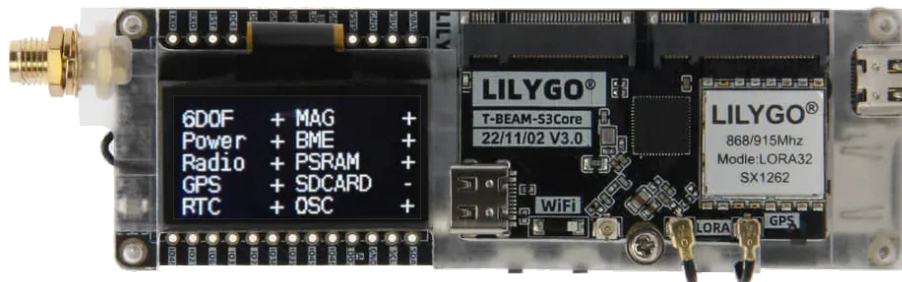
RNode has been designed as a system that is easy to replicate across time and space. You can put together a functioning transceiver using commonly available components, and a few open source software tools. While you can design and build RNodes completely from scratch, to your exact desired specifications, this chapter will explain the easiest possible approach to creating RNodes: Using common LoRa development boards. This approach can be boiled down to two simple steps:

1. Obtain one or more *supported development boards*
2. Install the RNode firmware with the *automated installer*

Once the firmware has been installed and provisioned by the install script, it is ready to use with any software that supports RNodes, including Reticulum. The device can be used with Reticulum by adding an *RNodeInterface* to the configuration.

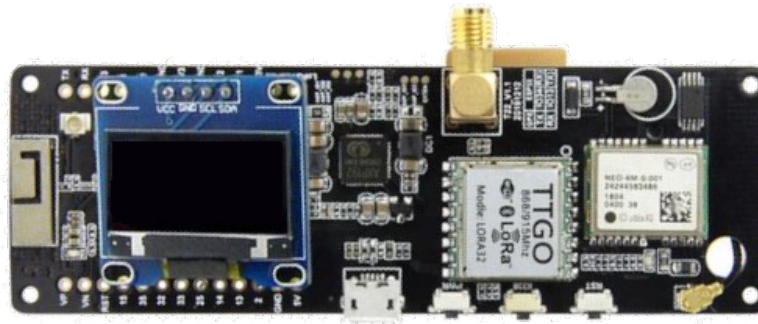
7.2.2 Supported Boards and Devices

To create one or more RNodes, you will need to obtain supported development boards or completed devices. The following boards and devices are supported by the auto-installer.



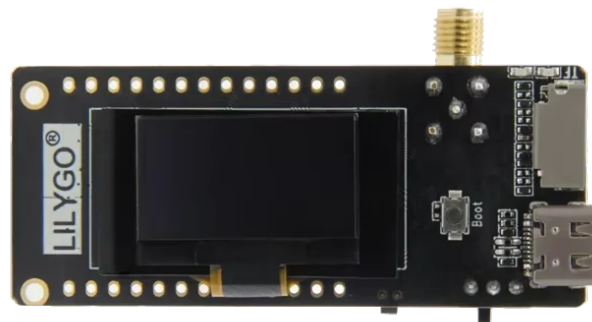
LilyGO T-Beam Supreme

- **Transceiver IC** Semtech SX1262 or SX1268
 - **Device Platform** ESP32
 - **Manufacturer** LilyGO
-



LilyGO T-Beam

- **Transceiver IC** Semtech SX1262, SX1268, SX1276 or SX1278
 - **Device Platform** ESP32
 - **Manufacturer** [LilyGO](#)
-



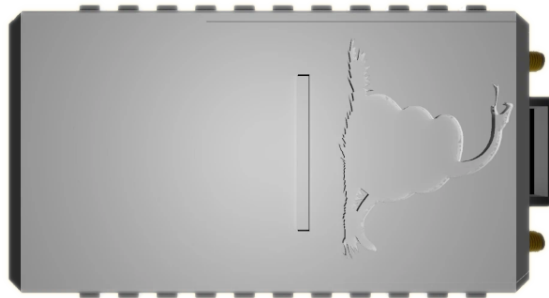
LilyGO T3S3

- **Transceiver IC** Semtech SX1262, SX1268, SX1276 or SX1278
 - **Device Platform** ESP32
 - **Manufacturer** [LilyGO](#)
-



RAK4631-based Boards

- **Transceiver IC** Semtech SX1262 or SX1268
 - **Device Platform** nRF52
 - **Manufacturer** [RAK Wireless](#)
-



OpenCom XL

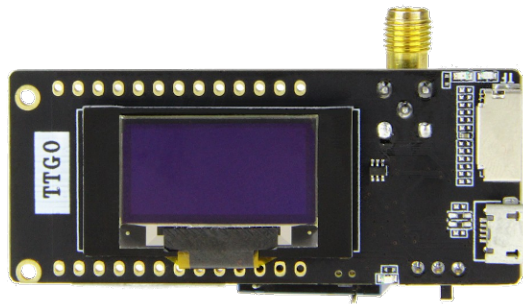
- **Transceiver ICs** Semtech SX1262 and SX1280 (dual transceiver)
 - **Device Platform** nRF52
 - **Manufacturer** [Liberated Embedded Systems](#)
-



Unsigned RNode v2.x

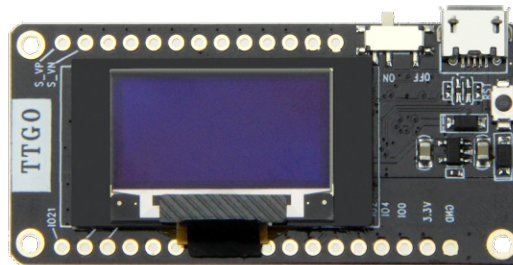
- **Transceiver IC** Semtech SX1276 or SX1278
- **Device Platform** ESP32

- **Manufacturer** unsigned.io



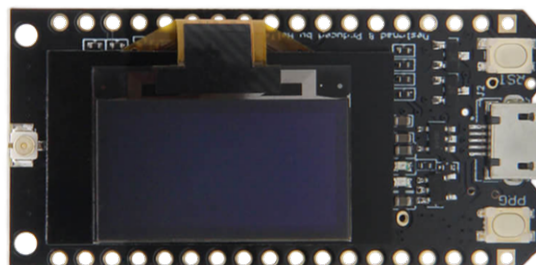
LilyGO LoRa32 v2.1

- **Transceiver IC** Semtech SX1276 or SX1278
- **Device Platform** ESP32
- **Manufacturer** [LilyGO](https://lilygo.cc)



LilyGO LoRa32 v2.0

- **Transceiver IC** Semtech SX1276 or SX1278
- **Device Platform** ESP32
- **Manufacturer** [LilyGO](https://lilygo.cc)



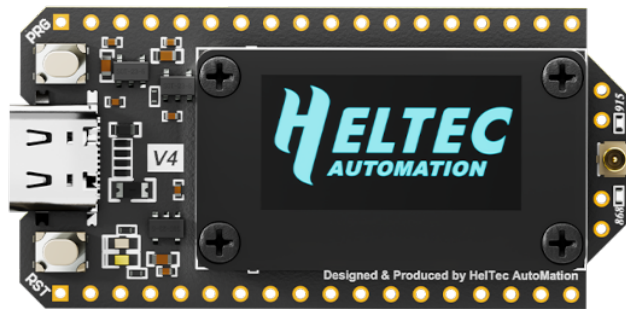
LilyGO T-Echo

- **Transceiver IC** Semtech SX1262 or SX1268
 - **Device Platform** nRF52
 - **Manufacturer** [LilyGO](#)
-



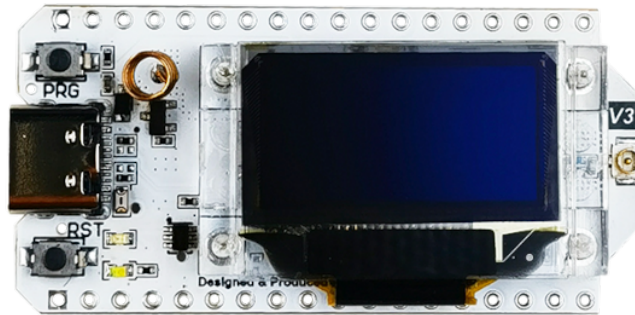
Heltec T114

- **Transceiver IC** Semtech SX1262 or SX1268
 - **Device Platform** nRF52
 - **Manufacturer** [Heltec Automation](#)
-



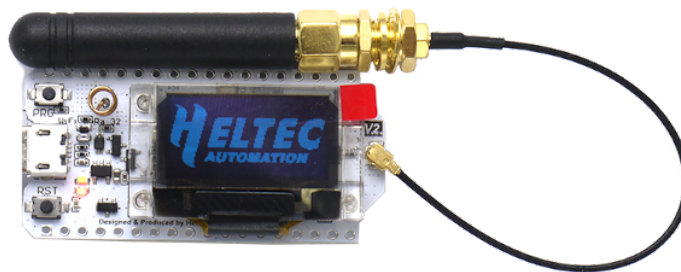
Heltec LoRa32 v4.0

- **Transceiver IC** Semtech SX1262
 - **Device Platform** ESP32
 - **Manufacturer** [Heltec Automation](#)
-



Heltec LoRa32 v3.0

- **Transceiver IC** Semtech SX1262 or SX1268
 - **Device Platform** ESP32
 - **Manufacturer** [Heltec Automation](#)
-



Heltec LoRa32 v2.0

- **Transceiver IC** Semtech SX1276 or SX1278
 - **Device Platform** ESP32
 - **Manufacturer** [Heltec Automation](#)
-

7.2.3 Installation

Once you have obtained compatible boards, you can install the [RNode Firmware](#) using the [RNode Configuration Utility](#). If you have installed Reticulum on your system, the `rnodeconf` program will already be available. If not, make sure that Python3 and `pip` is installed on your system, and then install Reticulum with `pip`:

```
pip install rns
```

Once installation has completed, it is time to start installing the firmware on your devices. Run `rnodeconf` in auto-install mode like so:

```
rnodeconf --autoinstall
```

The utility will guide you through the installation process by asking a series of questions about your hardware. Simply follow the guide, and the utility will auto-install and configure your devices.

7.2.4 Usage with Reticulum

When the devices have been installed and provisioned, you can use them with Reticulum by adding the *relevant interface section* to the configuration file of Reticulum. In the configuration you can specify all interface parameters, such as serial port and on-air parameters.

7.3 WiFi-based Hardware

It is possible to use all kinds of both short- and long-range WiFi-based hardware with Reticulum. Any kind of hardware that fully supports bridged Ethernet over the WiFi interface will work with the *AutoInterface* in Reticulum. Most devices will behave like this by default, or allow it via configuration options.

This means that you can simply configure the physical links of the WiFi based devices, and start communicating over them using Reticulum. It is not necessary to enable any IP infrastructure such as DHCP servers, DNS or similar, as long as at least Ethernet is available, and packets are passed transparently over the physical WiFi-based devices.

Below is a list of example WiFi (and similar) radios that work well for high capacity Reticulum links over long distances:

- Ubiquiti airMAX radios
- Ubiquiti LTU radios
- MikroTik radios

This list is by no means exhaustive, and only serves as a few examples of radio hardware that is relatively cheap while providing long range and high capacity for Reticulum networks. As in all other cases, it is also possible for Reticulum to co-exist with IP networks running concurrently on such devices.

7.4 Ethernet-based Hardware

Reticulum can run over any kind of hardware that can provide a switched Ethernet-based medium. This means that anything from a plain Ethernet switch, to fiber-optic systems, to data radios with Ethernet interfaces can be used by Reticulum.

The Ethernet medium does not need to have any IP infrastructure such as DHCP servers or routing set up, but in case such infrastructure does exist, Reticulum will simply co-exist with.

To use Reticulum over Ethernet-based mediums, it is generally enough to use the included *AutoInterface*. This interface also works over any kind of virtual networking adapter, such as `tun` and `tap` devices in Linux.

7.5 Serial Lines & Devices

Using Reticulum over any kind of raw serial line is also possible with the *SerialInterface*. This interface type is also useful for using Reticulum over communications hardware that provides a serial port interface.

7.6 Packet Radio Modems

Any packet radio modem that provides a standard KISS interface over USB, serial or TCP can be used with Reticulum. This includes virtual software modems such as *FreeDV TNC* and *Dire Wolf*.

CONFIGURING INTERFACES

Reticulum supports using many kinds of devices as networking interfaces, and allows you to mix and match them in any way you choose. The number of distinct network topologies you can create with Reticulum is more or less endless, but common to them all is that you will need to define one or more *interfaces* for Reticulum to use.

The following sections describe the interfaces currently available in Reticulum, and gives example configurations for the respective interface types.

For a high-level overview of how networks can be formed over different interface types, have a look at the [Building Networks](#) chapter of this manual.

8.1 Custom Interfaces

In addition to the built-in interface types, Reticulum is **fully extensible** with custom, user- or community-supplied interfaces, and creating custom interface modules is straightforward. Please see the [custom interface](#) example for basic interface code to build upon.

8.2 Auto Interface

The `AutoInterface` enables communication with other discoverable Reticulum nodes over any kind of local Ethernet or WiFi-based medium. Even though it uses IPv6 for peer discovery, and UDP for packet transport, it **does not** need any functional IP infrastructure like routers or DHCP servers, on your physical network.

Warning

If you have **firewall** software running on your computer, it may block traffic required for `AutoInterface` to work. If this is the case, you will have to allow UDP traffic on port 29716 and 42671.

As long as there is at least some sort of switching medium present between peers (a wired switch, a hub, a WiFi access point or similar, or simply two devices connected directly by Ethernet cable), it will work without any configuration, setup or intermediary devices.

For `AutoInterface` peer discovery to work, it's also required that link-local IPv6 support is available on your system, which it should be by default in all current operating systems, both desktop and mobile.

Note

Almost all current Ethernet and WiFi hardware will work without any kind of configuration or setup with `AutoInterface`, but a small subset of devices turn on options that limit device-to-device communication by de-

fault, resulting in AutoInterface peer discovery being blocked. This issue is most commonly seen on very cheap, ISP-supplied WiFi routers, and can sometimes be turned off in the router configuration.

```
# This example demonstrates a bare-minimum setup
# of an Auto Interface. It will allow communica-
# tion with all other reachable devices on all
# usable physical ethernet-based devices that
# are available on the system.
[[Default Interface]]
    type = AutoInterface
    enabled = yes

# This example demonstrates an more specifically
# configured Auto Interface, that only uses spe-
# cific physical interfaces, and has a number of
# other configuration options set.
[[Default Interface]]
    type = AutoInterface
    enabled = yes

# You can create multiple isolated Reticulum
# networks on the same physical LAN by
# specifying different Group IDs.
group_id = reticulum

# You can also choose the multicast address type:
# temporary (default, Temporary Multicast Address)
# or permanent (Permanent Multicast Address)
multicast_address_type = permanent

# You can also select specifically which
# kernel networking devices to use.
devices = wlan0,eth1

# Or let AutoInterface use all suitable
# devices except for a list of ignored ones.
ignored_devices = tun0,eth0
```

If you are connected to the Internet with IPv6, and your provider will route IPv6 multicast, you can potentially configure the Auto Interface to globally autodiscover other Reticulum nodes within your selected Group ID. You can specify the discovery scope by setting it to one of link, admin, site, organisation or global.

```
[[Default Interface]]
    type = AutoInterface
    enabled = yes

# Configure global discovery

group_id = custom_network_name
discovery_scope = global

# Other configuration options
```

(continues on next page)

(continued from previous page)

```
discovery_port = 48555
data_port = 49555
```

8.3 Backbone Interface

The Backbone interface is a very fast and resource efficient interface type, primarily intended for interconnecting Reticulum instances over many different types of mediums. It uses a kernel-event I/O backend, and can handle thousands of interfaces and/or clients with relatively low system resource utilisation. **This interface type is currently only supported on Linux and Android.**

Note

The Backbone Interface is fully compatible with the TCPServerInterface and TCPClientInterface types, and they can be used interchangeably, and cross-connect with each other. On systems that support BackboneInterface, it is generally recommended to use it, unless you need specific options or features that the TCP server and client interfaces provide.

While the goal is to support *all* socket types and I/O devices provided by the underlying operating system, the initial release only provides support for TCP connections over IPv4 and IPv6.

For all types of connections over a BackboneInterface, Reticulum will gracefully handle intermittency, link loss, and connections that come and go.

8.3.1 Listeners

The following examples illustrates various ways to set up BackboneInterface listeners.

```
# This example demonstrates a backbone interface
# that listens for incoming connections on the
# specified IP address and port number.
[[Backbone Listener]]
  type = BackboneInterface
  enabled = yes
  listen_on = 0.0.0.0
  port = 4242

# Alternatively you can bind to a specific IP
[[Backbone Listener]]
  type = BackboneInterface
  enabled = yes
  listen_on = 10.0.0.88
  port = 4242

# Or a specific network device
[[Backbone Listener]]
  type = BackboneInterface
  enabled = yes
  device = eth0
  port = 4242
```

If you are using the interface on a device which has both IPv4 and IPv6 addresses available, you can use the `prefer_ipv6` option to bind to the IPv6 address:

```
# This example demonstrates a backbone interface
# listening on the IPv6 address of a specified
# kernel networking device.
[[Backbone Listener]]
    type = BackboneInterface
    enabled = yes
    prefer_ipv6 = yes
    device = eth0
    port = 4242
```

To use the `BackboneInterface` over `Yggdrasil`, you can simply specify the `Yggdrasil tun` device and a listening port, like so:

```
# This example demonstrates a backbone interface
# listening for connections over Yggdrasil.
[[Yggdrasil Backbone Interface]]
    type = BackboneInterface
    enabled = yes
    device = tun0
    port = 4343
```

8.3.2 Connecting Remotes

The following examples illustrates various ways to connect to remote `BackboneInterface` listeners. As noted above, `BackboneInterface` interfaces can also connect to remote `TCPServerInterface`, and as such these interface types can be used interchangeably.

```
# Here's an example of a backbone interface that
# connects to a remote listener.
[[Backbone Remote]]
    type = BackboneInterface
    enabled = yes
    remote = amsterdam.connect.reticulum.network
    target_port = 4251
```

To connect to remotes over `Yggdrasil`, simply specify the target `Yggdrasil IPv6` address and port, like so:

```
[[Yggdrasil Remote]]
    type = BackboneInterface
    enabled = yes
    target_host = 201:5d78:af73:5caf:a4de:a79f:3278:71e5
    target_port = 4343
```

8.4 TCP Server Interface

The TCP Server interface is suitable for allowing other peers to connect over the Internet or private IPv4 and IPv6 networks. When a TCP server interface has been configured, other Reticulum peers can connect to it with a TCP Client interface.


```
# This example demonstrates a TCP server interface.
# It will listen for incoming connections on all IP
# interfaces on port 4242.
[[TCP Server Interface]]
    type = TCPServerInterface
    enabled = yes
    listen_ip = 0.0.0.0
    listen_port = 4242

# Alternatively you can bind to a specific IP
[[TCP Server Interface]]
    type = TCPServerInterface
    enabled = yes
    listen_ip = 10.0.0.88
    listen_port = 4242

# Or a specific network device
[[TCP Server Interface]]
    type = TCPServerInterface
    enabled = yes
    device = eth0
    listen_port = 4242
```

If you are using the interface on a device which has both IPv4 and IPv6 addresses available, you can use the `prefer_ipv6` option to bind to the IPv6 address:

```
# This example demonstrates a TCP server interface.
# It will listen for incoming connections on the
# specified IP address and port number.

[[TCP Server Interface]]
    type = TCPServerInterface
    enabled = yes
    prefer_ipv6 = True
    device = eth0
    port = 4242
```

To use the TCP Server Interface over Yggdrasil, you can simply specify the Yggdrasil tun device and a listening port, like so:

```
[[Yggdrasil TCP Server Interface]]
    type = TCPServerInterface
    enabled = yes
    device = tun0
    listen_port = 4343
```

Note

The TCP interfaces support tunneling over I2P, but to do so reliably, you must use the `i2p_tunneled` option:

```
[[TCP Server on I2P]]
```

(continues on next page)

(continued from previous page)

```

type = TCPServerInterface
enabled = yes
listen_ip = 127.0.0.1
listen_port = 5001
i2p_tunneled = yes

```

In almost all cases, it is easier to use the dedicated `I2PInterface`, but for complete control, and using I2P routers running on external systems, this option also exists.

8.5 TCP Client Interface

To connect to a TCP server interface, you can use the TCP client interface. Many TCP Client interfaces from different peers can connect to the same TCP Server interface at the same time.

The TCP interface types can also tolerate intermittency in the IP link layer. This means that Reticulum will gracefully handle IP links that go up and down, and restore connectivity after a failure, once the other end of a TCP interface reappears.

```

# Here's an example of a TCP Client interface. The
# target_host can be a hostname or an IPv4 or IPv6 address.
[[TCP Client Interface]]
type = TCPClientInterface
enabled = yes
target_host = 127.0.0.1
target_port = 4242

```

To use the TCP Client Interface over Yggdrasil, simply specify the target Yggdrasil IPv6 address and port, like so:

```

[[Yggdrasil TCP Client Interface]]
type = TCPClientInterface
enabled = yes
target_host = 201:5d78:af73:5caf:a4de:a79f:3278:71e5
target_port = 4343

```

It is also possible to use this interface type to connect via other programs or hardware devices that expose a KISS interface on a TCP port, for example software-based soundmodems. To do this, use the `kiss_framing` option:

```

# Here's an example of a TCP Client interface that connects
# to a software TNC soundmodem on a KISS over TCP port.
[[TCP KISS Interface]]
type = TCPClientInterface
enabled = yes
kiss_framing = True
target_host = 127.0.0.1
target_port = 8001
fixed_mtu = 500

```

Caution! Only use the KISS framing option when connecting to external devices and programs like soundmodems and similar over TCP. When using the `TCPClientInterface` in conjunction with the `TCPServerInterface` you should never enable `kiss_framing`, since this will disable internal reliability and recovery mechanisms that greatly improves performance over unreliable and intermittent TCP links.

For KISS devices that need only supports a particular MTU, you can use the `fixed_mtu` option.

Note

The TCP interfaces support tunneling over I2P, but to do so reliably, you must use the `i2p_tunneled` option:

[[TCP Client over I2P]]

```
type = TCPClientInterface
enabled = yes
target_host = 127.0.0.1
target_port = 5001
i2p_tunneled = yes
```

8.6 UDP Interface

A UDP interface can be useful for communicating over IP networks, both private and the internet. It can also allow broadcast communication over IP networks, so it can provide an easy way to enable connectivity with all other peers on a local area network.

Warning

Using broadcast UDP traffic has performance implications, especially on WiFi. If your goal is simply to enable easy communication with all peers in your local Ethernet broadcast domain, the *Auto Interface* performs *much* better, and is even easier to use.

```
# This example enables communication with other
# local Reticulum peers over UDP.

[[UDP Interface]]
type = UDPInterface
enabled = yes

listen_ip = 0.0.0.0
listen_port = 4242
forward_ip = 255.255.255.255
forward_port = 4242

# The above configuration will allow communication
# within the local broadcast domains of all local
# IP interfaces.

# Instead of specifying listen_ip, listen_port,
# forward_ip and forward_port, you can also bind
# to a specific network device like below.

# device = eth0
# port = 4242

# Assuming the eth0 device has the address
# 10.55.0.72/24, the above configuration would
# be equivalent to the following manual setup.
# Note that we are both listening and forwarding to
```

(continues on next page)

(continued from previous page)

```
# the broadcast address of the network segments.

# listen_ip = 10.55.0.255
# listen_port = 4242
# forward_ip = 10.55.0.255
# forward_port = 4242

# You can of course also communicate only with
# a single IP address

# listen_ip = 10.55.0.15
# listen_port = 4242
# forward_ip = 10.55.0.16
# forward_port = 4242
```

8.7 I2P Interface

The I2P interface lets you connect Reticulum instances over the [Invisible Internet Protocol](#). This can be especially useful in cases where you want to host a globally reachable Reticulum instance, but do not have access to any public IP addresses, have a frequently changing IP address, or have firewalls blocking inbound traffic.

Using the I2P interface, you will get a globally reachable, portable and persistent I2P address that your Reticulum instance can be reached at.

To use the I2P interface, you must have an I2P router running on your system. The easiest way to achieve this is to download and install the [latest release](#) of the `i2pd` package. For more details about I2P, see the [geti2p.net website](#).

When an I2P router is running on your system, you can simply add an I2P interface to Reticulum:

```
[[I2P]]
type = I2PInterface
enabled = yes
connectable = yes
```

On the first start, Reticulum will generate a new I2P address for the interface and start listening for inbound traffic on it. This can take a while the first time, especially if your I2P router was also just started, and is not yet well-connected to the I2P network. When ready, you should see I2P base32 address printed to your log file. You can also inspect the status of the interface using the `rnstatus` utility.

To connect to other Reticulum instances over I2P, just add a comma-separated list of I2P base32 addresses to the `peers` option of the interface:

```
[[I2P]]
type = I2PInterface
enabled = yes
connectable = yes
peers = 5urvji7q3ybztsef4i5ow2aq4soktfj7zedz53s47r54jnqq.b32.i2p
```

It can take anywhere from a few seconds to a few minutes to establish I2P connections to the desired peers, so Reticulum handles the process in the background, and will output relevant events to the log.

Note

While the I2P interface is the simplest way to use Reticulum over I2P, it is also possible to tunnel the TCP server and client interfaces over I2P manually. This can be useful in situations where more control is needed, but requires manual tunnel setup through the I2P daemon configuration.

It is important to note that the two methods are *interchangably compatible*. You can use the I2PInterface to connect to a TCPServerInterface that was manually tunneled over I2P, for example. This offers a high degree of flexibility in network setup, while retaining ease of use in simpler use-cases.

8.8 RNode LoRa Interface

To use Reticulum over LoRa, the `RNode` interface can be used, and offers full control over LoRa parameters.

Warning

Radio frequency spectrum is a legally controlled resource, and legislation varies widely around the world. It is your responsibility to be aware of any relevant regulation for your location, and to make decisions accordingly.

```
# Here's an example of how to add a LoRa interface
# using the RNode LoRa transceiver.
```

```
[[RNode LoRa Interface]]
```

```
type = RNodeInterface
```

```
# Enable interface if you want use it!
enabled = yes
```

```
# Serial port for the device
port = /dev/ttyUSB0
```

```
# You can connect wirelessly to the
# RNode device if it supports WiFi.
```

```
# Connect by IP address
# port = tcp://10.0.0.1
```

```
# Or, connect by hostname
# port = tcp://rnodef3b9.local
```

```
# It is also possible to use BLE devices
# instead of wired serial ports. The
# target RNode must be paired with the
# host device before connecting. BLE
# devices can be connected by name,
# BLE MAC address or by any available.
```

```
# Connect to specific device by name
# port = ble://RNode 3B87
```

```
# Or by BLE MAC address
# port = ble://F4:12:73:29:4E:89
```

(continues on next page)

(continued from previous page)

```
# Or connect to the first available,
# paired device
# port = ble://

# Set frequency to 867.2 MHz
frequency = 867200000

# Set LoRa bandwidth to 125 KHz
bandwidth = 125000

# Set TX power to 7 dBm (5 mW)
txpower = 7

# Select spreading factor 8. Valid
# range is 7 through 12, with 7
# being the fastest and 12 having
# the longest range.
spreadingfactor = 8

# Select coding rate 5. Valid range
# is 5 through 8, with 5 being the
# fastest, and 8 the longest range.
codingrate = 5

# You can configure the RNode to send
# out identification on the channel with
# a set interval by configuring the
# following two parameters.

# id_callsign = MYCALL-0
# id_interval = 600

# For certain homebrew RNode interfaces
# with low amounts of RAM, using packet
# flow control can be useful. By default
# it is disabled.

# flow_control = False

# It is possible to limit the airtime
# utilisation of an RNode by using the
# following two configuration options.
# The short-term limit is applied in a
# window of approximately 15 seconds,
# and the long-term limit is enforced
# over a rolling 60 minute window. Both
# options are specified in percent.

# airtime_limit_long = 1.5
# airtime_limit_short = 33
```

8.9 RNode Multi Interface

For RNodes that support multiple LoRa transceivers, the RNode Multi interface can be used to configure sub-interfaces individually.

Warning

Radio frequency spectrum is a legally controlled resource, and legislation varies widely around the world. It is your responsibility to be aware of any relevant regulation for your location, and to make decisions accordingly.

```
# Here's an example of how to add an RNode Multi interface
# using the RNode LoRa transceiver.

[[RNode Multi Interface]]
type = RNodeMultiInterface

# Enable interface if you want to use it!
enabled = yes

# Serial port for the device
port = /dev/ttyACM0

# You can configure the RNode to send
# out identification on the channel with
# a set interval by configuring the
# following two parameters.

# id_callsign = MYCALL-0
# id_interval = 600

# A subinterface
[[[High Datarate]]]
# Subinterfaces can be enabled and disabled in of themselves
enabled = yes

# Set frequency to 2.4GHz
frequency = 2400000000

# Set LoRa bandwidth to 1625 KHz
bandwidth = 1625000

# Set TX power to 0 dBm (0.12 mW)
txpower = 0

# The virtual port, only the manufacturer
# or the person who wrote the board config
# can tell you what it will be for which
# physical hardware interface
vport = 1

# Select spreading factor 5. Valid
# range is 5 through 12, with 5
```

(continues on next page)

(continued from previous page)

```

# being the fastest and 12 having
# the longest range.
spreadingfactor = 5

# Select coding rate 5. Valid range
# is 5 through 8, with 5 being the
# fastest, and 8 the longest range.
codingrate = 5

# It is possible to limit the airtime
# utilisation of an RNode by using the
# following two configuration options.
# The short-term limit is applied in a
# window of approximately 15 seconds,
# and the long-term limit is enforced
# over a rolling 60 minute window. Both
# options are specified in percent.

# airtime_limit_long = 100
# airtime_limit_short = 100

```

[[[Low Datarate]]]

```

# Subinterfaces can be enabled and disabled in of themselves
enabled = yes

# Set frequency to 865.6 MHz
frequency = 865600000

# The virtual port, only the manufacturer
# or the person who wrote the board config
# can tell you what it will be for which
# physical hardware interface
vport = 0

# Set LoRa bandwidth to 125 KHz
bandwidth = 125000

# Set TX power to 0 dBm (0.12 mW)
txpower = 0

# Select spreading factor 7. Valid
# range is 5 through 12, with 5
# being the fastest and 12 having
# the longest range.
spreadingfactor = 7

# Select coding rate 5. Valid range
# is 5 through 8, with 5 being the
# fastest, and 8 the longest range.
codingrate = 5

# It is possible to limit the airtime

```

(continues on next page)

(continued from previous page)

```

# utilisation of an RNode by using the
# following two configuration options.
# The short-term limit is applied in a
# window of approximately 15 seconds,
# and the long-term limit is enforced
# over a rolling 60 minute window. Both
# options are specified in percent.

# airtime_limit_long = 100
# airtime_limit_short = 100

```

8.10 Serial Interface

Reticulum can be used over serial ports directly, or over any device with a serial port, that will transparently pass data. Useful for communicating directly over a wire-pair, or for using devices such as data radios and lasers.

```

[[Serial Interface]]
type = SerialInterface
enabled = yes

# Serial port for the device
port = /dev/ttyUSB0

# Set the serial baud-rate and other
# configuration parameters.
speed = 115200
databits = 8
parity = none
stopbits = 1

```

8.11 Pipe Interface

Using this interface, Reticulum can use any program as an interface via *stdin* and *stdout*. This can be used to easily create virtual interfaces, or to interface with custom hardware or other systems.

```

[[Pipe Interface]]
type = PipeInterface
enabled = yes

# External command to execute
command = netcat -l 5757

# Optional respawn delay, in seconds
respawn_delay = 5

```

Reticulum will write all packets to *stdin* of the *command* option, and will continuously read and scan its *stdout* for Reticulum packets. If EOF is reached, Reticulum will try to respawn the program after waiting for *respawn_interval* seconds.

8.12 KISS Interface

With the KISS interface, you can use Reticulum over a variety of packet radio modems and TNCs, including [OpenModem](#). KISS interfaces can also be configured to periodically send out beacons for station identification purposes.

Warning

Radio frequency spectrum is a legally controlled resource, and legislation varies widely around the world. It is your responsibility to be aware of any relevant regulation for your location, and to make decisions accordingly.

[[Packet Radio KISS Interface]]

```
type = KISSInterface
enabled = yes

# Serial port for the device
port = /dev/ttyUSB1

# Set the serial baud-rate and other
# configuration parameters.
speed = 115200
databits = 8
parity = none
stopbits = 1

# Set the modem preamble.
preamble = 150

# Set the modem TX tail.
txtail = 10

# Configure CDMA parameters. These
# settings are reasonable defaults.
persistence = 200
slottime = 20

# You can configure the interface to send
# out identification on the channel with
# a set interval by configuring the
# following two parameters. The KISS
# interface will only ID if the set
# interval has elapsed since it's last
# actual transmission. The interval is
# configured in seconds.
# This option is commented out and not
# used by default.
# id_callsign = MYCALL-0
# id_interval = 600

# Whether to use KISS flow-control.
# This is useful for modems that have
# a small internal packet buffer, but
# support packet flow control instead.
```

(continues on next page)

(continued from previous page)

```
flow_control = false
```

8.13 AX.25 KISS Interface

If you're using Reticulum on amateur radio spectrum, you might want to use the AX.25 KISS interface. This way, Reticulum will automatically encapsulate it's traffic in AX.25 and also identify your stations transmissions with your callsign and SSID.

Only do this if you really need to! Reticulum doesn't need the AX.25 layer for anything, and it incurs extra overhead on every packet to encapsulate in AX.25.

A more efficient way is to use the plain KISS interface with the beaconing functionality described above.

Warning

Radio frequency spectrum is a legally controlled resource, and legislation varies widely around the world. It is your responsibility to be aware of any relevant regulation for your location, and to make decisions accordingly.

[[Packet Radio AX.25 KISS Interface]]

```
type = AX25KISSInterface

# Set the station callsign and SSID
callsign = N01CLL
ssid = 0

# Enable interface if you want use it!
enabled = yes

# Serial port for the device
port = /dev/ttyUSB2

# Set the serial baud-rate and other
# configuration parameters.
speed = 115200
databits = 8
parity = none
stopbits = 1

# Set the modem preamble. A 150ms
# preamble should be a reasonable
# default, but may need to be
# increased for radios with slow-
# opening squelch and long TX/RX
# turnaround
preamble = 150

# Set the modem TX tail. In most
# cases this should be kept as low
# as possible to not waste airtime.
txtail = 10
```

(continues on next page)

(continued from previous page)

```
# Configure CDMA parameters. These
# settings are reasonable defaults.
persistence = 200
slottime = 20

# Whether to use KISS flow-control.
# This is useful for modems with a
# small internal packet buffer.
flow_control = false
```

8.14 Discoverable Interfaces

Reticulum includes a powerful system for publishing your local interfaces to the wider network, allowing other peers to *discover, validate, and automatically connect to them*. This feature is particularly useful for creating decentralized networks where peers can dynamically find endpoints, such as public Internet gateways or local radio access points, without relying on static configuration files or centralized directories.

When an interface is made **discoverable**, your Reticulum instance will periodically broadcast an announce packet containing the connection details and parameters required for other peers to establish a connection. These announces are propagated over the network using the standard Reticulum announce mechanism using the `rnstransport.discovery.interface` destination type.

Note

To use the interface discovery functionality, the LXMf module must be installed in your Python environment. You can install it using pip:

```
pip install lxmfm
```

8.14.1 Enabling Discovery

Interface discovery is enabled on a per-interface basis. To make a specific interface discoverable, you must add the `discoverable` option to that interface's configuration block and set it to `yes`.

```
[[My Public Gateway]]
type = BackboneInterface
...
discoverable = yes
```

Once enabled, Reticulum will automatically handle the generation, signing, stamping, and broadcasting of the discovery announces. It is not *required* to enable Transport to publish interface discovery information, but for most use cases where you want others to connect to you, you will likely want `enable_transport` set to `yes` in the `[reticulum]` section of your configuration.

8.14.2 Discovery Parameters

When `discoverable` is enabled, a variety of additional options become available to control how the interface is presented to the network. These parameters allow you to fine-tune the metadata, security requirements, and visibility of your interface.

Basic Metadata

discovery_name

A human-readable name for the interface. This name will be displayed to users on remote systems when they list discovered interfaces. If not specified, the interface name (the section header) will be used.

announce_interval

The interval in minutes between successive discovery announces for this interface. Default is 360 minutes (6 hours). For stable, long-running infrastructure, higher intervals (12 to 22 hours) are usually sufficient and reduce network load. Minimum allowed value is 5 minutes (but expect to have your announces throttled if using intervals below one hour).

Connectivity Specification**reachable_on**

Specifies the address that remote peers should use to connect to this interface.

- For TCP and Backbone interfaces, this is typically the public IP address or hostname. Do not include the port, this is fetched automatically from the interface.
- For I2P interfaces, this is usually the I2P b32 address. This value is fetched automatically from the `I2PInterface` once it is up and connected to the I2P network, so you should not set this manually, unless you absolutely know what you're doing.

Dynamic Resolution: This option also accepts a path to an external executable script or binary. If a path is provided, Reticulum will execute the script and use its `stdout` as the reachability address. This is useful for devices behind dynamic DNS, NATs, or complex cloud environments where the external IP is not known locally. The script must simply print the address to `stdout` and exit.

Note

When using an executable script for `reachable_on`, Reticulum expects the script to output only the IP address or hostname to `stdout`, followed by a newline character. Any additional output or errors may cause the resolution to fail. Ensure the script has executable permissions and is robust against temporary network failures.

A minimal example of a script that resolves the externally available, public IP of an internet-connected system could look like this:

```
#!/bin/bash
curl -s ip.me
exit $?
```

On a real system, you should make the script robust enough to deal with intermittent Internet or service failures, such that the script *always* returns a sensible value, or if not possible at least exits with a non-zero exit return code, so Reticulum knows the output is invalid.

Security & Cost**discovery_stamp_value**

Defines the proof-of-work difficulty for the cryptographic stamp included in the announce. This value acts as a cost barrier to prevent network flooding. The default value is 14. Increasing this value makes it computationally more expensive to generate an announce, which can be useful to prevent spam on very large networks, but it also increases CPU load on your system when generating announces. Stamps are cached, and only generated if interface information changes, or at instance restart. If you have the computational resources, it is generally advisable to use as high a stamp value as possible.

Privacy & Encryption**discovery_encrypt**

If set to yes, the discovery announce payload will be encrypted. To decrypt the announce, remote peers must

possess the *network identity* configured for your instance (see `network_identity` in the [reticulum] section). This allows you to publish private interfaces that are only discoverable to specific trusted networks.

Important

If you enable `discovery_encrypt` but do not configure a valid `network_identity` in the [reticulum] section of your configuration, Reticulum will abort the interface discovery announce. Encryption requires a valid network identity key to function.

publish_ifac

If set to `yes`, the Interface Access Code (IFAC) name and passphrase for this interface will be included in the discovery announce. This allows peers to automatically configure the correct authentication parameters when connecting to the interface.

Physical Location**latitude, longitude, height**

Optional physical coordinates for the interface. These are useful for mapping discovered interfaces geographically or for clients to automatically select the nearest access point. Coordinates should be in decimal degrees, height in meters.

Radio Parameters

For physical radio interfaces like `RNodeInterface` or `KISSInterface`, the following optional parameters allow you to broadcast the operating frequency and characteristics, allowing clients to verify compatibility before connecting:

discovery_frequency

The operating frequency in Hz. Auto-configured on `RNode` interfaces. Necessary on KISS-based radio interfaces and `TCPClientInterfaces` connecting to radio modems.

discovery_bandwidth

The signal bandwidth in Hz. Auto-configured on `RNode` interfaces. Useful on KISS-based radio interfaces and `TCPClientInterfaces` connecting to radio modems.

discovery_modulation

The modulation type or scheme. Auto-configured on `RNode` interfaces, but highly advisable to include on other radio-based interfaces.

8.14.3 Interface Modes

When you enable discovery on an interface, Reticulum enforces certain interface modes to ensure the interface is actually useful for remote peers.

If an interface is configured as `discoverable`, but its mode is not explicitly set to `gateway` (for server-style interfaces like `BackboneInterface` or `TCPServerInterface`) or `access_point` (for radio interfaces like `RNodeInterface`), Reticulum will automatically configure the appropriate mode and log a notice.

For example, if you enable discovery on a `RNodeInterface` without specifying the mode, Reticulum will automatically set it to `access_point` mode.

8.14.4 Security Considerations

When making interfaces discoverable, you are effectively broadcasting an invitation to connect to your system. It is important to understand the security implications of the configuration options you choose.

Publishing Credentials

If you enable `publish_ifac = yes`, your interface's authentication passphrase will be included in the announce. If you are operating a public network and want anyone to connect, this is acceptable. However, if you wish to restrict access

to a specific group of users, you **must** enable `discovery_encrypt = yes`. This ensures that only peers possessing the correct `network_identity` can decode the passphrase.

Topology Exposure

A discoverable interface announces its presence, location (if configured), and capabilities to the network. Even if the connection details are encrypted, the *fact* that a connectable node exists within a certain network becomes public information. In high-security or scenarios requiring operational secrecy, consider the implications of advertising your infrastructure's existence.

8.14.5 Example Configuration

Below is an example configuration for a public backbone gateway. This configuration publishes a high-value, publicly discoverable interface, that anyone can connect to.

```
[[My Public Gateway]]
  type = BackboneInterface
  mode = gateway
  listen_on = 0.0.0.0
  port = 4242

  # Enable Discovery
  discoverable = yes

  # Interface Details
  discovery_name = Region A Public Entrypoint
  announce_interval = 720

  # Use external script to resolve dynamic IP
  reachable_on = /usr/local/bin/get_external_ip.sh

  # Generate high stamp value
  discovery_stamp_value = 24

  # Optional location data
  latitude = 51.99714
  longitude = -0.74195
  height = 15
```

The next example create an encrypted discovery-enabled interface, requiring a specific network identity to decode, and includes IFAC credentials for seamless authentication.

```
[[My Private Gateway]]
  type = BackboneInterface
  mode = gateway
  listen_on = 0.0.0.0
  port = 5858
  network_name = internal_1
  passphrase = Mevpekyafshak5Wr

  # Enable Discovery
  discoverable = yes

  # Interface Details
  discovery_name = Region A Private Backbone
```

(continues on next page)

(continued from previous page)

```
announce_interval = 720

# Use external script to resolve dynamic IP
reachable_on = /usr/local/bin/get_external_ip.sh

# Target stamp value
discovery_stamp_value = 22

# Encrypt announces for our network only
discovery_encrypt = yes

# Include credentials so trusted
# peers can connect automatically
publish_ifac = yes

# Optional location data
latitude = 34.06915
longitude = -118.44318
height = 15
```

In the `[reticulum]` section of your configuration, you would define the network identity used for encryption as follows:

```
[reticulum]
...
# The identity used to sign/encrypt discovery announces
network_identity = ~/.reticulum/storage/identities/my_network_identity
...
```

With these configuration options applied, your Reticulum instance will actively participate in the network's discovery ecosystem. Other peers running Reticulum with discovery enabled will be able to see your interface, validate its cryptographic stamp, and (depending on their configuration) automatically connect to it.

For information on how to use these discovered interfaces and configure your system to auto-connect to them, refer to the *Discovering Interfaces* chapter.

8.15 Common Interface Options

A number of general configuration options are available on most interfaces. These can be used to control various aspects of interface behaviour.

- The `enabled` option tells Reticulum whether or not to bring up the interface. Defaults to `False`. For any interface to be brought up, the `enabled` option must be set to `True` or `Yes`.
- The `mode` option allows selecting the high-level behaviour of the interface from a number of options.
 - The default value is `full`. In this mode, all discovery, meshing and transport functionality is available.
 - In the `access_point` (or shorthand `ap`) mode, the interface will operate as a network access point. In this mode, announces will not be automatically broadcasted on the interface, and paths to destinations on the interface will have a much shorter expiry time. This mode is useful for creating interfaces that are mostly quiet, unless when someone is actually using them. An example of this could be a radio interface serving a wide area, where users are expected to connect momentarily, use the network, and then disappear again.
- The `outgoing` option sets whether an interface is allowed to transmit. Defaults to `True`. If set to `False` or `No` the interface will only receive data, and never transmit.

- The `network_name` option sets the virtual network name for the interface. This allows multiple separate network segments to exist on the same physical channel or medium.
- The `passphrase` option sets an authentication passphrase on the interface. This option can be used in conjunction with the `network_name` option, or be used alone.
- The `ifac_size` option allows customising the length of the Interface Authentication Codes carried by each packet on named and/or authenticated network segments. It is set by default to a size suitable for the interface in question, but can be set to a custom size between 8 and 512 bits by using this option. In normal usage, this option should not be changed from the default.
- The `announce_cap` option lets you configure the maximum bandwidth to allocate, at any given time, to propagating announces and other network upkeep traffic. It is configured at 2% by default, and should normally not need to be changed. Can be set to any value between 1 and 100.

If an interface exceeds its announce cap, it will queue announces for later transmission. Reticulum will always prioritise propagating announces from nearby nodes first. This ensures that the local topology is prioritised, and that slow networks are not overwhelmed by interconnected fast networks.

Destinations that are rapidly re-announcing will be down-prioritised further. Trying to get “first-in-line” by announce spamming will have the exact opposite effect: Getting moved to the back of the queue every time a new announce from the excessively announcing destination is received.

This means that it is always beneficial to select a balanced announce rate, and not announce more often than is actually necessary for your application to function.

- The `bitrate` option configures the interface bitrate. Reticulum will use interface speeds reported by hardware, or try to guess a suitable rate when the hardware doesn't report any. In most cases, the automatically found rate should be sufficient, but it can be configured by using the `bitrate` option, to set the interface speed in *bits per second*.
- The `bootstrap_only` option designates an interface as a temporary bridge for initial connectivity. If this option is enabled, the interface will be monitored and automatically detached once the number of auto-connected interfaces reaches the limit configured by `autoconnect_discovered_interfaces`. This is particularly useful for using a slow or expensive connection (such as a single LoRa link or a remote TCP tunnel) solely to discover better local infrastructure, which then supersedes the bootstrap interface.

8.16 Interface Modes

The optional `mode` setting is available on all interfaces, and allows selecting the high-level behaviour of the interface from a number of modes. These modes affect how Reticulum selects paths in the network, how announces are propagated, how long paths are valid and how paths are discovered.

Configuring modes on interfaces is **not** strictly necessary, but can be useful when building or connecting to more complex networks. If your Reticulum instance is not running a Transport Node, it is rarely useful to configure interface modes, and in such cases interfaces should generally be left in the default mode.

- The default mode is `full`. In this mode, all discovery, meshing and transport functionality is activated.
- The `gateway` mode (or shorthand `gw`) also has all discovery, meshing and transport functionality available, but will additionally try to discover unknown paths on behalf of other nodes residing on the `gateway` interface. If Reticulum receives a path request for an unknown destination, from a node on a `gateway` interface, it will try to discover this path via all other active interfaces, and forward the discovered path to the requestor if one is found.

If you want to allow other nodes to widely resolve paths or connect to a network via an interface, it might be useful to put it in this mode. By creating a chain of `gateway` interfaces, other nodes will be able to immediately discover paths to any destination along the chain.

Please note! It is the interface *facing the clients* that must be put into gateway mode for this to work, not the interface facing the wider network (for this, the boundary mode can be useful, though).

- In the `access_point` (or shorthand `ap`) mode, the interface will operate as a network access point. In this mode, announces will not be automatically broadcasted on the interface, and paths to destinations on the interface will have a much shorter expiry time. In addition, path requests from clients on the access point interface will be handled in the same way as the gateway interface.

This mode is useful for creating interfaces that remain quiet, until someone actually starts using them. An example of this could be a radio interface serving a wide area, where users are expected to connect momentarily, use the network, and then disappear again.

- The `roaming` mode should be used on interfaces that are roaming (physically mobile), seen from the perspective of other nodes in the network. As an example, if a vehicle is equipped with an external LoRa interface, and an internal, WiFi-based interface, that serves devices that are moving *with* the vehicle, the external LoRa interface should be configured as `roaming`, and the internal interface can be left in the default mode. With transport enabled, such a setup will allow all internal devices to reach each other, and all other devices that are available on the LoRa side of the network, when they are in range. Devices on the LoRa side of the network will also be able to reach devices internal to the vehicle, when it is in range. Paths via `roaming` interfaces also expire faster.
- The purpose of the `boundary` mode is to specify interfaces that establish connectivity with network segments that are significantly different than the one this node exists on. As an example, if a Reticulum instance is part of a LoRa-based network, but also has a high-speed connection to a public Transport Node available on the Internet, the interface connecting over the Internet should be set to boundary mode.

For a table describing the impact of all modes on announce propagation, please see the [Announce Propagation Rules](#) section.

8.17 Announce Rate Control

The built-in announce control mechanisms and the default `announce_cap` option described above are sufficient most of the time, but in some cases, especially on fast interfaces, it may be useful to control the target announce rate. Using the `announce_rate_target`, `announce_rate_grace` and `announce_rate_penalty` options, this can be done on a per-interface basis, and moderates the *rate at which received announces are re-broadcasted to other interfaces*.

- The `announce_rate_target` option sets the minimum amount of time, in seconds, that should pass between received announces, for any one destination. As an example, setting this value to `3600` means that announces *received* on this interface will only be re-transmitted and propagated to other interfaces once every hour, no matter how often they are received.
- The optional `announce_rate_grace` defines the number of times a destination can violate the announce rate before the target rate is enforced.
- The optional `announce_rate_penalty` configures an extra amount of time that is added to the normal rate target. As an example, if a penalty of `7200` seconds is defined, once the rate target is enforced, the destination in question will only have its announces propagated every 3 hours, until it lowers its actual announce rate to within the target.

These mechanisms, in conjunction with the `annouce_cap` mechanisms mentioned above means that it is essential to select a balanced announce strategy for your destinations. The more balanced you can make this decision, the easier it will be for your destinations to make it into slower networks that many hops away. Or you can prioritise only reaching high-capacity networks with more frequent announces.

Current statistics and information about announce rates can be viewed using the `rnpath -r` command.

It is important to note that there is no one right or wrong way to set up announce rates. Slower networks will naturally tend towards using less frequent announces to conserve bandwidth, while very fast networks can support applications that need very frequent announces. Reticulum implements these mechanisms to ensure that a large span of network types can seamlessly *co-exist* and interconnect.

8.18 New Destination Rate Limiting

On public interfaces, where anyone may connect and announce new destinations, it can be useful to control the rate at which announces for *new* destinations are processed.

If a large influx of announces for newly created or previously unknown destinations occur within a short amount of time, Reticulum will place these announces on hold, so that announce traffic for known and previously established destinations can continue to be processed without interruptions.

After the burst subsides, and an additional waiting period has passed, the held announces will be released at a slow rate, until the hold queue is cleared. This also means, that should a node decide to connect to a public interface, announce a large amount of bogus destinations, and then disconnect, these destination will never make it into path tables and waste network bandwidth on retransmitted announces.

It's important to note that the ingress control works at the level of *individual sub-interfaces*. As an example, this means that one client on a *TCP Server Interface* cannot disrupt processing of incoming announces for other connected clients on the same *TCP Server Interface*. All other clients on the same interface will still have new announces processed without interruption.

By default, Reticulum will handle this automatically, and ingress announce control will be enabled on interface where it is sensible to do so. It should generally not be necessary to modify the ingress control configuration, but all the parameters are exposed for configuration if needed.

- The `ingress_control` option tells Reticulum whether or not to enable announce ingress control on the interface. Defaults to `True`.
- The `ic_new_time` option configures how long (in seconds) an interface is considered newly spawned. Defaults to `2*60*60` seconds. This option is useful on publicly accessible interfaces that spawn new sub-interfaces when a new client connects.
- The `ic_burst_freq_new` option sets the maximum announce ingress frequency for newly spawned interfaces. Defaults to `3.5` announces per second.
- The `ic_burst_freq` option sets the maximum announce ingress frequency for other interfaces. Defaults to `12` announces per second.

If an interface exceeds its burst frequency, incoming announces for unknown destinations will be temporarily held in a queue, and not processed until later.

- The `ic_max_held_announces` option sets the maximum amount of unique announces that will be held in the queue. Any additional unique announces will be dropped. Defaults to `256` announces.
- The `ic_burst_hold` option sets how much time (in seconds) must pass after the burst frequency drops below its threshold, for the announce burst to be considered cleared. Defaults to `60` seconds.
- The `ic_burst_penalty` option sets how much time (in seconds) must pass after the burst is considered cleared, before held announces can start being released from the queue. Defaults to `5*60` seconds.
- The `ic_held_release_interval` option sets how much time (in seconds) must pass between releasing each held announce from the queue. Defaults to `30` seconds.

BUILDING NETWORKS

This chapter will provide you with the high-level knowledge needed to build networks with Reticulum. It will not, however tell you all you need to know to successfully design and configure every kind of network you can imagine. For this, you will most likely need to read this manual in its entirety, invest significant time into experimenting with the stack, and learning functionality intuitively.

Still, after reading this chapter, you should be well equipped to *start* that journey. While Reticulum is **fundamentally different** compared to other networking technologies, it can often be easier than using traditional stacks. If you’ve built networks before, you will probably have to forget, or at least temporarily ignore, a lot of things at this point. It will all make sense in the end though. Hopefully.

If you’re used to protocols like IP, let’s at least start with some relief: You don’t have to worry about coordinating addresses, subnets and routing for an entire network that you might not know how will evolve in the future. With Reticulum, you can simply add more segments to your network when it becomes necessary, and Reticulum will handle the convergence of the entire network automatically. There’s plenty more neat aspects like that to Reticulum, but we’re getting ahead of ourselves. Let’s cover the basics first.

9.1 Concepts & Overview

Before you start building your own networks, it’s important to understand the fundamental principles that distinguish Reticulum networks from traditional networking approaches. These principles shape how you design your network, what trade-offs you encounter, and what capabilities you can rely on.

Reticulum is not a single network you “join”, it is a toolkit for *creating* networks. You decide what mediums to use, how nodes connect, what trust boundaries exist, and what the network’s purpose is. Reticulum provides the cryptographic foundation, the transport mechanisms, and the convergence algorithms that make your design workable. You provide the intent and the structure.

This approach offers tremendous flexibility, but it requires thinking in terms of different abstractions than those used in conventional networking.

9.1.1 Introductory Considerations

There are important points that need to be kept in mind when building networks with Reticulum:

- In a Reticulum network, any node can autonomously generate as many addresses (called *destinations* in Reticulum terminology) as it needs, which become globally reachable to the rest of the network. There is no central point of control over the address space.
- Reticulum was designed to handle both very small, and very large networks. While the address space can support billions of endpoints, Reticulum is also very useful when just a few devices need to communicate.
- Low-bandwidth networks, like LoRa and packet radio, can interoperate and interconnect with much larger and higher bandwidth networks without issue. Reticulum automatically manages the flow of information to and from various network segments, and when bandwidth is limited, local traffic is prioritised. You will, however,

need to configure your interfaces correctly. If you tell Reticulum to pass all announce traffic from a gigabit link to a LoRa interfaces, it will try as best as possible to comply with this, while still respecting bandwidth limits, but you *will* waste a lot of precious bandwidth and airtime, and your LoRa network will not work very well.

- Reticulum provides sender/initiator anonymity by default. There is no way to filter traffic or discriminate it based on the source of the traffic.
- All traffic is encrypted using ephemeral keys generated by an Elliptic Curve Diffie-Hellman key exchange on Curve25519. There is no way to inspect traffic contents, and no way to prioritise or throttle certain kinds of traffic. All transport and routing layers are thus completely agnostic to traffic type, and will pass all traffic equally.
- Reticulum can function both with and without infrastructure. When *transport nodes* are available, they can route traffic over multiple hops for other nodes, and will function as a distributed cryptographic keystore. When there is no transport nodes available, all nodes that are within communication range can still communicate.
- Every node can become a transport node, simply by enabling it in it's configuration, but there is no need for every node on the network to be a transport node. Letting every node be a transport node will in most cases degrade the performance and reliability of the network.

In general terms, if a node is stationary, well-connected and kept running most of the time, it is a good candidate to be a transport node. For optimal performance, a network should contain the amount of transport nodes that provides connectivity to the intended area / topography, and not many more than that.

- Reticulum is designed to work reliably in open, trustless environments. This means you can use it to create open-access networks, where participants can join and leave in a free and unorganised manner. This property allows an entirely new, and so far, mostly unexplored class of networked applications, where networks, and the information flow within them can form and dissolve organically.
- You can just as easily create closed networks, since Reticulum allows you to add authentication to any interface. This means you can restrict access on any interface type, even when using legacy devices, such as modems. You can also mix authenticated and open interfaces on the same system. See the [Common Interface Options](#) section of the [Interfaces](#) chapter of this manual for information on how to set up interface authentication.

Reticulum allows you to mix very different kinds of networking mediums into a unified mesh, or to keep everything within one medium. You could build a “virtual network” running entirely over the Internet, where all nodes communicate over TCP and UDP “channels”. You could also build such a network using other already-established communications channels as the underlying carrier for Reticulum.

However, most real-world networks will probably involve either some form of wireless or direct hardline communications. To allow Reticulum to communicate over any type of medium, you must specify it in the configuration file, by default located at `~/.reticulum/config`. See the [Supported Interfaces](#) chapter of this manual for interface configuration examples.

Any number of interfaces can be configured, and Reticulum will automatically decide which are suitable to use in any given situation, depending on where traffic needs to flow.

9.1.2 Destinations, Not Addresses

In traditional networking, addresses are allocated from a managed space. If you want to communicate with another node, you need to know its address, and that address must be unique within the network segment. This requires coordination, either through manual assignment, DHCP servers, or other allocation mechanisms.

Reticulum replaces addresses with **destinations**. A destination is identified by a 16-byte hash (128 bits) derived from a SHA-256 hash of the destination's identifying characteristics. This hash serves as the address on the network. On the network, it is represented in binary, but when displayed to human users, it will usually look something like this `<13425ec15b621c1d928589718000d814>`.

The critical difference is that *any node can generate as many destinations as it needs, without coordination*. A destination's uniqueness is guaranteed by the collision resistance of SHA-256 and the inclusion of the node's public key in the hash calculation. Two nodes can both use the destination name `messenger.user.inbox`, but they will have different destination hashes because their public keys differ. Both can coexist on the same network without conflict.

This has profound implications for network design:

- **No address allocation planning:** You never need to reserve address ranges, plan subnets, or coordinate with other network operators. Nodes simply generate destinations and announce them.
- **Global portability:** A destination is not tied to a physical location or network segment. A node can move its destinations across interfaces, mediums, or even between entirely separate Reticulum networks simply by sending an announce on the new medium.
- **Implicit authentication:** Because destinations are bound to public keys, communication to a destination is inherently cryptographically authenticated. Only the holder of the corresponding private key can decrypt and respond to traffic addressed to that destination. This also makes application-level authentication *much* simpler, since it can directly use the foundational identity verification built into the core networking layer.
- **Identity abstraction:** A single Reticulum Identity can create multiple destinations. This allows a single entity (a person, a device, a service) to present multiple endpoints without needing multiple cryptographic keypairs.

9.1.3 Transport Nodes and Instances

Reticulum distinguishes between two types of nodes: **Instances** and **Transport Nodes**. Every node running Reticulum is an Instance, but not every Instance is a Transport Node.

A **Reticulum Instance** is any system running the Reticulum stack. It can create destinations, send and receive packets, establish links, and communicate with other nodes. It can also host destinations that are connectable for *anyone* else in the network. This means you can easily host globally available services from any location, including your home or office. Network-wide, global connectivity for all destinations is guaranteed, as long as there is *some* physical way to actually transport the packets. Instances are the default state and are appropriate for most end-user devices, such as phones, laptops, sensors, or any device that primarily consumes network services.

A **Transport Node** is an Instance that has been explicitly configured to participate in network-wide transport. Transport nodes forward packets across hops, propagate announces, maintain path tables, and serve path requests on behalf of other nodes. When a destination sends an announce, Transport Nodes receive it, remember the path, and rebroadcast it to other interfaces. When a node needs to reach a destination it doesn't have a path for, Transport Nodes help resolve the path through the network.

Even devices hosting services or serving content should probably just be configured as instances, and themselves connect to wider networks via a Transport Node. In some situations, this may not be practical though, and as an example, it is entirely viable to host a personal Transport Node on a Raspberry Pi, while it is at the same time running an LXMF propagation node, and hosting your personal site or files over Reticulum.

The distinction is important. **Not** every node should be a Transport Node:

- **Resource consumption:** Transport nodes maintain path tables, process announces, and forward traffic. This requires memory and CPU resources that may be limited on low-powered devices.
- **Stability requirements:** Transport nodes contribute to network convergence. If Transport Nodes frequently go offline, path tables become stale and convergence suffers. Stable, always-on nodes make better Transport Nodes.
- **Bandwidth considerations:** Transport nodes process and rebroadcast network maintenance traffic. On very low-bandwidth mediums, having too many Transport Nodes will consume capacity that should be used for actual data.

In practice, a network typically has a relatively small number of Transport Nodes strategically placed to provide coverage and connectivity. End-user devices run as Instances, connecting through nearby Transport Nodes to reach the

wider network. This pattern mirrors traditional networking where routers forward traffic while end hosts simply consume connectivity, but with the crucial difference that any node *can* become a router if needed, and the decision is yours to make based on your network's requirements.

Transport nodes also function as distributed cryptographic keystores. When a destination announces itself, Transport Nodes cache the public key and destination information. Other nodes can request unknown public keys from the network, and Transport Nodes respond with the cached information. This eliminates the need for a central directory service while ensuring that public keys remain available throughout the network.

9.1.4 Trustless Networking

Traditional network security models assume high levels of trust at specific layers. You might trust your ISP to deliver packets without inspection, or trust your VPN provider to handle your traffic, or trust the network administrator to configure firewalls appropriately. These trust relationships create vulnerabilities and dependencies.

Reticulum is designed to function in **open, trustless environments**. This means the protocol makes no assumptions about the trustworthiness of the network infrastructure, the other participants, or the transport mediums. Every aspect of communication is secured cryptographically:

- **Traffic encryption:** All traffic to single destinations is encrypted using ephemeral keys.
- **Source anonymity:** Reticulum packets do not include source addresses. An observer intercepting a packet cannot determine who sent it, only who it is addressed to (unless IFAC is enabled, in which case nothing can be determined). This provides initiator anonymity by default.
- **Path verification:** The announce mechanism includes cryptographic signatures that prove the authenticity of destination announcements.
- **Unforgeable delivery confirmations:** When a destination proves receipt of a packet, the proof is signed with the destination's identity key. This prevents false acknowledgments and ensures reliable delivery verification.
- **Interface authentication:** When using Interface Access Codes (IFAC), packets on authenticated interfaces carry signatures derived from a shared secret. Only nodes with the correct network name and passphrase can generate valid packets, allowing creation of virtual private networks on shared mediums.

The trustless design has important consequences for network design:

- **Open-access networks are viable:** You can build networks that anyone can join without pre-approval. Because traffic is encrypted and authenticated end- to-end, participants cannot interfere with each other's private communication, even if they share the same transport infrastructure.
- **No traffic inspection or prioritization:** Because traffic contents and sources are opaque to intermediate nodes, there is no mechanism for filtering, prioritizing, or throttling traffic based on its type or origin. All traffic is treated equally. From a neutrality perspective, this is a feature.
- **Adversarial resilience:** The network can operate even if some nodes are malicious or controlled by adversaries. While a malicious Transport Node could refuse to forward certain traffic or drop packets, it cannot decrypt, modify, or impersonate legitimate traffic. Redundant paths and multiple Transport Nodes mitigate the impact of malicious nodes.

Of course, you can also create closed networks. Interface Access Codes allow you to restrict participation on specific interfaces. Network Identities enable you to verify that discovered interfaces belong to trusted operators. Blackhole management lets you block malicious identities. Reticulum provides both the tools for open networks and the controls for closed ones. The choice is yours based on your requirements.

9.1.5 Heterogeneous Connectivity

In conventional networking, mixing different transport mediums typically requires gateways, translation layers, and careful configuration. A WiFi network doesn't natively interoperate with a packet radio network without additional infrastructure, and you can't just download a car over a serial port, or send an encrypted message in a QR code.

Reticulum treats **heterogeneity as a core premise**. The protocol is designed to seamlessly mix mediums with vastly different characteristics:

- **Bandwidth:** LoRa links operating at a few hundred bits per second can interconnect with gigabit Ethernet backbones. Reticulum automatically manages the flow of information, prioritizing local traffic on slow segments while allowing global convergence.
- **Latency:** Satellite links with multi-second latency can coexist with local links measured in milliseconds. The transport system handles timing, asynchronous delivery and retransmissions transparently.
- **Topology:** Point-to-point microwave links, broadcast radio networks, switched Ethernet fabrics, and virtual tunnels over the Internet can all be part of the same Reticulum network.
- **Reliability:** Intermittent connections that come and go (such as mobile devices or opportunistic radio contacts) can participate alongside always-on infrastructure. Reticulum gracefully handles link loss and reconnection.

This heterogeneity is achieved through several design elements:

- **Expandable, medium-agnostic interface system:** Reticulum communicates with the physical world through interface modules. Adding support for a new medium is a matter of implementing an interface class. The protocol itself remains unchanged.
- **Interface modes:** Different modes (`full`, `gateway`, `access_point`, `roaming`, `boundary`) allow you to configure how interfaces interact with the wider network based on their characteristics and role.
- **Announce propagation rules:** Announces are forwarded between interfaces according to rules that account for bandwidth limitations and interface modes. Slow segments are not overwhelmed by traffic from fast segments.
- **Local traffic prioritization:** When bandwidth is constrained, Reticulum prioritizes announces for nearby destinations. This ensures that local connectivity remains functional even when global convergence is incomplete.

For network designers, this means you are free to use whatever mediums are available, affordable, or appropriate for your situation. You might use LoRa for wide-area low-bandwidth coverage, WiFi for local high-capacity links, I2P for anonymous Internet connectivity, and Ethernet for infrastructure backhauls, all within the same network. Reticulum handles the translation and coordination automatically.

The key design consideration is not whether different mediums can work together (they can), but **how** they should work together based on your goals. A node with multiple interfaces spanning heterogeneous mediums needs to be configured with appropriate interface modes so that traffic flows efficiently. A gateway connecting a slow LoRa segment to a fast Internet backbone should be configured differently than a mobile device roaming between radio cells.

SUPPORT RETICULUM

You can help support the continued development of open, free and private communications systems by donating, providing feedback and contributing code and learning resources.

10.1 Donations

Donations are gratefully accepted via the following channels:

Monero:
84FpY1QbxHcgdseePYNmhTHcrgMX4nFfBYtz2GKYToqHVHhJp8Eaw1Z1EdRnKD19b3B8NiLCGVxzKV17UMmmeEsCrPyA5w

Bitcoin:
bc1pgqgu8h8xvj4jtafslq396v7ju7hkgymrzyqft4llfslz5vp99psqfk3a6

Ethereum:
0x91C421DdfB8a30a49A71d63447ddb54cEBe3465E

Liberapay:
<https://liberapay.com/Reticulum/>

Ko-Fi:
<https://ko-fi.com/markqvist>

Are certain features in the development roadmap are important to you or your organisation? Make them a reality quickly by sponsoring their implementation.

10.2 Provide Feedback

Feedback on the usage, functioning and potential dysfunctioning of any and all components of the system is very valuable to the continued development and improvement of Reticulum. But...

Warning

Think before you speak. As time has shown, over 80% of the “feedback”, “bug reports” and “advice” the Reticulum project has received has been irrelevant noise, stemming from erroneous assumptions, misunderstanding the foundational functionality or philosophy behind the system, or simply the malinformed (but overly opinionated) personal preferences of individual drive-by architects. This wastes the time of everyone involved.

The Reticulum project is not a public teahouse for serving the attention needs of random bypassers, but a highly complex system engineered and refined over more than a decade, designed to provide communication and connectivity guarantees in highly adversarial environments.

If you want to voice your opinion, it better be well-informed, and we expect you to have a comprehensive and solid foundation for your points of view. Everything else will be ignored.

Absolutely no automated analytics, telemetry, error reporting or statistics is collected and reported by Reticulum under any circumstances, so we rely on old-fashioned human feedback.

CODE EXAMPLES

A number of examples are included in the source distribution of Reticulum. You can use these examples to learn how to write your own programs.

11.1 Minimal

The *Minimal* example demonstrates the bare-minimum setup required to connect to a Reticulum network from your program. In about five lines of code, you will have the Reticulum Network Stack initialised, and ready to pass traffic in your program.

```
#####
# This RNS example demonstrates a minimal setup, that  #
# will start up the Reticulum Network Stack, generate a #
# new destination, and let the user send an announce.  #
#####

import argparse
import sys
import RNS

# Let's define an app name. We'll use this for all
# destinations we create. Since this basic example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

# This initialisation is executed when the program is started
def program_setup(configpath):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our example
    identity = RNS.Identity()

    # Using the identity we just created, we create a destination.
    # Destinations are endpoints in Reticulum, that can be addressed
    # and communicated with. Destinations can also announce their
    # existence, which will let the network know they are reachable
    # and automatically create paths to them, from anywhere else
    # in the network.
    destination = RNS.Destination(
```

(continues on next page)

(continued from previous page)

```

        identity,
        RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "minimalsample"
    )

    # We configure the destination to automatically prove all
    # packets addressed to it. By doing this, RNS will automatically
    # generate a proof for each incoming packet and transmit it
    # back to the sender of that packet. This will let anyone that
    # tries to communicate with the destination know whether their
    # communication was received correctly.
    destination.set_proof_strategy(RNS.Destination.PROVE_ALL)

    # Everything's ready!
    # Let's hand over control to the announce loop
    announceLoop(destination)

def announceLoop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Minimal example "+
        RNS.prettyhexrep(destination.hash)+
        " running, hit enter to manually send an announce (Ctrl-C to quit)"
    )

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.
    while True:
        entered = input()
        destination.announce()
        RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

#####
#### Program Startup #####
#####

# This part of the program gets run at startup,
# and parses input from the user, and then starts
# the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="Minimal example to start Reticulum and create a destination"
        )

        parser.add_argument(

```

(continues on next page)

(continued from previous page)

```

        "--config",
        action="store",
        default=None,
        help="path to alternative Reticulum config directory",
        type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    program_setup(configarg)

except KeyboardInterrupt:
    print("")
    sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Minimal.py>.

11.2 Announce

The *Announce* example builds upon the previous example by exploring how to announce a destination on the network, and how to let your program receive notifications about announces from relevant destinations.

```

#####
# This RNS example demonstrates setting up announce      #
# callbacks, which will let an application receive a    #
# notification when an announce relevant for it arrives #
#####

import argparse
import random
import sys
import RNS

# Let's define an app name. We'll use this for all
# destinations we create. Since this basic example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

# We initialise two lists of strings to use as app_data
fruits = ["Peach", "Quince", "Date", "Tangerine", "Pomelo", "Carambola", "Grape"]
noble_gases = ["Helium", "Neon", "Argon", "Krypton", "Xenon", "Radon", "Oganesson"]

# This initialisation is executed when the program is started
def program_setup(configpath):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

```

(continues on next page)

(continued from previous page)

```

# Randomly create a new identity for our example
identity = RNS.Identity()

# Using the identity we just created, we create two destinations
# in the "example_utilities.announcesample" application space.
#
# Destinations are endpoints in Reticulum, that can be addressed
# and communicated with. Destinations can also announce their
# existence, which will let the network know they are reachable
# and automatically create paths to them, from anywhere else
# in the network.
destination_1 = RNS.Destination(
    identity,
    RNS.Destination.IN,
    RNS.Destination.SINGLE,
    APP_NAME,
    "announcesample",
    "fruits"
)

destination_2 = RNS.Destination(
    identity,
    RNS.Destination.IN,
    RNS.Destination.SINGLE,
    APP_NAME,
    "announcesample",
    "noble_gases"
)

# We configure the destinations to automatically prove all
# packets addressed to it. By doing this, RNS will automatically
# generate a proof for each incoming packet and transmit it
# back to the sender of that packet. This will let anyone that
# tries to communicate with the destination know whether their
# communication was received correctly.
destination_1.set_proof_strategy(RNS.Destination.PROVE_ALL)
destination_2.set_proof_strategy(RNS.Destination.PROVE_ALL)

# We create an announce handler and configure it to only ask for
# announces from "example_utilities.announcesample.fruits".
# Try changing the filter and see what happens.
announce_handler = ExampleAnnounceHandler(
    aspect_filter="example_utilities.announcesample.fruits"
)

# We register the announce handler with Reticulum
RNS.Transport.register_announce_handler(announce_handler)

# Everything's ready!
# Let's hand over control to the announce loop
announceLoop(destination_1, destination_2)

```

(continues on next page)

(continued from previous page)

```

def announceLoop(destination_1, destination_2):
    # Let the user know that everything is ready
    RNS.log("Announce example running, hit enter to manually send an announce (Ctrl-C to_
    ↪quit)")

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.
    while True:
        entered = input()

        # Randomly select a fruit
        fruit = fruits[random.randint(0,len(fruits)-1)]

        # Send the announce including the app data
        destination_1.announce(app_data=fruit.encode("utf-8"))
        RNS.log(
            "Sent announce from "+
            RNS.prettyhexrep(destination_1.hash)+
            " (" +destination_1.name+)"
        )

        # Randomly select a noble gas
        noble_gas = noble_gases[random.randint(0,len(noble_gases)-1)]

        # Send the announce including the app data
        destination_2.announce(app_data=noble_gas.encode("utf-8"))
        RNS.log(
            "Sent announce from "+
            RNS.prettyhexrep(destination_2.hash)+
            " (" +destination_2.name+)"
        )

# We will need to define an announce handler class that
# Reticulum can message when an announce arrives.
class ExampleAnnounceHandler:
    # The initialisation method takes the optional
    # aspect_filter argument. If aspect_filter is set to
    # None, all announces will be passed to the instance.
    # If only some announces are wanted, it can be set to
    # an aspect string.
    def __init__(self, aspect_filter=None):
        self.aspect_filter = aspect_filter

    # This method will be called by Reticulum's Transport
    # system when an announce arrives that matches the
    # configured aspect filter. Filters must be specific,
    # and cannot use wildcards.
    def received_announce(self, destination_hash, announced_identity, app_data):

```

(continues on next page)

(continued from previous page)

```

RNS.log(
    "Received an announce from "+
    RNS.prettyhexrep(destination_hash)
)

if app_data:
    RNS.log(
        "The announce contained the following app data: "+
        app_data.decode("utf-8")
    )

#####
#### Program Startup #####
#####

# This part of the program gets run at startup,
# and parses input from the user, and then starts
# the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="Reticulum example that demonstrates announces and announce_
→handlers"
        )

        parser.add_argument(
            "--config",
            action="store",
            default=None,
            help="path to alternative Reticulum config directory",
            type=str
        )

        args = parser.parse_args()

        if args.config:
            configarg = args.config
        else:
            configarg = None

        program_setup(configarg)

    except KeyboardInterrupt:
        print("")
        sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Announce.py>.

11.3 Broadcast

The *Broadcast* example explores how to transmit plaintext broadcast messages over the network.

```
#####
# This RNS example demonstrates broadcasting unencrypted #
# information to any listening destinations.             #
#####

import sys
import argparse
import RNS

# Let's define an app name. We'll use this for all
# destinations we create. Since this basic example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

# This initialisation is executed when the program is started
def program_setup(configpath, channel=None):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # If the user did not select a "channel" we use
    # a default one called "public_information".
    # This "channel" is added to the destination name-
    # space, so the user can select different broadcast
    # channels.
    if channel == None:
        channel = "public_information"

    # We create a PLAIN destination. This is an unencrypted endpoint
    # that anyone can listen to and send information to.
    broadcast_destination = RNS.Destination(
        None,
        RNS.Destination.IN,
        RNS.Destination.PLAIN,
        APP_NAME,
        "broadcast",
        channel
    )

    # We specify a callback that will get called every time
    # the destination receives data.
    broadcast_destination.set_packet_callback(packet_callback)

    # Everything's ready!
    # Let's hand over control to the main loop
    broadcastLoop(broadcast_destination)

def packet_callback(data, packet):
    # Simply print out the received data
```

(continues on next page)

(continued from previous page)

```

print("")
print("Received data: "+data.decode("utf-8")+"\r\n> ", end="")
sys.stdout.flush()

def broadcastLoop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Broadcast example "+
        RNS.prettyhexrep(destination.hash)+
        " running, enter text and hit enter to broadcast (Ctrl-C to quit)"
    )

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will send the information
    # that the user entered into the prompt.
    while True:
        print("> ", end="")
        entered = input()

        if entered != "":
            data = entered.encode("utf-8")
            packet = RNS.Packet(destination, data)
            packet.send()

#####
#### Program Startup #####
#####

# This part of the program gets run at startup,
# and parses input from the user, and then starts
# the program.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="Reticulum example demonstrating sending and receiving broadcasts
↪"
        )

        parser.add_argument(
            "--config",
            action="store",
            default=None,
            help="path to alternative Reticulum config directory",
            type=str
        )

        parser.add_argument(
            "--channel",
            action="store",
            default=None,

```

(continues on next page)

(continued from previous page)

```

        help="broadcast channel name",
        type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    if args.channel:
        channelarg = args.channel
    else:
        channelarg = None

    program_setup(configarg, channelarg)

except KeyboardInterrupt:
    print("")
    sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Broadcast.py>.

11.4 Echo

The *Echo* example demonstrates communication between two destinations using the Packet interface.

```

#####
# This RNS example demonstrates a simple client/server #
# echo utility. A client can send an echo request to the #
# server, and the server will respond by proving receipt #
# of the packet. #
#####

import argparse
import sys
import RNS

# Let's define an app name. We'll use this for all
# destinations we create. Since this echo example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

#####
#### Server Part #####
#####

# This initialisation is executed when the users chooses
# to run as a server

```

(continues on next page)

(continued from previous page)

```

def server(configpath):
    global reticulum

    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our echo server
    server_identity = RNS.Identity()

    # We create a destination that clients can query. We want
    # to be able to verify echo replies to our clients, so we
    # create a "single" destination that can receive encrypted
    # messages. This way the client can send a request and be
    # certain that no-one else than this destination was able
    # to read it.
    echo_destination = RNS.Destination(
        server_identity,
        RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "echo",
        "request"
    )

    # We configure the destination to automatically prove all
    # packets addressed to it. By doing this, RNS will automatically
    # generate a proof for each incoming packet and transmit it
    # back to the sender of that packet.
    echo_destination.set_proof_strategy(RNS.Destination.PROVE_ALL)

    # Tell the destination which function in our program to
    # run when a packet is received. We do this so we can
    # print a log message when the server receives a request
    echo_destination.set_packet_callback(server_callback)

    # Everything's ready!
    # Let's Wait for client requests or user input
    announceLoop(echo_destination)

def announceLoop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Echo server "+
        RNS.prettyhexrep(destination.hash)+
        " running, hit enter to manually send an announce (Ctrl-C to quit)"
    )

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.

```

(continues on next page)

(continued from previous page)

```

while True:
    entered = input()
    destination.announce()
    RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

def server_callback(message, packet):
    global reticulum

    # Tell the user that we received an echo request, and
    # that we are going to send a reply to the requester.
    # Sending the proof is handled automatically, since we
    # set up the destination to prove all incoming packets.

    reception_stats = ""
    if reticulum.is_connected_to_shared_instance:
        reception_rssi = reticulum.get_packet_rssi(packet.packet_hash)
        reception_snr = reticulum.get_packet_snr(packet.packet_hash)

        if reception_rssi != None:
            reception_stats += " [RSSI "+str(reception_rssi)+" dBm]"

        if reception_snr != None:
            reception_stats += " [SNR "+str(reception_snr)+" dBm]"

    else:
        if packet.rssi != None:
            reception_stats += " [RSSI "+str(packet.rssi)+" dBm]"

        if packet.snr != None:
            reception_stats += " [SNR "+str(packet.snr)+" dB]"

    RNS.log("Received packet from echo client, proof sent"+reception_stats)

#####
#### Client Part #####
#####

# This initialisation is executed when the users chooses
# to run as a client
def client(destination_hexhash, configpath, timeout=None):
    global reticulum

    # We need a binary representation of the destination
    # hash that was entered on the command line
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Destination length is invalid, must be {hex} hexadecimal characters (
↪{byte} bytes)".format(hex=dest_len, byte=dest_len//2)

```

(continues on next page)

(continued from previous page)

```

    )

    destination_hash = bytes.fromhex(destination_hexhash)
except Exception as e:
    RNS.log("Invalid destination entered. Check your input!")
    RNS.log(str(e)+"\n")
    sys.exit(0)

# We must first initialise Reticulum
reticulum = RNS.Reticulum(configpath)

# We override the loglevel to provide feedback when
# an announce is received
if RNS.loglevel < RNS.LOG_INFO:
    RNS.loglevel = RNS.LOG_INFO

# Tell the user that the client is ready!
RNS.log(
    "Echo client ready, hit enter to send echo request to "+
    destination_hexhash+
    " (Ctrl-C to quit)"
)

# We enter a loop that runs until the user exits.
# If the user hits enter, we will try to send an
# echo request to the destination specified on the
# command line.
while True:
    input()

    # Let's first check if RNS knows a path to the destination.
    # If it does, we'll load the server identity and create a packet
    if RNS.Transport.has_path(destination_hash):

        # To address the server, we need to know it's public
        # key, so we check if Reticulum knows this destination.
        # This is done by calling the "recall" method of the
        # Identity module. If the destination is known, it will
        # return an Identity instance that can be used in
        # outgoing destinations.
        server_identity = RNS.Identity.recall(destination_hash)

        # We got the correct identity instance from the
        # recall method, so let's create an outgoing
        # destination. We use the naming convention:
        # example_utilities.echo.request
        # This matches the naming we specified in the
        # server part of the code.
        request_destination = RNS.Destination(
            server_identity,
            RNS.Destination.OUT,
            RNS.Destination.SINGLE,

```

(continues on next page)

(continued from previous page)

```

        APP_NAME,
        "echo",
        "request"
    )

    # The destination is ready, so let's create a packet.
    # We set the destination to the request_destination
    # that was just created, and the only data we add
    # is a random hash.
    echo_request = RNS.Packet(request_destination, RNS.Identity.get_random_
↪hash())

    # Send the packet! If the packet is successfully
    # sent, it will return a PacketReceipt instance.
    packet_receipt = echo_request.send()

    # If the user specified a timeout, we set this
    # timeout on the packet receipt, and configure
    # a callback function, that will get called if
    # the packet times out.
    if timeout != None:
        packet_receipt.set_timeout(timeout)
        packet_receipt.set_timeout_callback(packet_timed_out)

    # We can then set a delivery callback on the receipt.
    # This will get automatically called when a proof for
    # this specific packet is received from the destination.
    packet_receipt.set_delivery_callback(packet_delivered)

    # Tell the user that the echo request was sent
    RNS.log("Sent echo request to "+RNS.prettyhexrep(request_destination.hash))
else:
    # If we do not know this destination, tell the
    # user to wait for an announce to arrive.
    RNS.log("Destination is not yet known. Requesting path...")
    RNS.log("Hit enter to manually retry once an announce is received.")
    RNS.Transport.request_path(destination_hash)

# This function is called when our reply destination
# receives a proof packet.
def packet_delivered(receipt):
    global reticulum

    if receipt.status == RNS.PacketReceipt.DELIVERED:
        rtt = receipt.get_rtt()
        if (rtt >= 1):
            rtt = round(rtt, 3)
            rttstring = str(rtt)+" seconds"
        else:
            rtt = round(rtt*1000, 3)
            rttstring = str(rtt)+" milliseconds"

```

(continues on next page)

(continued from previous page)

```

reception_stats = ""
if reticulum.is_connected_to_shared_instance:
    reception_rssi = reticulum.get_packet_rssi(receipt.proof_packet.packet_hash)
    reception_snr = reticulum.get_packet_snr(receipt.proof_packet.packet_hash)

    if reception_rssi != None:
        reception_stats += " [RSSI "+str(reception_rssi)+" dBm]"

    if reception_snr != None:
        reception_stats += " [SNR "+str(reception_snr)+" dB]"

else:
    if receipt.proof_packet != None:
        if receipt.proof_packet.rssi != None:
            reception_stats += " [RSSI "+str(receipt.proof_packet.rssi)+" dBm]"

        if receipt.proof_packet.snr != None:
            reception_stats += " [SNR "+str(receipt.proof_packet.snr)+" dB]"

RNS.log(
    "Valid reply received from "+
    RNS.prettyhexrep(receipt.destination.hash)+
    ", round-trip time is "+rttstring+
    reception_stats
)

# This function is called if a packet times out.
def packet_timed_out(receipt):
    if receipt.status == RNS.PacketReceipt.FAILED:
        RNS.log("Packet "+RNS.prettyhexrep(receipt.hash)+" timed out")

#####
#### Program Startup #####
#####

# This part of the program gets run at startup,
# and parses input from the user, and then starts
# the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Simple echo server and client_
↳ utility")

        parser.add_argument(
            "-s",
            "--server",
            action="store_true",
            help="wait for incoming packets from clients"
        )

        parser.add_argument(

```

(continues on next page)

(continued from previous page)

```

        "-t",
        "--timeout",
        action="store",
        metavar="s",
        default=None,
        help="set a reply timeout in seconds",
        type=float
    )

    parser.add_argument("--config",
        action="store",
        default=None,
        help="path to alternative Reticulum config directory",
        type=str
    )

    parser.add_argument(
        "destination",
        nargs="?",
        default=None,
        help="hexadecimal hash of the server destination",
        type=str
    )

    args = parser.parse_args()

    if args.server:
        configarg=None
        if args.config:
            configarg = args.config
        server(configarg)
    else:
        if args.config:
            configarg = args.config
        else:
            configarg = None

        if args.timeout:
            timeoutarg = float(args.timeout)
        else:
            timeoutarg = None

        if (args.destination == None):
            print("")
            parser.print_help()
            print("")
        else:
            client(args.destination, configarg, timeout=timeoutarg)
    except KeyboardInterrupt:
        print("")
        sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Echo.py>.

11.5 Link

The *Link* example explores establishing an encrypted link to a remote destination, and passing traffic back and forth over the link.

```
#####
# This RNS example demonstrates how to set up a link to #
# a destination, and pass data back and forth over it. #
#####

import os
import sys
import time
import argparse
import RNS

# Let's define an app name. We'll use this for all
# destinations we create. Since this echo example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

#####
#### Server Part #####
#####

# A reference to the latest client link that connected
latest_client_link = None

# This initialisation is executed when the users chooses
# to run as a server
def server(configpath):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our link example
    server_identity = RNS.Identity()

    # We create a destination that clients can connect to. We
    # want clients to create links to this destination, so we
    # need to create a "single" destination type.
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "linkexample"
    )

    # We configure a function that will get called every time
    # a new client creates a link to this destination.
    server_destination.set_link_established_callback(client_connected)
```

(continues on next page)

(continued from previous page)

```

# Everything's ready!
# Let's Wait for client requests or user input
server_loop(server_destination)

def server_loop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Link example "+
        RNS.prettyhexrep(destination.hash)+
        " running, waiting for a connection."
    )

    RNS.log("Hit enter to manually send an announce (Ctrl-C to quit)")

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.
    while True:
        entered = input()
        destination.announce()
        RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

# When a client establishes a link to our server
# destination, this function will be called with
# a reference to the link.
def client_connected(link):
    global latest_client_link

    RNS.log("Client connected")
    link.set_link_closed_callback(client_disconnected)
    link.set_packet_callback(server_packet_received)
    latest_client_link = link

def client_disconnected(link):
    RNS.log("Client disconnected")

def server_packet_received(message, packet):
    global latest_client_link

    # When data is received over any active link,
    # it will all be directed to the last client
    # that connected.
    text = message.decode("utf-8")
    RNS.log("Received data on the link: "+text)

    reply_text = "I received \""+text+"\" over the link"
    reply_data = reply_text.encode("utf-8")
    RNS.Packet(latest_client_link, reply_data).send()

```

#####

(continues on next page)

(continued from previous page)

```

#### Client Part #####
#####

# A reference to the server link
server_link = None

# This initialisation is executed when the users chooses
# to run as a client
def client(destination_hexhash, configpath):
    # We need a binary representation of the destination
    # hash that was entered on the command line
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Destination length is invalid, must be {hex} hexadecimal characters (
↳{byte} bytes)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Invalid destination entered. Check your input!\n")
        sys.exit(0)

    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Check if we know a path to the destination
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Destination is not yet known. Requesting path and waiting for announce_
↳to arrive...")
        RNS.Transport.request_path(destination_hash)
        while not RNS.Transport.has_path(destination_hash):
            time.sleep(0.1)

    # Recall the server identity
    server_identity = RNS.Identity.recall(destination_hash)

    # Inform the user that we'll begin connecting
    RNS.log("Establishing link with server...")

    # When the server identity is known, we set
    # up a destination
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.OUT,
        RNS.Destination.SINGLE,
        APP_NAME,
        "linkexample"
    )

    # And create a link

```

(continues on next page)

(continued from previous page)

```

link = RNS.Link(server_destination)

# We set a callback that will get executed
# every time a packet is received over the
# link
link.set_packet_callback(client_packet_received)

# We'll also set up functions to inform the
# user when the link is established or closed
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Everything is set up, so let's enter a loop
# for the user to interact with the example
client_loop()

def client_loop():
    global server_link

    # Wait for the link to become active
    while not server_link:
        time.sleep(0.1)

    should_quit = False
    while not should_quit:
        try:
            print("> ", end=" ")
            text = input()

            # Check if we should quit the example
            if text == "quit" or text == "q" or text == "exit":
                should_quit = True
                server_link.teardown()

            # If not, send the entered text over the link
            if text != "":
                data = text.encode("utf-8")
                if len(data) <= RNS.Link.MDU:
                    RNS.Packet(server_link, data).send()
                else:
                    RNS.log(
                        "Cannot send this packet, the data size of "+
                        str(len(data))+" bytes exceeds the link packet MDU of "+
                        str(RNS.Link.MDU)+" bytes",
                        RNS.LOG_ERROR
                    )

        except Exception as e:
            RNS.log("Error while sending data over the link: "+str(e))
            should_quit = True
            server_link.teardown()

```

(continues on next page)

(continued from previous page)

```

# This function is called when a link
# has been established with the server
def link_established(link):
    # We store a reference to the link
    # instance for later use
    global server_link
    server_link = link

    # Inform the user that the server is
    # connected
    RNS.log("Link established with server, enter some text to send, or \"quit\" to quit")

# When a link is closed, we'll inform the
# user, and exit the program
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT:
        RNS.log("The link timed out, exiting now")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED:
        RNS.log("The link was closed by the server, exiting now")
    else:
        RNS.log("Link closed, exiting now")

    time.sleep(1.5)
    sys.exit(0)

# When a packet is received over the link, we
# simply print out the data.
def client_packet_received(message, packet):
    text = message.decode("utf-8")
    RNS.log("Received data on the link: "+text)
    print("> ", end=" ")
    sys.stdout.flush()

#####
#### Program Startup #####
#####

# This part of the program runs at startup,
# and parses input of from the user, and then
# starts up the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Simple link example")

        parser.add_argument(
            "-s",
            "--server",
            action="store_true",
            help="wait for incoming link requests from clients"
        )

```

(continues on next page)

(continued from previous page)

```

parser.add_argument(
    "--config",
    action="store",
    default=None,
    help="path to alternative Reticulum config directory",
    type=str
)

parser.add_argument(
    "destination",
    nargs="?",
    default=None,
    help="hexadecimal hash of the server destination",
    type=str
)

args = parser.parse_args()

if args.config:
    configarg = args.config
else:
    configarg = None

if args.server:
    server(configarg)
else:
    if (args.destination == None):
        print("")
        parser.print_help()
        print("")
    else:
        client(args.destination, configarg)

except KeyboardInterrupt:
    print("")
    sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Link.py>.

11.6 Identification

The *Identify* example explores identifying an initiator of a link, once the link has been established.

```

#####
# This RNS example demonstrates how to set up a link to #
# a destination, and identify the initiator to it's peer #
#####

import os
import sys
import time
import argparse

```

(continues on next page)

(continued from previous page)

```

import RNS

# Let's define an app name. We'll use this for all
# destinations we create. Since this echo example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

#####
#### Server Part #####
#####

# A reference to the latest client link that connected
latest_client_link = None

# This initialisation is executed when the users chooses
# to run as a server
def server(configpath):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our link example
    server_identity = RNS.Identity()

    # We create a destination that clients can connect to. We
    # want clients to create links to this destination, so we
    # need to create a "single" destination type.
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "identifyexample"
    )

    # We configure a function that will get called every time
    # a new client creates a link to this destination.
    server_destination.set_link_established_callback(client_connected)

    # Everything's ready!
    # Let's Wait for client requests or user input
    server_loop(server_destination)

def server_loop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Link identification example "+
        RNS.prettyhexrep(destination.hash)+
        " running, waiting for a connection."
    )

    RNS.log("Hit enter to manually send an announce (Ctrl-C to quit)")

```

(continues on next page)

(continued from previous page)

```

# We enter a loop that runs until the users exits.
# If the user hits enter, we will announce our server
# destination on the network, which will let clients
# know how to create messages directed towards it.
while True:
    entered = input()
    destination.announce()
    RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

# When a client establishes a link to our server
# destination, this function will be called with
# a reference to the link.
def client_connected(link):
    global latest_client_link

    RNS.log("Client connected")
    link.set_link_closed_callback(client_disconnected)
    link.set_packet_callback(server_packet_received)
    link.set_remote_identified_callback(remote_identified)
    latest_client_link = link

def client_disconnected(link):
    RNS.log("Client disconnected")

def remote_identified(link, identity):
    RNS.log("Remote identified as: "+str(identity))

def server_packet_received(message, packet):
    global latest_client_link

    # Get the originating identity for display
    remote_peer = "unidentified peer"
    if packet.link.get_remote_identity() != None:
        remote_peer = str(packet.link.get_remote_identity())

    # When data is received over any active link,
    # it will all be directed to the last client
    # that connected.
    text = message.decode("utf-8")

    RNS.log("Received data from "+remote_peer+": "+text)

    reply_text = "I received \""+text+"\" over the link from "+remote_peer
    reply_data = reply_text.encode("utf-8")
    RNS.Packet(latest_client_link, reply_data).send()

#####
#### Client Part #####
#####

```

(continues on next page)

(continued from previous page)

```

# A reference to the server link
server_link = None

# A reference to the client identity
client_identity = None

# This initialisation is executed when the users chooses
# to run as a client
def client(destination_hexhash, configpath):
    global client_identity
    # We need a binary representation of the destination
    # hash that was entered on the command line
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Destination length is invalid, must be {hex} hexadecimal characters (
↳{byte} bytes)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Invalid destination entered. Check your input!\n")
        sys.exit(0)

    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Create a new client identity
    client_identity = RNS.Identity()
    RNS.log(
        "Client created new identity "+
        str(client_identity)
    )

    # Check if we know a path to the destination
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Destination is not yet known. Requesting path and waiting for announce_
↳to arrive...")
        RNS.Transport.request_path(destination_hash)
        while not RNS.Transport.has_path(destination_hash):
            time.sleep(0.1)

    # Recall the server identity
    server_identity = RNS.Identity.recall(destination_hash)

    # Inform the user that we'll begin connecting
    RNS.log("Establishing link with server...")

    # When the server identity is known, we set
    # up a destination
    server_destination = RNS.Destination(

```

(continues on next page)

(continued from previous page)

```

        server_identity,
        RNS.Destination.OUT,
        RNS.Destination.SINGLE,
        APP_NAME,
        "identifyexample"
    )

    # And create a link
    link = RNS.Link(server_destination)

    # We set a callback that will get executed
    # every time a packet is received over the
    # link
    link.set_packet_callback(client_packet_received)

    # We'll also set up functions to inform the
    # user when the link is established or closed
    link.set_link_established_callback(link_established)
    link.set_link_closed_callback(link_closed)

    # Everything is set up, so let's enter a loop
    # for the user to interact with the example
    client_loop()

def client_loop():
    global server_link

    # Wait for the link to become active
    while not server_link:
        time.sleep(0.1)

    should_quit = False
    while not should_quit:
        try:
            print("> ", end=" ")
            text = input()

            # Check if we should quit the example
            if text == "quit" or text == "q" or text == "exit":
                should_quit = True
                server_link.teardown()

            # If not, send the entered text over the link
            if text != "":
                data = text.encode("utf-8")
                if len(data) <= RNS.Link.MDU:
                    RNS.Packet(server_link, data).send()
                else:
                    RNS.log(
                        "Cannot send this packet, the data size of "+
                        str(len(data))+ " bytes exceeds the link packet MDU of "+
                        str(RNS.Link.MDU)+ " bytes",

```

(continues on next page)

(continued from previous page)

```

        RNS.LOG_ERROR
    )

    except Exception as e:
        RNS.log("Error while sending data over the link: "+str(e))
        should_quit = True
        server_link.teardown()

# This function is called when a link
# has been established with the server
def link_established(link):
    # We store a reference to the link
    # instance for later use
    global server_link, client_identity
    server_link = link

    # Inform the user that the server is
    # connected
    RNS.log("Link established with server, identifying to remote peer...")

    link.identify(client_identity)

# When a link is closed, we'll inform the
# user, and exit the program
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT:
        RNS.log("The link timed out, exiting now")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED:
        RNS.log("The link was closed by the server, exiting now")
    else:
        RNS.log("Link closed, exiting now")

    time.sleep(1.5)
    sys.exit(0)

# When a packet is received over the link, we
# simply print out the data.
def client_packet_received(message, packet):
    text = message.decode("utf-8")
    RNS.log("Received data on the link: "+text)
    print("> ", end=" ")
    sys.stdout.flush()

#####
#### Program Startup #####
#####

# This part of the program runs at startup,
# and parses input of from the user, and then
# starts up the desired program mode.
if __name__ == "__main__":

```

(continues on next page)

(continued from previous page)

```

try:
    parser = argparse.ArgumentParser(description="Simple link example")

    parser.add_argument(
        "-s",
        "--server",
        action="store_true",
        help="wait for incoming link requests from clients"
    )

    parser.add_argument(
        "--config",
        action="store",
        default=None,
        help="path to alternative Reticulum config directory",
        type=str
    )

    parser.add_argument(
        "destination",
        nargs="?",
        default=None,
        help="hexadecimal hash of the server destination",
        type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    if args.server:
        server(configarg)
    else:
        if (args.destination == None):
            print("")
            parser.print_help()
            print("")
        else:
            client(args.destination, configarg)

except KeyboardInterrupt:
    print("")
    sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Identify.py>.

11.7 Requests & Responses

The *Request* example explores sending requests and receiving responses.

```
#####
# This RNS example demonstrates how to perform requests #
# and receive responses over a link.                    #
#####

import os
import sys
import time
import random
import argparse
import RNS

# Let's define an app name. We'll use this for all
# destinations we create. Since this echo example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

#####
#### Server Part #####
#####

# A reference to the latest client link that connected
latest_client_link = None

def random_text_generator(path, data, request_id, link_id, remote_identity, requested_
→at):
    RNS.log("Generating response to request "+RNS.prettyhexrep(request_id)+" on link
    →"+RNS.prettyhexrep(link_id))
    texts = ["They looked up", "On each full moon", "Becky was upset", "I'll stay away_
    →from it", "The pet shop stocks everything"]
    return texts[random.randint(0, len(texts)-1)]

# This initialisation is executed when the users chooses
# to run as a server
def server(configpath):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our link example
    server_identity = RNS.Identity()

    # We create a destination that clients can connect to. We
    # want clients to create links to this destination, so we
    # need to create a "single" destination type.
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.IN,
        RNS.Destination.SINGLE,
```

(continues on next page)

(continued from previous page)

```

        APP_NAME,
        "requestexample"
    )

    # We configure a function that will get called every time
    # a new client creates a link to this destination.
    server_destination.set_link_established_callback(client_connected)

    # We register a request handler for handling incoming
    # requests over any established links.
    server_destination.register_request_handler(
        "/random/text",
        response_generator = random_text_generator,
        allow = RNS.Destination.ALLOW_ALL
    )

    # Everything's ready!
    # Let's Wait for client requests or user input
    server_loop(server_destination)

def server_loop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Request example "+
        RNS.prettyhexrep(destination.hash)+
        " running, waiting for a connection."
    )

    RNS.log("Hit enter to manually send an announce (Ctrl-C to quit)")

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.
    while True:
        entered = input()
        destination.announce()
        RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

    # When a client establishes a link to our server
    # destination, this function will be called with
    # a reference to the link.
def client_connected(link):
    global latest_client_link

    RNS.log("Client connected")
    link.set_link_closed_callback(client_disconnected)
    latest_client_link = link

def client_disconnected(link):
    RNS.log("Client disconnected")

```

(continues on next page)

(continued from previous page)

```
#####
#### Client Part #####
#####

# A reference to the server link
server_link = None

# This initialisation is executed when the users chooses
# to run as a client
def client(destination_hexhash, configpath):
    # We need a binary representation of the destination
    # hash that was entered on the command line
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Destination length is invalid, must be {hex} hexadecimal characters (
↳ {byte} bytes)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Invalid destination entered. Check your input!\n")
        sys.exit(0)

    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Check if we know a path to the destination
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Destination is not yet known. Requesting path and waiting for announce_
↳ to arrive...")
        RNS.Transport.request_path(destination_hash)
        while not RNS.Transport.has_path(destination_hash):
            time.sleep(0.1)

    # Recall the server identity
    server_identity = RNS.Identity.recall(destination_hash)

    # Inform the user that we'll begin connecting
    RNS.log("Establishing link with server...")

    # When the server identity is known, we set
    # up a destination
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.OUT,
        RNS.Destination.SINGLE,
        APP_NAME,
        "requestexample"
    )

```

(continues on next page)

(continued from previous page)

```

# And create a link
link = RNS.Link(server_destination)

# We'll set up functions to inform the
# user when the link is established or closed
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Everything is set up, so let's enter a loop
# for the user to interact with the example
client_loop()

def client_loop():
    global server_link

    # Wait for the link to become active
    while not server_link:
        time.sleep(0.1)

    should_quit = False
    while not should_quit:
        try:
            print("> ", end=" ")
            text = input()

            # Check if we should quit the example
            if text == "quit" or text == "q" or text == "exit":
                should_quit = True
                server_link.teardown()

            else:
                server_link.request(
                    "/random/text",
                    data = None,
                    response_callback = got_response,
                    failed_callback = request_failed
                )

        except Exception as e:
            RNS.log("Error while sending request over the link: "+str(e))
            should_quit = True
            server_link.teardown()

def got_response(request_receipt):
    request_id = request_receipt.request_id
    response = request_receipt.response

    RNS.log("Got response for request "+RNS.prettyhexrep(request_id)+": "+str(response))

def request_received(request_receipt):

```

(continues on next page)

(continued from previous page)

```

RNS.log("The request "+RNS.prettyhexrep(request_receipt.request_id)+" was received_
↳by the remote peer.")

def request_failed(request_receipt):
    RNS.log("The request "+RNS.prettyhexrep(request_receipt.request_id)+" failed.")

# This function is called when a link
# has been established with the server
def link_established(link):
    # We store a reference to the link
    # instance for later use
    global server_link
    server_link = link

    # Inform the user that the server is
    # connected
    RNS.log("Link established with server, hit enter to perform a request, or type in \
↳quit\" to quit")

# When a link is closed, we'll inform the
# user, and exit the program
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT:
        RNS.log("The link timed out, exiting now")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED:
        RNS.log("The link was closed by the server, exiting now")
    else:
        RNS.log("Link closed, exiting now")

    time.sleep(1.5)
    sys.exit(0)

#####
#### Program Startup #####
#####

# This part of the program runs at startup,
# and parses input of from the user, and then
# starts up the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Simple request/response example")

        parser.add_argument(
            "-s",
            "--server",
            action="store_true",
            help="wait for incoming requests from clients"
        )

```

(continues on next page)

(continued from previous page)

```

parser.add_argument(
    "--config",
    action="store",
    default=None,
    help="path to alternative Reticulum config directory",
    type=str
)

parser.add_argument(
    "destination",
    nargs="?",
    default=None,
    help="hexadecimal hash of the server destination",
    type=str
)

args = parser.parse_args()

if args.config:
    configarg = args.config
else:
    configarg = None

if args.server:
    server(configarg)
else:
    if (args.destination == None):
        print("")
        parser.print_help()
        print("")
    else:
        client(args.destination, configarg)

except KeyboardInterrupt:
    print("")
    sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Request.py>.

11.8 Channel

The *Channel* example explores using a `Channel` to send structured data between peers of a `Link`.

```

#####
# This RNS example demonstrates how to set up a link to #
# a destination, and pass structured messages over it #
# using a channel. #
#####

import os
import sys
import time

```

(continues on next page)

(continued from previous page)

```

import argparse
from datetime import datetime

import RNS
from RNS.vendor import msgpack

# Let's define an app name. We'll use this for all
# destinations we create. Since this echo example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

#####
#### Shared Objects #####
#####

# Channel data must be structured in a subclass of
# MessageBase. This ensures that the channel will be able
# to serialize and deserialize the object and multiplex it
# with other objects. Both ends of a link will need the
# same object definitions to be able to communicate over
# a channel.
#
# Note: The objects we wish to use over the channel must
# be registered with the channel, and each link has a
# different channel instance. See the client_connected
# and link_established functions in this example to see
# how message types are registered.

# Let's make a simple message class called StringMessage
# that will convey a string with a timestamp.

class StringMessage(RNS.MessageBase):
    # The MSGTYPE class variable needs to be assigned a
    # 2 byte integer value. This identifier allows the
    # channel to look up your message's constructor when a
    # message arrives over the channel.
    #
    # MSGTYPE must be unique across all message types we
    # register with the channel. MSGTYPES >= 0xf000 are
    # reserved for the system.
    MSGTYPE = 0x0101

    # The constructor of our object must be callable with
    # no arguments. We can have parameters, but they must
    # have a default assignment.
    #
    # This is needed so the channel can create an empty
    # version of our message into which the incoming
    # message can be unpacked.
    def __init__(self, data=None):
        self.data = data

```

(continues on next page)

(continued from previous page)

```

        self.timestamp = datetime.now()

# Finally, our message needs to implement functions
# the channel can call to pack and unpack our message
# to/from the raw packet payload. We'll use the
# umsgpack package bundled with RNS. We could also use
# the struct package bundled with Python if we wanted
# more control over the structure of the packed bytes.
#
# Also note that packed message objects must fit
# entirely in one packet. The number of bytes
# available for message payloads can be queried from
# the channel using the Channel.MDU property. The
# channel MDU is slightly less than the link MDU due
# to encoding the message header.

# The pack function encodes the message contents into
# a byte stream.
def pack(self) -> bytes:
    return umsgpack.packb((self.data, self.timestamp))

# And the unpack function decodes a byte stream into
# the message contents.
def unpack(self, raw):
    self.data, self.timestamp = umsgpack.unpackb(raw)

#####
#### Server Part #####
#####

# A reference to the latest client link that connected
latest_client_link = None

# This initialisation is executed when the users chooses
# to run as a server
def server(configpath):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our link example
    server_identity = RNS.Identity()

    # We create a destination that clients can connect to. We
    # want clients to create links to this destination, so we
    # need to create a "single" destination type.
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "channelexample"

```

(continues on next page)

(continued from previous page)

```

)

# We configure a function that will get called every time
# a new client creates a link to this destination.
server_destination.set_link_established_callback(client_connected)

# Everything's ready!
# Let's Wait for client requests or user input
server_loop(server_destination)

def server_loop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Channel example "+
        RNS.prettyhexrep(destination.hash)+
        " running, waiting for a connection."
    )

    RNS.log("Hit enter to manually send an announce (Ctrl-C to quit)")

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.
    while True:
        entered = input()
        destination.announce()
        RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

# When a client establishes a link to our server
# destination, this function will be called with
# a reference to the link.
def client_connected(link):
    global latest_client_link
    latest_client_link = link

    RNS.log("Client connected")
    link.set_link_closed_callback(client_disconnected)

    # Register message types and add callback to channel
    channel = link.get_channel()
    channel.register_message_type(StringMessage)
    channel.add_message_handler(server_message_received)

def client_disconnected(link):
    RNS.log("Client disconnected")

def server_message_received(message):
    """
    A message handler
    @param message: An instance of a subclass of MessageBase
    @return: True if message was handled

```

(continues on next page)

(continued from previous page)

```

"""
global latest_client_link
# When a message is received over any active link,
# the replies will all be directed to the last client
# that connected.

# In a message handler, any deserializable message
# that arrives over the link's channel will be passed
# to all message handlers, unless a preceding handler indicates it
# has handled the message.
#
#
if isinstance(message, StringMessage):
    RNS.log("Received data on the link: " + message.data + " (message created at " +
↪str(message.timestamp) + ")")

    reply_message = StringMessage("I received \""+message.data+"\" over the link")
    latest_client_link.get_channel().send(reply_message)

    # Incoming messages are sent to each message
    # handler added to the channel, in the order they
    # were added.
    # If any message handler returns True, the message
    # is considered handled and any subsequent
    # handlers are skipped.
    return True

#####
#### Client Part #####
#####

# A reference to the server link
server_link = None

# This initialisation is executed when the users chooses
# to run as a client
def client(destination_hexhash, configpath):
    # We need a binary representation of the destination
    # hash that was entered on the command line
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Destination length is invalid, must be {hex} hexadecimal characters (
↪{byte} bytes)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Invalid destination entered. Check your input!\n")
        sys.exit(0)

```

(continues on next page)

(continued from previous page)

```

# We must first initialise Reticulum
reticulum = RNS.Reticulum(configpath)

# Check if we know a path to the destination
if not RNS.Transport.has_path(destination_hash):
    RNS.log("Destination is not yet known. Requesting path and waiting for announce_
↳to arrive...")
    RNS.Transport.request_path(destination_hash)
    while not RNS.Transport.has_path(destination_hash):
        time.sleep(0.1)

# Recall the server identity
server_identity = RNS.Identity.recall(destination_hash)

# Inform the user that we'll begin connecting
RNS.log("Establishing link with server...")

# When the server identity is known, we set
# up a destination
server_destination = RNS.Destination(
    server_identity,
    RNS.Destination.OUT,
    RNS.Destination.SINGLE,
    APP_NAME,
    "channelexample"
)

# And create a link
link = RNS.Link(server_destination)

# We'll also set up functions to inform the
# user when the link is established or closed
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Everything is set up, so let's enter a loop
# for the user to interact with the example
client_loop()

def client_loop():
    global server_link

    # Wait for the link to become active
    while not server_link:
        time.sleep(0.1)

    should_quit = False
    while not should_quit:
        try:
            print("> ", end=" ")
            text = input()

```

(continues on next page)

(continued from previous page)

```

    # Check if we should quit the example
    if text == "quit" or text == "q" or text == "exit":
        should_quit = True
        server_link.teardown()

    # If not, send the entered text over the link
    if text != "":
        message = StringMessage(text)
        packed_size = len(message.pack())
        channel = server_link.get_channel()
        if channel.is_ready_to_send():
            if packed_size <= channel.mdu:
                channel.send(message)
            else:
                RNS.log(
                    "Cannot send this packet, the data size of "+
                    str(packed_size)+" bytes exceeds the link packet MDU of "+
                    str(channel.MDU)+" bytes",
                    RNS.LOG_ERROR
                )
        else:
            RNS.log("Channel is not ready to send, please wait for " +
                "pending messages to complete.", RNS.LOG_ERROR)

    except Exception as e:
        RNS.log("Error while sending data over the link: "+str(e))
        should_quit = True
        server_link.teardown()

# This function is called when a link
# has been established with the server
def link_established(link):
    # We store a reference to the link
    # instance for later use
    global server_link
    server_link = link

    # Register messages and add handler to channel
    channel = link.get_channel()
    channel.register_message_type(StringMessage)
    channel.add_message_handler(client_message_received)

    # Inform the user that the server is
    # connected
    RNS.log("Link established with server, enter some text to send, or \"quit\" to quit")

# When a link is closed, we'll inform the
# user, and exit the program
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT:
        RNS.log("The link timed out, exiting now")

```

(continues on next page)

(continued from previous page)

```

elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED:
    RNS.log("The link was closed by the server, exiting now")
else:
    RNS.log("Link closed, exiting now")

time.sleep(1.5)
sys.exit(0)

# When a packet is received over the channel, we
# simply print out the data.
def client_message_received(message):
    if isinstance(message, StringMessage):
        RNS.log("Received data on the link: " + message.data + " (message created at " +
↳str(message.timestamp) + ")")
        print("> ", end=" ")
        sys.stdout.flush()

#####
#### Program Startup #####
#####

# This part of the program runs at startup,
# and parses input of from the user, and then
# starts up the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Simple channel example")

        parser.add_argument(
            "-s",
            "--server",
            action="store_true",
            help="wait for incoming link requests from clients"
        )

        parser.add_argument(
            "--config",
            action="store",
            default=None,
            help="path to alternative Reticulum config directory",
            type=str
        )

        parser.add_argument(
            "destination",
            nargs="?",
            default=None,
            help="hexadecimal hash of the server destination",
            type=str
        )

```

(continues on next page)

(continued from previous page)

```

args = parser.parse_args()

if args.config:
    configarg = args.config
else:
    configarg = None

if args.server:
    server(configarg)
else:
    if (args.destination == None):
        print("")
        parser.print_help()
        print("")
    else:
        client(args.destination, configarg)

except KeyboardInterrupt:
    print("")
    sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Channel.py>.

11.9 Buffer

The *Buffer* example explores using buffered readers and writers to send binary data between peers of a Link.

```

#####
# This RNS example demonstrates how to set up a link to #
# a destination, and pass binary data over it using a #
# channel buffer. #
#####
from __future__ import annotations
import os
import sys
import time
import argparse
from datetime import datetime

import RNS
from RNS.vendor import umsgpack

# Let's define an app name. We'll use this for all
# destinations we create. Since this echo example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

#####
#### Server Part #####
#####

```

(continues on next page)

(continued from previous page)

```

# A reference to the latest client link that connected
latest_client_link = None

# A reference to the latest buffer object
latest_buffer = None

# This initialisation is executed when the users chooses
# to run as a server
def server(configpath):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our example
    server_identity = RNS.Identity()

    # We create a destination that clients can connect to. We
    # want clients to create links to this destination, so we
    # need to create a "single" destination type.
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.IN,
        RNS.Destination.SINGLE,
        APP_NAME,
        "bufferexample"
    )

    # We configure a function that will get called every time
    # a new client creates a link to this destination.
    server_destination.set_link_established_callback(client_connected)

    # Everything's ready!
    # Let's Wait for client requests or user input
    server_loop(server_destination)

def server_loop(destination):
    # Let the user know that everything is ready
    RNS.log(
        "Link buffer example "+
        RNS.prettyhexrep(destination.hash)+
        " running, waiting for a connection."
    )

    RNS.log("Hit enter to manually send an announce (Ctrl-C to quit)")

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.
    while True:
        entered = input()
        destination.announce()

```

(continues on next page)

(continued from previous page)

```

    RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

# When a client establishes a link to our server
# destination, this function will be called with
# a reference to the link.
def client_connected(link):
    global latest_client_link, latest_buffer
    latest_client_link = link

    RNS.log("Client connected")
    link.set_link_closed_callback(client_disconnected)

    # If a new connection is received, the old reader
    # needs to be disconnected.
    if latest_buffer:
        latest_buffer.close()

    # Create buffer objects.
    # The stream_id parameter to these functions is
    # a bit like a file descriptor, except that it
    # is unique to the *receiver*.
    #
    # In this example, both the reader and the writer
    # use stream_id = 0, but there are actually two
    # separate unidirectional streams flowing in
    # opposite directions.
    #
    channel = link.get_channel()
    latest_buffer = RNS.Buffer.create_bidirectional_buffer(0, 0, channel, server_buffer_
↪ready)

def client_disconnected(link):
    RNS.log("Client disconnected")

def server_buffer_ready(ready_bytes: int):
    """
    Callback from buffer when buffer has data available

    :param ready_bytes: The number of bytes ready to read
    """
    global latest_buffer

    data = latest_buffer.read(ready_bytes)
    data = data.decode("utf-8")

    RNS.log("Received data over the buffer: " + data)

    reply_message = "I received \""+data+"\" over the buffer"
    reply_message = reply_message.encode("utf-8")
    latest_buffer.write(reply_message)
    latest_buffer.flush()

```

(continues on next page)

(continued from previous page)

```
#####
#### Client Part #####
#####

# A reference to the server link
server_link = None

# A reference to the buffer object, needed to share the
# object from the link connected callback to the client
# loop.
buffer = None

# This initialisation is executed when the users chooses
# to run as a client
def client(destination_hexhash, configpath):
    # We need a binary representation of the destination
    # hash that was entered on the command line
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Destination length is invalid, must be {hex} hexadecimal characters (
↳{byte} bytes)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Invalid destination entered. Check your input!\n")
        sys.exit(0)

    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Check if we know a path to the destination
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Destination is not yet known. Requesting path and waiting for announce_
↳to arrive...")
        RNS.Transport.request_path(destination_hash)
        while not RNS.Transport.has_path(destination_hash):
            time.sleep(0.1)

    # Recall the server identity
    server_identity = RNS.Identity.recall(destination_hash)

    # Inform the user that we'll begin connecting
    RNS.log("Establishing link with server...")

    # When the server identity is known, we set
```

(continues on next page)

(continued from previous page)

```

# up a destination
server_destination = RNS.Destination(
    server_identity,
    RNS.Destination.OUT,
    RNS.Destination.SINGLE,
    APP_NAME,
    "bufferexample"
)

# And create a link
link = RNS.Link(server_destination)

# We'll also set up functions to inform the
# user when the link is established or closed
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# Everything is set up, so let's enter a loop
# for the user to interact with the example
client_loop()

def client_loop():
    global server_link

    # Wait for the link to become active
    while not server_link:
        time.sleep(0.1)

    should_quit = False
    while not should_quit:
        try:
            print("> ", end=" ")
            text = input()

            # Check if we should quit the example
            if text == "quit" or text == "q" or text == "exit":
                should_quit = True
                server_link.teardown()
            else:
                # Otherwise, encode the text and write it to the buffer.
                text = text.encode("utf-8")
                buffer.write(text)
                # Flush the buffer to force the data to be sent.
                buffer.flush()

        except Exception as e:
            RNS.log("Error while sending data over the link buffer: "+str(e))
            should_quit = True
            server_link.teardown()

# This function is called when a link

```

(continues on next page)

(continued from previous page)

```

# has been established with the server
def link_established(link):
    # We store a reference to the link
    # instance for later use
    global server_link, buffer
    server_link = link

    # Create buffer, see server_client_connected() for
    # more detail about setting up the buffer.
    channel = link.get_channel()
    buffer = RNS.Buffer.create_bidirectional_buffer(0, 0, channel, client_buffer_ready)

    # Inform the user that the server is
    # connected
    RNS.log("Link established with server, enter some text to send, or \"quit\" to quit")

# When a link is closed, we'll inform the
# user, and exit the program
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT:
        RNS.log("The link timed out, exiting now")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED:
        RNS.log("The link was closed by the server, exiting now")
    else:
        RNS.log("Link closed, exiting now")

    time.sleep(1.5)
    sys.exit(0)

# When the buffer has new data, read it and write it to the terminal.
def client_buffer_ready(ready_bytes: int):
    global buffer
    data = buffer.read(ready_bytes)
    RNS.log("Received data over the link buffer: " + data.decode("utf-8"))
    print("> ", end=" ")
    sys.stdout.flush()

#####
#### Program Startup #####
#####

# This part of the program runs at startup,
# and parses input of from the user, and then
# starts up the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(description="Simple buffer example")

        parser.add_argument(
            "-s",
            "--server",

```

(continues on next page)

(continued from previous page)

```

        action="store_true",
        help="wait for incoming link requests from clients"
    )

    parser.add_argument(
        "--config",
        action="store",
        default=None,
        help="path to alternative Reticulum config directory",
        type=str
    )

    parser.add_argument(
        "destination",
        nargs="?",
        default=None,
        help="hexadecimal hash of the server destination",
        type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    if args.server:
        server(configarg)
    else:
        if (args.destination == None):
            print("")
            parser.print_help()
            print("")
        else:
            client(args.destination, configarg)

    except KeyboardInterrupt:
        print("")
        sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Buffer.py>.

11.10 Filetransfer

The *Filetransfer* example implements a basic file-server program that allow clients to connect and download files. The program uses the Resource interface to efficiently pass files of any size over a Reticulum *Link*.

```

#####
# This RNS example demonstrates a simple filetransfer #
# server and client program. The server will serve a #
# directory of files, and the clients can list and   #

```

(continues on next page)

(continued from previous page)

```

# download files from the server.                                #
#                                                                #
# Please note that using RNS Resources for large file           #
# transfers is not recommended, since compression,             #
# encryption and hashmap sequencing can take a long time       #
# on systems with slow CPUs, which will probably result        #
# in the client timing out before the resource sender           #
# can complete preparing the resource.                          #
#                                                                #
# If you need to transfer large files, use the Bundle           #
# class instead, which will automatically slice the data       #
# into chunks suitable for packing as a Resource.              #
#####

import os
import sys
import time
import threading
import argparse
import RNS
import RNS.vendor.umsgpack as umsgpack

# Let's define an app name. We'll use this for all
# destinations we create. Since this echo example
# is part of a range of example utilities, we'll put
# them all within the app namespace "example_utilities"
APP_NAME = "example_utilities"

# We'll also define a default timeout, in seconds
APP_TIMEOUT = 45.0

#####
#### Server Part #####
#####

serve_path = None

# This initialisation is executed when the users chooses
# to run as a server
def server(configpath, path):
    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Randomly create a new identity for our file server
    server_identity = RNS.Identity()

    global serve_path
    serve_path = path

    # We create a destination that clients can connect to. We
    # want clients to create links to this destination, so we
    # need to create a "single" destination type.

```

(continues on next page)

(continued from previous page)

```

server_destination = RNS.Destination(
    server_identity,
    RNS.Destination.IN,
    RNS.Destination.SINGLE,
    APP_NAME,
    "filetransfer",
    "server"
)

# We configure a function that will get called every time
# a new client creates a link to this destination.
server_destination.set_link_established_callback(client_connected)

# Everything's ready!
# Let's Wait for client requests or user input
announceLoop(server_destination)

def announceLoop(destination):
    # Let the user know that everything is ready
    RNS.log("File server "+RNS.prettyhexrep(destination.hash)+" running")
    RNS.log("Hit enter to manually send an announce (Ctrl-C to quit)")

    # We enter a loop that runs until the users exits.
    # If the user hits enter, we will announce our server
    # destination on the network, which will let clients
    # know how to create messages directed towards it.
    while True:
        entered = input()
        destination.announce()
        RNS.log("Sent announce from "+RNS.prettyhexrep(destination.hash))

# Here's a convenience function for listing all files
# in our served directory
def list_files():
    # We add all entries from the directory that are
    # actual files, and does not start with "."
    global serve_path
    return [file for file in os.listdir(serve_path) if os.path.isfile(os.path.join(serve_
    ↪path, file)) and file[:1] != "."]

# When a client establishes a link to our server
# destination, this function will be called with
# a reference to the link. We then send the client
# a list of files hosted on the server.
def client_connected(link):
    # Check if the served directory still exists
    if os.path.isdir(serve_path):
        RNS.log("Client connected, sending file list...")

        link.set_link_closed_callback(client_disconnected)

    # We pack a list of files for sending in a packet

```

(continues on next page)

(continued from previous page)

```

data = umsgpack.packb(list_files())

# Check the size of the packed data
if len(data) <= RNS.Link.MDU:
    # If it fits in one packet, we will just
    # send it as a single packet over the link.
    list_packet = RNS.Packet(link, data)
    list_receipt = list_packet.send()
    list_receipt.set_timeout(APP_TIMEOUT)
    list_receipt.set_delivery_callback(list_delivered)
    list_receipt.set_timeout_callback(list_timeout)
else:
    RNS.log("Too many files in served directory!", RNS.LOG_ERROR)
    RNS.log("You should implement a function to split the filelist over multiple_
↪packets.", RNS.LOG_ERROR)
    RNS.log("Hint: The client already supports it :)", RNS.LOG_ERROR)

    # After this, we're just going to keep the link
    # open until the client requests a file. We'll
    # configure a function that get's called when
    # the client sends a packet with a file request.
    link.set_packet_callback(client_request)
else:
    RNS.log("Client connected, but served path no longer exists!", RNS.LOG_ERROR)
    link.teardown()

def client_disconnected(link):
    RNS.log("Client disconnected")

def client_request(message, packet):
    global serve_path

    try:
        filename = message.decode("utf-8")
    except Exception as e:
        filename = None

    if filename in list_files():
        try:
            # If we have the requested file, we'll
            # read it and pack it as a resource
            RNS.log("Client requested \""+filename+"\"")
            file = open(os.path.join(serve_path, filename), "rb")

            file_resource = RNS.Resource(
                file,
                packet.link,
                callback=resource_sending_concluded
            )

            file_resource.filename = filename
        except Exception as e:

```

(continues on next page)

(continued from previous page)

```

        # If somethign went wrong, we close
        # the link
        RNS.log("Error while reading file \""+filename+"\"", RNS.LOG_ERROR)
        packet.link.teardown()
        raise e
    else:
        # If we don't have it, we close the link
        RNS.log("Client requested an unknown file")
        packet.link.teardown()

# This function is called on the server when a
# resource transfer concludes.
def resource_sending_concluded(resource):
    if hasattr(resource, "filename"):
        name = resource.filename
    else:
        name = "resource"

    if resource.status == RNS.Resource.COMPLETE:
        RNS.log("Done sending \""+name+"\" to client")
    elif resource.status == RNS.Resource.FAILED:
        RNS.log("Sending \""+name+"\" to client failed")

def list_delivered(receipt):
    RNS.log("The file list was received by the client")

def list_timeout(receipt):
    RNS.log("Sending list to client timed out, closing this link")
    link = receipt.destination
    link.teardown()

#####
#### Client Part #####
#####

# We store a global list of files available on the server
server_files = []

# A reference to the server link
server_link = None

# And a reference to the current download
current_download = None
current_filename = None

# Variables to store download statistics
download_started = 0
download_finished = 0
download_time = 0
transfer_size = 0
file_size = 0

```

(continues on next page)

(continued from previous page)

```

# This initialisation is executed when the users chooses
# to run as a client
def client(destination_hexhash, configpath):
    # We need a binary representation of the destination
    # hash that was entered on the command line
    try:
        dest_len = (RNS.Reticulum.TRUNCATED_HASHLENGTH//8)*2
        if len(destination_hexhash) != dest_len:
            raise ValueError(
                "Destination length is invalid, must be {hex} hexadecimal characters (
↳{byte} bytes)".format(hex=dest_len, byte=dest_len//2)
            )

        destination_hash = bytes.fromhex(destination_hexhash)
    except:
        RNS.log("Invalid destination entered. Check your input!\n")
        sys.exit(0)

    # We must first initialise Reticulum
    reticulum = RNS.Reticulum(configpath)

    # Check if we know a path to the destination
    if not RNS.Transport.has_path(destination_hash):
        RNS.log("Destination is not yet known. Requesting path and waiting for announce_
↳to arrive...")
        RNS.Transport.request_path(destination_hash)
        while not RNS.Transport.has_path(destination_hash):
            time.sleep(0.1)

    # Recall the server identity
    server_identity = RNS.Identity.recall(destination_hash)

    # Inform the user that we'll begin connecting
    RNS.log("Establishing link with server...")

    # When the server identity is known, we set
    # up a destination
    server_destination = RNS.Destination(
        server_identity,
        RNS.Destination.OUT,
        RNS.Destination.SINGLE,
        APP_NAME,
        "filetransfer",
        "server"
    )

    # We also want to automatically prove incoming packets
    server_destination.set_proof_strategy(RNS.Destination.PROVE_ALL)

    # And create a link

```

(continues on next page)

(continued from previous page)

```

link = RNS.Link(server_destination)

# We expect any normal data packets on the link
# to contain a list of served files, so we set
# a callback accordingly
link.set_packet_callback(filelist_received)

# We'll also set up functions to inform the
# user when the link is established or closed
link.set_link_established_callback(link_established)
link.set_link_closed_callback(link_closed)

# And set the link to automatically begin
# downloading advertised resources
link.set_resource_strategy(RNS.Link.ACCEPT_ALL)
link.set_resource_started_callback(download_began)
link.set_resource_concluded_callback(download_concluded)

menu()

# Requests the specified file from the server
def download(filename):
    global server_link, menu_mode, current_filename, transfer_size, download_started
    current_filename = filename
    download_started = 0
    transfer_size = 0

    # We just create a packet containing the
    # requested filename, and send it down the
    # link. We also specify we don't need a
    # packet receipt.
    request_packet = RNS.Packet(server_link, filename.encode("utf-8"), create_
↪receipt=False)
    request_packet.send()

    print("")
    print(("Requested \""+filename+"\" from server, waiting for download to begin..."))
    menu_mode = "download_started"

# This function runs a simple menu for the user
# to select which files to download, or quit
menu_mode = None
def menu():
    global server_files, server_link
    # Wait until we have a filelist
    while len(server_files) == 0:
        time.sleep(0.1)
    RNS.log("Ready!")
    time.sleep(0.5)

    global menu_mode
    menu_mode = "main"

```

(continues on next page)

(continued from previous page)

```

should_quit = False
while (not should_quit):
    print_menu()

    while not menu_mode == "main":
        # Wait
        time.sleep(0.25)

    user_input = input()
    if user_input == "q" or user_input == "quit" or user_input == "exit":
        should_quit = True
        print("")
    else:
        if user_input in server_files:
            download(user_input)
        else:
            try:
                if 0 <= int(user_input) < len(server_files):
                    download(server_files[int(user_input)])
            except:
                pass

    if should_quit:
        server_link.teardown()

# Prints out menus or screens for the
# various states of the client program.
# It's simple and quite uninteresting.
# I won't go into detail here. Just
# strings basically.
def print_menu():
    global menu_mode, download_time, download_started, download_finished, transfer_size, \
    ↪ file_size

    if menu_mode == "main":
        clear_screen()
        print_filelist()
        print("")
        print("Select a file to download by entering name or number, or q to quit")
        print(("> "), end=' ')
    elif menu_mode == "download_started":
        download_began = time.time()
        while menu_mode == "download_started":
            time.sleep(0.1)
            if time.time() > download_began+APP_TIMEOUT:
                print("The download timed out")
                time.sleep(1)
                server_link.teardown()

    if menu_mode == "downloading":
        print("Download started")
        print("")

```

(continues on next page)

(continued from previous page)

```

while menu_mode == "downloading":
    global current_download
    percent = round(current_download.get_progress() * 100.0, 1)
    print("\rProgress: "+str(percent)+" %  "), end=' ')
    sys.stdout.flush()
    time.sleep(0.1)

if menu_mode == "save_error":
    print("\rProgress: 100.0 %"), end=' ')
    sys.stdout.flush()
    print("")
    print("Could not write downloaded file to disk")
    current_download.status = RNS.Resource.FAILED
    menu_mode = "download_concluded"

if menu_mode == "download_concluded":
    if current_download.status == RNS.Resource.COMPLETE:
        print("\rProgress: 100.0 %"), end=' ')
        sys.stdout.flush()

        # Print statistics
        hours, rem = divmod(download_time, 3600)
        minutes, seconds = divmod(rem, 60)
        timestring = "{:0>2}:{:0>2}:{:05.2f}".format(int(hours),int(minutes),seconds)
        print("")
        print("")
        print("--- Statistics ----")
        print("\tTime taken      : "+timestring)
        print("\tFile size       : "+size_str(file_size))
        print("\tData transferred : "+size_str(transfer_size))
        print("\tEffective rate   : "+size_str(file_size/download_time, suffix='b')+
↪"/s")
        print("\tTransfer rate    : "+size_str(transfer_size/download_time, suffix='b'
↪')+"/s")
        print("")
        print("The download completed! Press enter to return to the menu.")
        print("")
        input()

    else:
        print("")
        print("The download failed! Press enter to return to the menu.")
        input()

    current_download = None
    menu_mode = "main"
    print_menu()

# This function prints out a list of files
# on the connected server.
def print_filelist():
    global server_files

```

(continues on next page)

(continued from previous page)

```

print("Files on server:")
for index,file in enumerate(server_files):
    print("\t("+str(index)+")\t"+file)

def filelist_received(filelist_data, packet):
    global server_files, menu_mode
    try:
        # Unpack the list and extend our
        # local list of available files
        filelist = umsgpack.unpackb(filelist_data)
        for file in filelist:
            if not file in server_files:
                server_files.append(file)

        # If the menu is already visible,
        # we'll update it with what was
        # just received
        if menu_mode == "main":
            print_menu()
    except:
        RNS.log("Invalid file list data received, closing link")
        packet.link.teardown()

# This function is called when a link
# has been established with the server
def link_established(link):
    # We store a reference to the link
    # instance for later use
    global server_link
    server_link = link

    # Inform the user that the server is
    # connected
    RNS.log("Link established with server")
    RNS.log("Waiting for filelist...")

    # And set up a small job to check for
    # a potential timeout in receiving the
    # file list
    thread = threading.Thread(target=filelist_timeout_job, daemon=True)
    thread.start()

# This job just sleeps for the specified
# time, and then checks if the file list
# was received. If not, the program will
# exit.
def filelist_timeout_job():
    time.sleep(APP_TIMEOUT)

    global server_files
    if len(server_files) == 0:
        RNS.log("Timed out waiting for filelist, exiting")

```

(continues on next page)

(continued from previous page)

```

        sys.exit(0)

# When a link is closed, we'll inform the
# user, and exit the program
def link_closed(link):
    if link.teardown_reason == RNS.Link.TIMEOUT:
        RNS.log("The link timed out, exiting now")
    elif link.teardown_reason == RNS.Link.DESTINATION_CLOSED:
        RNS.log("The link was closed by the server, exiting now")
    else:
        RNS.log("Link closed, exiting now")

    time.sleep(1.5)
    sys.exit(0)

# When RNS detects that the download has
# started, we'll update our menu state
# so the user can be shown a progress of
# the download.
def download_began(resource):
    global menu_mode, current_download, download_started, transfer_size, file_size
    current_download = resource

    if download_started == 0:
        download_started = time.time()

    transfer_size += resource.size
    file_size = resource.total_size

    menu_mode = "downloading"

# When the download concludes, successfully
# or not, we'll update our menu state and
# inform the user about how it all went.
def download_concluded(resource):
    global menu_mode, current_filename, download_started, download_finished, download_
    ↪time
    download_finished = time.time()
    download_time = download_finished - download_started

    saved_filename = current_filename

    if resource.status == RNS.Resource.COMPLETE:
        counter = 0
        while os.path.isfile(saved_filename):
            counter += 1
            saved_filename = current_filename+"."+str(counter)

        try:
            file = open(saved_filename, "wb")
            file.write(resource.data.read())

```

(continues on next page)

(continued from previous page)

```

        file.close()
        menu_mode = "download_concluded"
    except:
        menu_mode = "save_error"
else:
    menu_mode = "download_concluded"

# A convenience function for printing a human-
# readable file size
def size_str(num, suffix='B'):
    units = ['', 'Ki', 'Mi', 'Gi', 'Ti', 'Pi', 'Ei', 'Zi']
    last_unit = 'Yi'

    if suffix == 'b':
        num *= 8
        units = ['', 'K', 'M', 'G', 'T', 'P', 'E', 'Z']
        last_unit = 'Y'

    for unit in units:
        if abs(num) < 1024.0:
            return "%3.2f %s%s" % (num, unit, suffix)
        num /= 1024.0
    return "%.2f %s%s" % (num, last_unit, suffix)

# A convenience function for clearing the screen
def clear_screen():
    os.system('cls' if os.name=='nt' else 'clear')

#####
#### Program Startup #####
#####

# This part of the program runs at startup,
# and parses input of from the user, and then
# starts up the desired program mode.
if __name__ == "__main__":
    try:
        parser = argparse.ArgumentParser(
            description="Simple file transfer server and client utility"
        )

        parser.add_argument(
            "-s",
            "--serve",
            action="store",
            metavar="dir",
            help="serve a directory of files to clients"
        )

        parser.add_argument(
            "--config",
            action="store",

```

(continues on next page)

(continued from previous page)

```

        default=None,
        help="path to alternative Reticulum config directory",
        type=str
    )

    parser.add_argument(
        "destination",
        nargs="?",
        default=None,
        help="hexadecimal hash of the server destination",
        type=str
    )

    args = parser.parse_args()

    if args.config:
        configarg = args.config
    else:
        configarg = None

    if args.serve:
        if os.path.isdir(args.serve):
            server(configarg, args.serve)
        else:
            RNS.log("The specified directory does not exist")
    else:
        if (args.destination == None):
            print("")
            parser.print_help()
            print("")
        else:
            client(args.destination, configarg)

except KeyboardInterrupt:
    print("")
    sys.exit(0)

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/Filetransfer.py>.

11.11 Custom Interfaces

The *ExampleInterface* demonstrates creating custom interfaces for Reticulum. Any number of custom interfaces can be loaded and utilised by Reticulum, and will be fully on-par with natively included interfaces, including all supported *interface modes* and *common configuration options*.

```

# This example illustrates creating a custom interface
# definition, that can be loaded and used by Reticulum at
# runtime. Any number of custom interfaces can be created
# and loaded. To use the interface place it in the folder
# ~/.reticulum/interfaces, and add an interface entry to
# your Reticulum configuration file similar to this:

```

(continues on next page)

(continued from previous page)

```

# [[Example Custom Interface]]
#     type = ExampleInterface
#     enabled = no
#     mode = gateway
#     port = /dev/ttyUSB0
#     speed = 115200
#     databits = 8
#     parity = none
#     stopbits = 1

from time import sleep
import sys
import threading
import time

# This HDLC helper class is used by the interface
# to delimit and packetize data over the physical
# medium - in this case a serial connection.
class HDLC():
    # This example interface packetizes data using
    # simplified HDLC framing, similar to PPP
    FLAG      = 0x7E
    ESC       = 0x7D
    ESC_MASK  = 0x20

    @staticmethod
    def escape(data):
        data = data.replace(bytes([HDLC.ESC]), bytes([HDLC.ESC, HDLC.ESC^HDLC.ESC_MASK]))
        data = data.replace(bytes([HDLC.FLAG]), bytes([HDLC.ESC, HDLC.FLAG^HDLC.ESC_
→MASK]))
        return data

# Let's define our custom interface class. It must
# be a sub-class of the RNS "Interface" class.
class ExampleInterface(Interface):
    # All interface classes must define a default
    # IFAC size, used in IFAC setup when the user
    # has not specified a custom IFAC size. This
    # option is specified in bytes.
    DEFAULT_IFAC_SIZE = 8

    # The following properties are local to this
    # particular interface implementation.
    owner      = None
    port       = None
    speed      = None
    databits   = None
    parity     = None
    stopbits   = None
    serial     = None

    # All Reticulum interfaces must have an __init__

```

(continues on next page)

(continued from previous page)

```

# method that takes 2 positional arguments:
# The owner RNS Transport instance, and a dict
# of configuration values.
def __init__(self, owner, configuration):

    # The following lines demonstrate handling
    # potential dependencies required for the
    # interface to function correctly.
    import importlib
    if importlib.util.find_spec('serial') != None:
        import serial
    else:
        RNS.log("Using this interface requires a serial communication module to be
↳ installed.", RNS.LOG_CRITICAL)
        RNS.log("You can install one with the command: python3 -m pip install
↳ pyserial", RNS.LOG_CRITICAL)
        RNS.panic()

    # We start out by initialising the super-class
    super().__init__()

    # To make sure the configuration data is in the
    # correct format, we parse it through the following
    # method on the generic Interface class. This step
    # is required to ensure compatibility on all the
    # platforms that Reticulum supports.
    ifconf = Interface.get_config_obj(configuration)

    # Read the interface name from the configuration
    # and set it on our interface instance.
    name = ifconf["name"]
    self.name = name

    # We read configuration parameters from the supplied
    # configuration data, and provide default values in
    # case any are missing.
    port = ifconf["port"] if "port" in ifconf else None
    speed = int(ifconf["speed"]) if "speed" in ifconf else 9600
    databits = int(ifconf["databits"]) if "databits" in ifconf else 8
    parity = ifconf["parity"] if "parity" in ifconf else "N"
    stopbits = int(ifconf["stopbits"]) if "stopbits" in ifconf else 1

    # In case no port is specified, we abort setup by
    # raising an exception.
    if port == None:
        raise ValueError(f"No port specified for {self}")

    # All interfaces must supply a hardware MTU value
    # to the RNS Transport instance. This value should
    # be the maximum data packet payload size that the
    # underlying medium is capable of handling in all
    # cases without any segmentation.

```

(continues on next page)

(continued from previous page)

```

self.HW_MTU = 564

# We initially set the "online" property to false,
# since the interface has not actually been fully
# initialised and connected yet.
self.online = False

# In this case, we can also set the indicated bit-
# rate of the interface to the serial port speed.
self.bitrate = speed

# Configure internal properties on the interface
# according to the supplied configuration.
self.pyserial = serial
self.serial = None
self.owner = owner
self.port = port
self.speed = speed
self.databits = databits
self.parity = serial.PARITY_NONE
self.stopbits = stopbits
self.timeout = 100

if parity.lower() == "e" or parity.lower() == "even":
    self.parity = serial.PARITY_EVEN

if parity.lower() == "o" or parity.lower() == "odd":
    self.parity = serial.PARITY_ODD

# Since all required parameters are now configured,
# we will try opening the serial port.
try:
    self.open_port()
except Exception as e:
    RNS.log("Could not open serial port for interface "+str(self), RNS.LOG_ERROR)
    raise e

# If opening the port succeeded, run any post-open
# configuration required.
if self.serial.is_open:
    self.configure_device()
else:
    raise IOError("Could not open serial port")

# Open the serial port with supplied configuration
# parameters and store a reference to the open port.
def open_port(self):
    RNS.log("Opening serial port "+self.port+"...", RNS.LOG_VERBOSE)
    self.serial = self.pyserial.Serial(
        port = self.port,
        baudrate = self.speed,
        bytesize = self.databits,

```

(continues on next page)

(continued from previous page)

```

        parity = self.parity,
        stopbits = self.stopbits,
        xonxoff = False,
        rtscts = False,
        timeout = 0,
        inter_byte_timeout = None,
        write_timeout = None,
        dsrdtr = False,
    )

    # The only thing required after opening the port
    # is to wait a small amount of time for the
    # hardware to initialise and then start a thread
    # that reads any incoming data from the device.
    def configure_device(self):
        sleep(0.5)
        thread = threading.Thread(target=self.read_loop)
        thread.daemon = True
        thread.start()
        self.online = True
        RNS.log("Serial port "+self.port+" is now open", RNS.LOG_VERBOSE)

    # This method will be called from our read-loop
    # whenever a full packet has been received over
    # the underlying medium.
    def process_incoming(self, data):
        # Update our received bytes counter
        self.rxb += len(data)

        # And send the data packet to the Transport
        # instance for processing.
        self.owner.inbound(data, self)

    # The running Reticulum Transport instance will
    # call this method on the interface whenever the
    # interface must transmit a packet.
    def process_outgoing(self, data):
        if self.online:
            # First, escape and packetize the data
            # according to HDLC framing.
            data = bytes([HDLC.FLAG])+HDLC.escape(data)+bytes([HDLC.FLAG])

            # Then write the framed data to the port
            written = self.serial.write(data)

            # Update the transmitted bytes counter
            # and ensure that all data was written
            self.txb += len(data)
            if written != len(data):
                raise IOError("Serial interface only wrote "+str(written)+" bytes of
→ "+str(len(data)))

```

(continues on next page)

(continued from previous page)

```

# This read loop runs in a thread and continuously
# receives bytes from the underlying serial port.
# When a full packet has been received, it will
# be sent to the process_incoming method, which
# will in turn pass it to the Transport instance.
def read_loop(self):
    try:
        in_frame = False
        escape = False
        data_buffer = b""
        last_read_ms = int(time.time()*1000)

        while self.serial.is_open:
            if self.serial.in_waiting:
                byte = ord(self.serial.read(1))
                last_read_ms = int(time.time()*1000)

                if (in_frame and byte == HDLC.FLAG):
                    in_frame = False
                    self.process_incoming(data_buffer)
                elif (byte == HDLC.FLAG):
                    in_frame = True
                    data_buffer = b""
                elif (in_frame and len(data_buffer) < self.HW_MTU):
                    if (byte == HDLC.ESC):
                        escape = True
                    else:
                        if (escape):
                            if (byte == HDLC.FLAG ^ HDLC.ESC_MASK):
                                byte = HDLC.FLAG
                            if (byte == HDLC.ESC ^ HDLC.ESC_MASK):
                                byte = HDLC.ESC
                            escape = False
                        data_buffer = data_buffer+bytes([byte])

                else:
                    time_since_last = int(time.time()*1000) - last_read_ms
                    if len(data_buffer) > 0 and time_since_last > self.timeout:
                        data_buffer = b""
                        in_frame = False
                        escape = False
                    sleep(0.08)

            except Exception as e:
                self.online = False
                RNS.log("A serial port error occurred, the contained exception was: "+str(e),
↳ RNS.LOG_ERROR)
                RNS.log("The interface "+str(self)+" experienced an unrecoverable error and_
↳ is now offline.", RNS.LOG_ERROR)

                if RNS.Reticulum.panic_on_interface_error:

```

(continues on next page)

(continued from previous page)

```

        RNS.panic()

        RNS.log("Reticulum will attempt to reconnect the interface periodically.",
↳RNS.LOG_ERROR)

        self.online = False
        self.serial.close()
        self.reconnect_port()

        # This method handles serial port disconnects.
        def reconnect_port(self):
            while not self.online:
                try:
                    time.sleep(5)
                    RNS.log("Attempting to reconnect serial port "+str(self.port)+" for
↳"+str(self)+"...", RNS.LOG_VERBOSE)
                    self.open_port()
                    if self.serial.is_open:
                        self.configure_device()
                except Exception as e:
                    RNS.log("Error while reconnecting port, the contained exception was:
↳"+str(e), RNS.LOG_ERROR)

            RNS.log("Reconnected serial port for "+str(self))

        # Signal to Reticulum that this interface should
        # not perform any ingress limiting.
        def should_ingress_limit(self):
            return False

        # We must provide a string representation of this
        # interface, that is used whenever the interface
        # is printed in logs or external programs.
        def __str__(self):
            return "ExampleInterface["+self.name+"]"

        # Finally, register the defined interface class as the
        # target class for Reticulum to use as an interface
        interface_class = ExampleInterface

```

This example can also be found at <https://github.com/markqvist/Reticulum/blob/master/Examples/ExampleInterface.py>.

RETICULUM LICENSE

Reticulum License

Copyright (c) 2016-2026 Mark Qvist

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

- The Software shall not be used in any kind of system which includes amongst its functions the ability to purposefully do harm to human beings.
- The Software shall not be used, directly or indirectly, in the creation of an artificial intelligence, machine learning or language model training dataset, including but not limited to any use that contributes to the training or development of such a model or algorithm.
- The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

API REFERENCE

Communication over Reticulum networks is achieved by using a simple set of classes exposed by the RNS API. This chapter lists and explains all classes exposed by the Reticulum Network Stack API, along with their method signatures and usage. It can be used as a reference while writing applications that utilise Reticulum, or it can be read in entirety to gain an understanding of the complete functionality of RNS from a developers perspective.

13.1 Reticulum

```
class RNS.Reticulum(configdir=None, loglevel=None, logdest=None, verbosity=None,  
                    require_shared_instance=False, shared_instance_type=None)
```

This class is used to initialise access to Reticulum within a program. You must create exactly one instance of this class before carrying out any other RNS operations, such as creating destinations or sending traffic. Every independently executed program must create their own instance of the Reticulum class, but Reticulum will automatically handle inter-program communication on the same system, and expose all connected programs to external interfaces as well.

As soon as an instance of this class is created, Reticulum will start opening and configuring any hardware devices specified in the supplied configuration.

Currently the first running instance must be kept running while other local instances are connected, as the first created instance will act as a master instance that directly communicates with external hardware such as modems, TNCs and radios. If a master instance is asked to exit, it will not exit until all client processes have terminated (unless killed forcibly).

If you are running Reticulum on a system with several different programs that use RNS starting and terminating at different times, it will be advantageous to run a master RNS instance as a daemon for other programs to use on demand.

MTU = 500

The MTU that Reticulum adheres to, and will expect other peers to adhere to. By default, the MTU is 500 bytes. In custom RNS network implementations, it is possible to change this value, but doing so will completely break compatibility with all other RNS networks. An identical MTU is a prerequisite for peers to communicate in the same network.

Unless you really know what you are doing, the MTU should be left at the default value.

LINK_MTU_DISCOVERY = True

Whether automatic link MTU discovery is enabled by default in this release. Link MTU discovery significantly increases throughput over fast links, but requires all intermediary hops to also support it. Support for this feature was added in RNS version 0.9.0. This option will become enabled by default in the near future. Please update your RNS instances.

ANNOUNCE_CAP = 2

The maximum percentage of interface bandwidth that, at any given time, may be used to propagate announcements. If an announce was scheduled for broadcasting on an interface, but doing so would exceed the allowed bandwidth allocation, the announce will be queued for transmission when there is bandwidth available.

Reticulum will always prioritise propagating announces with fewer hops, ensuring that distant, large networks with many peers on fast links don't overwhelm the capacity of smaller networks on slower mediums. If an announce remains queued for an extended amount of time, it will eventually be dropped.

This value will be applied by default to all created interfaces, but it can be configured individually on a per-interface basis. In general, the global default setting should not be changed, and any alterations should be made on a per-interface basis instead.

MINIMUM_BITRATE = 5

Minimum bitrate required across a medium for Reticulum to be able to successfully establish links. Currently 5 bits per second.

static get_instance()

Return the currently running Reticulum instance

static should_use_implicit_proof()

Returns whether proofs sent are explicit or implicit.

Returns

True if the current running configuration specifies to use implicit proofs. False if not.

static transport_enabled()

Returns whether Transport is enabled for the running instance.

When Transport is enabled, Reticulum will route traffic for other peers, respond to path requests and pass announces over the network.

Returns

True if Transport is enabled, False if not.

static link_mtu_discovery()

Returns whether link MTU discovery is enabled for the running instance.

When link MTU discovery is enabled, Reticulum will automatically upgrade link MTUs to the highest supported value, increasing transfer speed and efficiency.

Returns

True if link MTU discovery is enabled, False if not.

static remote_management_enabled()

Returns whether remote management is enabled for the running instance.

When remote management is enabled, authenticated peers can remotely query and manage this instance.

Returns

True if remote management is enabled, False if not.

static required_discovery_value()

Returns the required stamp value for a discovered interface to be considered valid and remembered.

Returns

The required stamp value as an integer.

static publish_blackhole_enabled()

Returns whether blackhole list publishing is enabled for the running instance.

Returns

True if blackhole list publishing is enabled, False if not.

static blackhole_sources()

Returns the list of transport identity hashes from which blackhole lists are sourced.

Returns

A list of identity hashes.

static discovered_interfaces()

Returns a list of interfaces discovered over the network.

Returns

A list of discovered interfaces.

static interface_discovery_sources()

Returns the list of network identity hashes from which interfaces are discovered.

Returns

A list of identity hashes.

13.2 Identity

class RNS.Identity(*create_keys=True*)

This class is used to manage identities in Reticulum. It provides methods for encryption, decryption, signatures and verification, and is the basis for all encrypted communication over Reticulum networks.

Parameters

create_keys – Specifies whether new encryption and signing keys should be generated.

CURVE = 'Curve25519'

The curve used for Elliptic Curve DH key exchanges

KEYSIZE = 512

X.25519 key size in bits. A complete key is the concatenation of a 256 bit encryption key, and a 256 bit signing key.

RATCHETSIZE = 256

X.25519 ratchet key size in bits.

RATCHET_EXPIRY = 2592000

The expiry time for received ratchets in seconds, defaults to 30 days. Reticulum will always use the most recently announced ratchet, and remember it for up to RATCHET_EXPIRY since receiving it, after which it will be discarded. If a newer ratchet is announced in the meantime, it will be replace the already known ratchet.

TRUNCATED_HASHLENGTH = 128

Constant specifying the truncated hash length (in bits) used by Reticulum for addressable hashes and other purposes. Non-configurable.

static recall(*target_hash, from_identity_hash=False*)

Recall identity for a destination or identity hash. By default, this function will return the identity associated with a given *destination* hash. As an example, if you know the `lxf.delivery` destination hash of an endpoint, this function will return the associated underlying identity. You can also search for an identity from a known *identity hash*, by setting the `from_identity_hash` argument.

Parameters

- **target_hash** – Destination or identity hash as *bytes*.
- **from_identity_hash** – Whether to search based on identity hash instead of destination hash as *bool*.

Returns

An *RNS.Identity* instance that can be used to create an outgoing *RNS.Destination*, or *None* if the destination is unknown.

static recall_app_data(destination_hash)

Recall last heard app_data for a destination hash.

Parameters

destination_hash – Destination hash as *bytes*.

Returns

Bytes containing app_data, or *None* if the destination is unknown.

static full_hash(data)

Get a SHA-256 hash of passed data.

Parameters

data – Data to be hashed as *bytes*.

Returns

SHA-256 hash as *bytes*.

static truncated_hash(data)

Get a truncated SHA-256 hash of passed data.

Parameters

data – Data to be hashed as *bytes*.

Returns

Truncated SHA-256 hash as *bytes*.

static get_random_hash()

Get a random SHA-256 hash.

Parameters

data – Data to be hashed as *bytes*.

Returns

Truncated SHA-256 hash of random data as *bytes*.

static current_ratchet_id(destination_hash)

Get the ID of the currently used ratchet key for a given destination hash

Parameters

destination_hash – A destination hash as *bytes*.

Returns

A ratchet ID as *bytes* or *None*.

static from_bytes(prv_bytes)

Create a new *RNS.Identity* instance from *bytes* of private key. Can be used to load previously created and saved identities into Reticulum.

Parameters

prv_bytes – The *bytes* of private a saved private key. **HAZARD!** Never use this to generate a new key by feeding random data in prv_bytes.

Returns

A *RNS.Identity* instance, or *None* if the *bytes* data was invalid.

static from_file(path)

Create a new *RNS.Identity* instance from a file. Can be used to load previously created and saved identities into Reticulum.

Parameters

path – The full path to the saved *RNS.Identity* data

Returns

A *RNS.Identity* instance, or *None* if the loaded data was invalid.

to_file(path)

Saves the identity to a file. This will write the private key to disk, and anyone with access to this file will be able to decrypt all communication for the identity. Be very careful with this method.

Parameters

path – The full path specifying where to save the identity.

Returns

True if the file was saved, otherwise False.

get_private_key()**Returns**

The private key as *bytes*

get_public_key()**Returns**

The public key as *bytes*

load_private_key(prv_bytes)

Load a private key into the instance.

Parameters

prv_bytes – The private key as *bytes*.

Returns

True if the key was loaded, otherwise False.

load_public_key(pub_bytes)

Load a public key into the instance.

Parameters

pub_bytes – The public key as *bytes*.

Returns

True if the key was loaded, otherwise False.

encrypt(plaintext, ratchet=None)

Encrypts information for the identity.

Parameters

plaintext – The plaintext to be encrypted as *bytes*.

Returns

Ciphertext token as *bytes*.

Raises

KeyError if the instance does not hold a public key.

decrypt(*ciphertext_token*, *ratchets=None*, *enforce_ratchets=False*, *ratchet_id_receiver=None*)

Decrypts information for the identity.

Parameters

ciphertext – The ciphertext to be decrypted as *bytes*.

Returns

Plaintext as *bytes*, or *None* if decryption fails.

Raises

KeyError if the instance does not hold a private key.

sign(*message*)

Signs information by the identity.

Parameters

message – The message to be signed as *bytes*.

Returns

Signature as *bytes*.

Raises

KeyError if the instance does not hold a private key.

validate(*signature*, *message*)

Validates the signature of a signed message.

Parameters

- **signature** – The signature to be validated as *bytes*.
- **message** – The message to be validated as *bytes*.

Returns

True if the signature is valid, otherwise False.

Raises

KeyError if the instance does not hold a public key.

13.3 Destination

class `RNS.Destination`(*identity*, *direction*, *type*, *app_name*, **aspects*)

A class used to describe endpoints in a Reticulum Network. Destination instances are used both to create outgoing and incoming endpoints. The destination type will decide if encryption, and what type, is used in communication with the endpoint. A destination can also announce its presence on the network, which will distribute necessary keys for encrypted communication with it.

Parameters

- **identity** – An instance of [RNS.Identity](#). Can hold only public keys for an outgoing destination, or holding private keys for an ingoing.
- **direction** – `RNS.Destination.IN` or `RNS.Destination.OUT`.
- **type** – `RNS.Destination.SINGLE`, `RNS.Destination.GROUP` or `RNS.Destination.PLAIN`.
- **app_name** – A string specifying the app name.
- ***aspects** – Any non-zero number of string arguments.

RATCHET_COUNT = 512

The default number of generated ratchet keys a destination will retain, if it has ratchets enabled.

RATCHET_INTERVAL = 1800

The minimum interval between rotating ratchet keys, in seconds.

static expand_name(identity, app_name, *aspects)

Returns

A string containing the full human-readable name of the destination, for an app_name and a number of aspects.

static app_and_aspects_from_name(full_name)

Returns

A tuple containing the app name and a list of aspects, for a full-name string.

static hash_from_name_and_identity(full_name, identity)

Returns

A destination name in adressable hash form, for a full name string and Identity instance.

static hash(identity, app_name, *aspects)

Returns

A destination name in adressable hash form, for an app_name and a number of aspects.

announce(app_data=None, path_response=False, attached_interface=None, tag=None, send=True)

Creates an announce packet for this destination and broadcasts it on all relevant interfaces. Application specific data can be added to the announce.

Parameters

- **app_data** – bytes containing the app_data.
- **path_response** – Internal flag used by *RNS.Transport*. Ignore.

accepts_links(accepts=None)

Set or query whether the destination accepts incoming link requests.

Parameters

accepts – If True or False, this method sets whether the destination accepts incoming link requests. If not provided or None, the method returns whether the destination currently accepts link requests.

Returns

True or False depending on whether the destination accepts incoming link requests, if the *accepts* parameter is not provided or None.

set_link_established_callback(callback)

Registers a function to be called when a link has been established to this destination.

Parameters

callback – A function or method with the signature *callback(link)* to be called when a new link is established with this destination.

set_packet_callback(callback)

Registers a function to be called when a packet has been received by this destination.

Parameters

callback – A function or method with the signature *callback(data, packet)* to be called when this destination receives a packet.

set_proof_requested_callback(*callback*)

Registers a function to be called when a proof has been requested for a packet sent to this destination. Allows control over when and if proofs should be returned for received packets.

Parameters

callback – A function or method to with the signature *callback(packet)* be called when a packet that requests a proof is received. The callback must return one of True or False. If the callback returns True, a proof will be sent. If it returns False, a proof will not be sent.

set_proof_strategy(*proof_strategy*)

Sets the destinations proof strategy.

Parameters

proof_strategy – One of `RNS.Destination.PROVE_NONE`, `RNS.Destination.PROVE_ALL` or `RNS.Destination.PROVE_APP`. If `RNS.Destination.PROVE_APP` is set, the *proof_requested_callback* will be called to determine whether a proof should be sent or not.

register_request_handler(*path*, *response_generator=None*, *allow=ALLOW_NONE*, *allowed_list=None*, *auto_compress=True*)

Registers a request handler.

Parameters

- **path** – The path for the request handler to be registered.
- **response_generator** – A function or method with the signature *response_generator(path, data, request_id, link_id, remote_identity, requested_at)* to be called. Whatever this function returns will be sent as a response to the requester. If the function returns None, no response will be sent.
- **allow** – One of `RNS.Destination.ALLOW_NONE`, `RNS.Destination.ALLOW_ALL` or `RNS.Destination.ALLOW_LIST`. If `RNS.Destination.ALLOW_LIST` is set, the request handler will only respond to requests for identified peers in the supplied list.
- **allowed_list** – A list of *bytes-like* [RNS.Identity](#) hashes.
- **auto_compress** – If True or False, determines whether automatic compression of responses should be carried out. If set to an integer value, responses will only be auto-compressed if under this size in bytes. If omitted, the default compression settings will be followed.

Raises

`ValueError` if any of the supplied arguments are invalid.

deregister_request_handler(*path*)

Deregisters a request handler.

Parameters

path – The path for the request handler to be deregistered.

Returns

True if the handler was deregistered, otherwise False.

enable_ratchets(*ratchets_path*)

Enables ratchets on the destination. When ratchets are enabled, Reticulum will automatically rotate the keys used to encrypt packets to this destination, and include the latest ratchet key in announces.

Enabling ratchets on a destination will provide forward secrecy for packets sent to that destination, even when sent outside a Link. The normal Reticulum Link establishment procedure already performs its own

ephemeral key exchange for each link establishment, which means that ratchets are not necessary to provide forward secrecy for links.

Enabling ratchets will have a small impact on announce size, adding 32 bytes to every sent announce.

Parameters

ratchets_path – The path to a file to store ratchet data in.

Returns

True if the operation succeeded, otherwise False.

enforce_ratchets()

When ratchet enforcement is enabled, this destination will never accept packets that use its base Identity key for encryption, but only accept packets encrypted with one of the retained ratchet keys.

set_retained_ratchets(*retained_ratchets*)

Sets the number of previously generated ratchet keys this destination will retain, and try to use when decrypting incoming packets. Defaults to `Destination.RATCHET_COUNT`.

Parameters

retained_ratchets – The number of generated ratchets to retain.

Returns

True if the operation succeeded, False if not.

set_ratchet_interval(*interval*)

Sets the minimum interval in seconds between ratchet key rotation. Defaults to `Destination.RATCHET_INTERVAL`.

Parameters

interval – The minimum interval in seconds.

Returns

True if the operation succeeded, False if not.

create_keys()

For a `RNS.Destination.GROUP` type destination, creates a new symmetric key.

Raises

`TypeError` if called on an incompatible type of destination.

get_private_key()

For a `RNS.Destination.GROUP` type destination, returns the symmetric private key.

Raises

`TypeError` if called on an incompatible type of destination.

load_private_key(*key*)

For a `RNS.Destination.GROUP` type destination, loads a symmetric private key.

Parameters

key – A *bytes-like* containing the symmetric key.

Raises

`TypeError` if called on an incompatible type of destination.

encrypt(*plaintext*)

Encrypts information for `RNS.Destination.SINGLE` or `RNS.Destination.GROUP` type destination.

Parameters

plaintext – A *bytes-like* containing the plaintext to be encrypted.

Raises

ValueError if destination does not hold a necessary key for encryption.

decrypt(*ciphertext*)

Decrypts information for RNS.Destination.SINGLE or RNS.Destination.GROUP type destination.

Parameters

ciphertext – Bytes containing the ciphertext to be decrypted.

Raises

ValueError if destination does not hold a necessary key for decryption.

sign(*message*)

Signs information for RNS.Destination.SINGLE type destination.

Parameters

message – Bytes containing the message to be signed.

Returns

A bytes-like containing the message signature, or None if the destination could not sign the message.

set_default_app_data(*app_data=None*)

Sets the default app_data for the destination. If set, the default app_data will be included in every announce sent by the destination, unless other app_data is specified in the *announce* method.

Parameters

app_data – A bytes-like containing the default app_data, or a callable returning a bytes-like containing the app_data.

clear_default_app_data()

Clears default app_data previously set for the destination.

13.4 Packet

class RNS.Packet(*destination, data, create_receipt=True*)

The Packet class is used to create packet instances that can be sent over a Reticulum network. Packets will automatically be encrypted if they are addressed to a RNS.Destination.SINGLE destination, RNS.Destination.GROUP destination or a [RNS.Link](#).

For RNS.Destination.GROUP destinations, Reticulum will use the pre-shared key configured for the destination. All packets to group destinations are encrypted with the same AES-256 key.

For RNS.Destination.SINGLE destinations, Reticulum will use a newly derived ephemeral AES-256 key for every packet.

For [RNS.Link](#) destinations, Reticulum will use per-link ephemeral keys, and offers **Forward Secrecy**.

Parameters

- **destination** – A [RNS.Destination](#) instance to which the packet will be sent.
- **data** – The data payload to be included in the packet as *bytes*.
- **create_receipt** – Specifies whether a [RNS.PacketReceipt](#) should be created when instantiating the packet.

ENCRYPTED_MDU = 383

The maximum size of the payload data in a single encrypted packet

PLAIN_MDU = 464

The maximum size of the payload data in a single unencrypted packet

send()

Sends the packet.

Returns

A *RNS.PacketReceipt* instance if *create_receipt* was set to *True* when the packet was instantiated, if not returns *None*. If the packet could not be sent *False* is returned.

resend()

Re-sends the packet.

Returns

A *RNS.PacketReceipt* instance if *create_receipt* was set to *True* when the packet was instantiated, if not returns *None*. If the packet could not be sent *False* is returned.

get_rssi()

Returns

The physical layer *Received Signal Strength Indication* if available, otherwise *None*.

get_snr()

Returns

The physical layer *Signal-to-Noise Ratio* if available, otherwise *None*.

get_q()

Returns

The physical layer *Link Quality* if available, otherwise *None*.

13.5 Packet Receipt

class RNS.PacketReceipt

The PacketReceipt class is used to receive notifications about *RNS.Packet* instances sent over the network. Instances of this class are never created manually, but always returned from the *send()* method of a *RNS.Packet* instance.

get_status()

Returns

The status of the associated *RNS.Packet* instance. Can be one of *RNS.PacketReceipt.SENT*, *RNS.PacketReceipt.DELIVERED*, *RNS.PacketReceipt.FAILED* or *RNS.PacketReceipt.CULLED*.

get_rtt()

Returns

The round-trip-time in seconds

set_timeout(timeout)

Sets a timeout in seconds

Parameters

timeout – The timeout in seconds.

set_delivery_callback(*callback*)

Sets a function that gets called if a successful delivery has been proven.

Parameters

callback – A *callable* with the signature *callback(packet_receipt)*

set_timeout_callback(*callback*)

Sets a function that gets called if the delivery times out.

Parameters

callback – A *callable* with the signature *callback(packet_receipt)*

13.6 Link

class `RNS.Link`(*destination*, *established_callback*=None, *closed_callback*=None)

This class is used to establish and manage links to other peers. When a link instance is created, Reticulum will attempt to establish verified and encrypted connectivity with the specified destination.

Parameters

- **destination** – A *RNS.Destination* instance which to establish a link to.
- **established_callback** – An optional function or method with the signature *callback(link)* to be called when the link has been established.
- **closed_callback** – An optional function or method with the signature *callback(link)* to be called when the link is closed.

CURVE = 'Curve25519'

The curve used for Elliptic Curve DH key exchanges

ESTABLISHMENT_TIMEOUT_PER_HOP = 6

Timeout for link establishment in seconds per hop to destination.

KEEPALIVE_TIMEOUT_FACTOR = 4

RTT timeout factor used in link timeout calculation.

STALE_GRACE = 5

Grace period in seconds used in link timeout calculation.

KEEPALIVE = 360

Default interval for sending keep-alive packets on established links in seconds.

STALE_TIME = 720

If no traffic or keep-alive packets are received within this period, the link will be marked as stale, and a final keep-alive packet will be sent. If after this no traffic or keep-alive packets are received within $RTT * KEEPALIVE_TIMEOUT_FACTOR + STALE_GRACE$, the link is considered timed out, and will be torn down.

identify(*identity*)

Identifies the initiator of the link to the remote peer. This can only happen once the link has been established, and is carried out over the encrypted link. The identity is only revealed to the remote peer, and initiator anonymity is thus preserved. This method can be used for authentication.

Parameters

identity – An *RNS.Identity* instance to identify as.

request(*path*, *data=None*, *response_callback=None*, *failed_callback=None*, *progress_callback=None*, *timeout=None*)

Sends a request to the remote peer.

Parameters

- **path** – The request path.
- **response_callback** – An optional function or method with the signature *response_callback(request_receipt)* to be called when a response is received. See the [Request Example](#) for more info.
- **failed_callback** – An optional function or method with the signature *failed_callback(request_receipt)* to be called when a request fails. See the [Request Example](#) for more info.
- **progress_callback** – An optional function or method with the signature *progress_callback(request_receipt)* to be called when progress is made receiving the response. Progress can be accessed as a float between 0.0 and 1.0 by the *request_receipt.progress* property.
- **timeout** – An optional timeout in seconds for the request. If *None* is supplied it will be calculated based on link RTT.

Returns

A *RNS.RequestReceipt* instance if the request was sent, or *False* if it was not.

track_phy_stats(*track*)

You can enable physical layer statistics on a per-link basis. If this is enabled, and the link is running over an interface that supports reporting physical layer statistics, you will be able to retrieve stats such as *RSSI*, *SNR* and physical *Link Quality* for the link.

Parameters

track – Whether or not to keep track of physical layer statistics. Value must be *True* or *False*.

get_rssi()

Returns

The physical layer *Received Signal Strength Indication* if available, otherwise *None*. Physical layer statistics must be enabled on the link for this method to return a value.

get_snr()

Returns

The physical layer *Signal-to-Noise Ratio* if available, otherwise *None*. Physical layer statistics must be enabled on the link for this method to return a value.

get_q()

Returns

The physical layer *Link Quality* if available, otherwise *None*. Physical layer statistics must be enabled on the link for this method to return a value.

get_establishment_rate()

Returns

The data transfer rate at which the link establishment procedure occurred, in bits per second.

get_mtu()

Returns

The MTU of an established link.

get_mdu()

Returns

The packet MDU of an established link.

get_expected_rate()

Returns

The packet expected in-flight data rate of an established link.

get_mode()

Returns

The mode of an established link.

get_age()

Returns

The time in seconds since this link was established.

no_inbound_for()

Returns

The time in seconds since last inbound packet on the link. This includes keepalive packets.

no_outbound_for()

Returns

The time in seconds since last outbound packet on the link. This includes keepalive packets.

no_data_for()

Returns

The time in seconds since payload data traversed the link. This excludes keepalive packets.

inactive_for()

Returns

The time in seconds since activity on the link. This includes keepalive packets.

get_remote_identity()

Returns

The identity of the remote peer, if it is known. Calling this method will not query the remote initiator to reveal its identity. Returns `None` if the link initiator has not already independently called the `identify(identity)` method.

teardown()

Closes the link and purges encryption keys. New keys will be used if a new link to the same destination is established.

get_channel()

Get the `Channel` for this link.

Returns

`Channel` object

set_link_closed_callback(callback)

Registers a function to be called when a link has been torn down.

Parameters

callback – A function or method with the signature *callback(link)* to be called.

set_packet_callback(callback)

Registers a function to be called when a packet has been received over this link.

Parameters

callback – A function or method with the signature *callback(message, packet)* to be called.

set_resource_callback(callback)

Registers a function to be called when a resource has been advertised over this link. If the function returns *True* the resource will be accepted. If it returns *False* it will be ignored.

Parameters

callback – A function or method with the signature *callback(resource)* to be called. Please note that only the basic information of the resource is available at this time, such as *get_transfer_size()*, *get_data_size()*, *get_parts()* and *is_compressed()*.

set_resource_started_callback(callback)

Registers a function to be called when a resource has begun transferring over this link.

Parameters

callback – A function or method with the signature *callback(resource)* to be called.

set_resource_concluded_callback(callback)

Registers a function to be called when a resource has concluded transferring over this link.

Parameters

callback – A function or method with the signature *callback(resource)* to be called.

set_remote_identified_callback(callback)

Registers a function to be called when an initiating peer has identified over this link.

Parameters

callback – A function or method with the signature *callback(link, identity)* to be called.

set_resource_strategy(resource_strategy)

Sets the resource strategy for the link.

Parameters

resource_strategy – One of *RNS.Link.ACCEPT_NONE*, *RNS.Link.ACCEPT_ALL* or *RNS.Link.ACCEPT_APP*. If *RNS.Link.ACCEPT_APP* is set, the *resource_callback* will be called to determine whether the resource should be accepted or not.

Raises

TypeError if the resource strategy is unsupported.

13.7 Request Receipt

class RNS.RequestReceipt

An instance of this class is returned by the *request* method of *RNS.Link* instances. It should never be instantiated manually. It provides methods to check status, response time and response data when the request concludes.

get_request_id()

Returns

The request ID as *bytes*.

get_status()

Returns

The current status of the request, one of `RNS.RequestReceipt.FAILED`, `RNS.RequestReceipt.SENT`, `RNS.RequestReceipt.DELIVERED`, `RNS.RequestReceipt.READY`.

get_progress()

Returns

The progress of a response being received as a *float* between 0.0 and 1.0.

get_response()

Returns

The response as *bytes* if it is ready, otherwise *None*.

get_response_time()

Returns

The response time of the request in seconds.

concluded()

Returns

True if the associated request has concluded (successfully or with a failure), otherwise False.

13.8 Resource

```
class RNS.Resource(data, link, advertise=True, auto_compress=True, callback=None, progress_callback=None,
                    timeout=None)
```

The Resource class allows transferring arbitrary amounts of data over a link. It will automatically handle sequencing, compression, coordination and checksumming.

Parameters

- **data** – The data to be transferred. Can be *bytes* or an open *file handle*. See the [Filetransfer Example](#) for details.
- **link** – The [RNS.Link](#) instance on which to transfer the data.
- **advertise** – Optional. Whether to automatically advertise the resource. Can be *True* or *False*.
- **auto_compress** – Optional. Whether to auto-compress the resource. Can be *True* or *False*.
- **callback** – An optional *callable* with the signature `callback(resource)`. Will be called when the resource transfer concludes.
- **progress_callback** – An optional *callable* with the signature `callback(resource)`. Will be called whenever the resource transfer progress is updated.

advertise()

Advertise the resource. If the other end of the link accepts the resource advertisement it will begin transferring.

cancel()

Cancels transferring the resource.

get_progress()

Returns

The current progress of the resource transfer as a *float* between 0.0 and 1.0.

get_transfer_size()

Returns

The number of bytes needed to transfer the resource.

get_data_size()

Returns

The total data size of the resource.

get_parts()

Returns

The number of parts the resource will be transferred in.

get_segments()

Returns

The number of segments the resource is divided into.

get_hash()

Returns

The hash of the resource.

is_compressed()

Returns

Whether the resource is compressed.

13.9 Channel

class RNS.Channel.Channel

Provides reliable delivery of messages over a link.

Channel differs from Request and Resource in some important ways:

Continuous

Messages can be sent or received as long as the Link is open.

Bi-directional

Messages can be sent in either direction on the Link; neither end is the client or server.

Size-constrained

Messages must be encoded into a single packet.

Channel is similar to Packet, except that it provides reliable delivery (automatic retries) as well as a structure for exchanging several types of messages over the Link.

Channel is not instantiated directly, but rather obtained from a Link with `get_channel()`.

register_message_type(*message_class*: Type[MessageBase])

Register a message class for reception over a Channel.

Message classes must extend MessageBase.

Parameters

message_class – Class to register

add_message_handler(*callback*: MessageCallbackType)

Add a handler for incoming messages. A handler has the following signature:

(message: MessageBase) -> bool

Handlers are processed in the order they are added. If any handler returns True, processing of the message stops; handlers after the returning handler will not be called.

Parameters

callback – Function to call

remove_message_handler(*callback*: MessageCallbackType)

Remove a handler added with add_message_handler.

Parameters

callback – handler to remove

is_ready_to_send() → bool

Check if Channel is ready to send.

Returns

True if ready

send(*message*: MessageBase) → Envelope

Send a message. If a message send is attempted and Channel is not ready, an exception is thrown.

Parameters

message – an instance of a MessageBase subclass

property mdu

Maximum Data Unit: the number of bytes available for a message to consume in a single send. This value is adjusted from the Link MDU to accommodate message header information.

Returns

number of bytes available

13.10 MessageBase

class RNS.MessageBase

Base type for any messages sent or received on a Channel. Subclasses must define the two abstract methods as well as the MSGTYPE class variable.

MSGTYPE = None

Defines a unique identifier for a message class.

- Must be unique within all classes registered with a Channel
- Must be less than 0xf000. Values greater than or equal to 0xf000 are reserved.

abstractmethod pack() → bytes

Create and return the binary representation of the message

Returns

binary representation of message

abstractmethod unpack(*raw: bytes*)

Populate message from binary representation

Parameters

raw – binary representation

13.11 Buffer

class RNS.Buffer

Static functions for creating buffered streams that send and receive over a `Channel`.

These functions use `BufferedReader`, `BufferedWriter`, and `BufferedRWPair` to add buffering to `RawChannelReader` and `RawChannelWriter`.

static create_reader(*stream_id: int, channel: Channel, ready_callback: Callable[[int], None] | None = None*) → `BufferedReader`

Create a buffered reader that reads binary data sent over a `Channel`, with an optional callback when new data is available.

Callback signature: (ready_bytes: int) -> None

For more information on the reader-specific functions of this object, see the Python documentation for `BufferedReader`

Parameters

- **stream_id** – the local stream id to receive from
- **channel** – the channel to receive on
- **ready_callback** – function to call when new data is available

Returns

a `BufferedReader` object

static create_writer(*stream_id: int, channel: Channel*) → `BufferedWriter`

Create a buffered writer that writes binary data over a `Channel`.

For more information on the writer-specific functions of this object, see the Python documentation for `BufferedWriter`

Parameters

- **stream_id** – the remote stream id to send to
- **channel** – the channel to send on

Returns

a `BufferedWriter` object

static create_bidirectional_buffer(*receive_stream_id: int, send_stream_id: int, channel: Channel, ready_callback: Callable[[int], None] | None = None*) → `BufferedRWPair`

Create a buffered reader/writer pair that reads and writes binary data over a `Channel`, with an optional callback when new data is available.

Callback signature: (ready_bytes: int) -> None

For more information on the reader-specific functions of this object, see the Python documentation for `BufferedRWPair`

Parameters

- **receive_stream_id** – the local stream id to receive at
- **send_stream_id** – the remote stream id to send to
- **channel** – the channel to send and receive on
- **ready_callback** – function to call when new data is available

Returns

a `BufferedRWPair` object

13.12 RawChannelReader

class `RNS.RawChannelReader`(*stream_id: int, channel: Channel*)

An implementation of `RawIOBase` that receives binary stream data sent over a `Channel`.

This class generally need not be instantiated directly. Use `RNS.Buffer.create_reader()`, `RNS.Buffer.create_writer()`, and `RNS.Buffer.create_bidirectional_buffer()` functions to create buffered streams with optional callbacks.

For additional information on the API of this object, see the Python documentation for `RawIOBase`.

__init__(*stream_id: int, channel: Channel*)

Create a raw channel reader.

Parameters

- **stream_id** – local stream id to receive at
- **channel** – `Channel` object to receive from

add_ready_callback(*cb: Callable[[int], None]*)

Add a function to be called when new data is available. The function should have the signature (ready_bytes: int) -> None

Parameters

cb – function to call

remove_ready_callback(*cb: Callable[[int], None]*)

Remove a function added with `RNS.RawChannelReader.add_ready_callback()`

Parameters

cb – function to remove

13.13 RawChannelWriter

class `RNS.RawChannelWriter`(*stream_id: int, channel: Channel*)

An implementation of `RawIOBase` that receives binary stream data sent over a channel.

This class generally need not be instantiated directly. Use `RNS.Buffer.create_reader()`, `RNS.Buffer.create_writer()`, and `RNS.Buffer.create_bidirectional_buffer()` functions to create buffered streams with optional callbacks.

For additional information on the API of this object, see the Python documentation for `RawIOBase`.

__init__(*stream_id: int, channel: Channel*)

Create a raw channel writer.

Parameters

- **stream_id** – remote stream id to sent do
- **channel** – Channel object to send on

13.14 Transport

class RNS.Transport

Through static methods of this class you can interact with the Transport system of Reticulum.

PATHFINDER_M = 128

Maximum amount of hops that Reticulum will transport a packet.

static register_announce_handler(*handler*)

Registers an announce handler.

Parameters

handler – Must be an object with an *aspect_filter* attribute and a *received_announce(destination_hash, announced_identity, app_data)* or *received_announce(destination_hash, announced_identity, app_data, announce_packet_hash)* or *received_announce(destination_hash, announced_identity, app_data, announce_packet_hash, is_path_response)* callable. Can optionally have a *receive_path_responses* attribute set to *True*, to also receive all path responses, in addition to live announces. See the [Announce Example](#) for more info.

static deregister_announce_handler(*handler*)

Deregisters an announce handler.

Parameters

handler – The announce handler to be deregistered.

static has_path(*destination_hash*)

Parameters

destination_hash – A destination hash as *bytes*.

Returns

True if a path to the destination is known, otherwise *False*.

static hops_to(*destination_hash*)

Parameters

destination_hash – A destination hash as *bytes*.

Returns

The number of hops to the specified destination, or `RNS.Transport.PATHFINDER_M` if the number of hops is unknown.

static next_hop(*destination_hash*)

Parameters

destination_hash – A destination hash as *bytes*.

Returns

The destination hash as *bytes* for the next hop to the specified destination, or *None* if the next hop is unknown.

static next_hop_interface(*destination_hash*)

Parameters

destination_hash – A destination hash as *bytes*.

Returns

The interface for the next hop to the specified destination, or *None* if the interface is unknown.

static await_path(*destination_hash*, *timeout=None*, *on_interface=None*)

Requests a path to the destination from the network and blocks until the path is available, or the timeout is reached.

Parameters

- **destination_hash** – A destination hash as *bytes*.
- **timeout** – An optional timeout in seconds.
- **on_interface** – If specified, the path request will only be sent on this interface. In normal use, Reticulum handles this automatically, and this parameter should not be used.

Returns

True if a path to the destination is found, otherwise *False*.

static request_path(*destination_hash*, *on_interface=None*, *tag=None*, *recursive=False*)

Requests a path to the destination from the network. If another reachable peer on the network knows a path, it will announce it.

Parameters

- **destination_hash** – A destination hash as *bytes*.
- **on_interface** – If specified, the path request will only be sent on this interface. In normal use, Reticulum handles this automatically, and this parameter should not be used.

Symbols

`__init__()` (*RNS.RawChannelReader* method), 212
`__init__()` (*RNS.RawChannelWriter* method), 212

A

`accepts_links()` (*RNS.Destination* method), 199
`add_message_handler()` (*RNS.Channel.Channel* method), 210
`add_ready_callback()` (*RNS.RawChannelReader* method), 212
`advertise()` (*RNS.Resource* method), 208
`announce()` (*RNS.Destination* method), 199
`ANNOUNCE_CAP` (*RNS.Reticulum* attribute), 193
`app_and_aspects_from_name()` (*RNS.Destination* static method), 199
`await_path()` (*RNS.Transport* static method), 214

B

`blackhole_sources()` (*RNS.Reticulum* static method), 195
`Buffer` (*class in RNS*), 211

C

`cancel()` (*RNS.Resource* method), 208
`Channel` (*class in RNS.Channel*), 209
`clear_default_app_data()` (*RNS.Destination* method), 202
`concluded()` (*RNS.RequestReceipt* method), 208
`create_bidirectional_buffer()` (*RNS.Buffer* static method), 211
`create_keys()` (*RNS.Destination* method), 201
`create_reader()` (*RNS.Buffer* static method), 211
`create_writer()` (*RNS.Buffer* static method), 211
`current_ratchet_id()` (*RNS.Identity* static method), 196
`CURVE` (*RNS.Identity* attribute), 195
`CURVE` (*RNS.Link* attribute), 204

D

`decrypt()` (*RNS.Destination* method), 202
`decrypt()` (*RNS.Identity* method), 197

`deregister_announce_handler()` (*RNS.Transport* static method), 213
`deregister_request_handler()` (*RNS.Destination* method), 200
`Destination` (*class in RNS*), 198
`discovered_interfaces()` (*RNS.Reticulum* static method), 195

E

`enable_ratchets()` (*RNS.Destination* method), 200
`encrypt()` (*RNS.Destination* method), 201
`encrypt()` (*RNS.Identity* method), 197
`ENCRYPTED_MDU` (*RNS.Packet* attribute), 202
`enforce_ratchets()` (*RNS.Destination* method), 201
`ESTABLISHMENT_TIMEOUT_PER_HOP` (*RNS.Link* attribute), 204
`expand_name()` (*RNS.Destination* static method), 199

F

`from_bytes()` (*RNS.Identity* static method), 196
`from_file()` (*RNS.Identity* static method), 197
`full_hash()` (*RNS.Identity* static method), 196

G

`get_age()` (*RNS.Link* method), 206
`get_channel()` (*RNS.Link* method), 206
`get_data_size()` (*RNS.Resource* method), 209
`get_establishment_rate()` (*RNS.Link* method), 205
`get_expected_rate()` (*RNS.Link* method), 206
`get_hash()` (*RNS.Resource* method), 209
`get_instance()` (*RNS.Reticulum* static method), 194
`get_mdu()` (*RNS.Link* method), 206
`get_mode()` (*RNS.Link* method), 206
`get_mtu()` (*RNS.Link* method), 205
`get_parts()` (*RNS.Resource* method), 209
`get_private_key()` (*RNS.Destination* method), 201
`get_private_key()` (*RNS.Identity* method), 197
`get_progress()` (*RNS.RequestReceipt* method), 208
`get_progress()` (*RNS.Resource* method), 209
`get_public_key()` (*RNS.Identity* method), 197
`get_q()` (*RNS.Link* method), 205
`get_q()` (*RNS.Packet* method), 203

[get_random_hash\(\)](#) (*RNS.Identity static method*), 196
[get_remote_identity\(\)](#) (*RNS.Link method*), 206
[get_request_id\(\)](#) (*RNS.RequestReceipt method*), 207
[get_response\(\)](#) (*RNS.RequestReceipt method*), 208
[get_response_time\(\)](#) (*RNS.RequestReceipt method*), 208
[get_rssi\(\)](#) (*RNS.Link method*), 205
[get_rssi\(\)](#) (*RNS.Packet method*), 203
[get_rtt\(\)](#) (*RNS.PacketReceipt method*), 203
[get_segments\(\)](#) (*RNS.Resource method*), 209
[get_snr\(\)](#) (*RNS.Link method*), 205
[get_snr\(\)](#) (*RNS.Packet method*), 203
[get_status\(\)](#) (*RNS.PacketReceipt method*), 203
[get_status\(\)](#) (*RNS.RequestReceipt method*), 208
[get_transfer_size\(\)](#) (*RNS.Resource method*), 209

H

[has_path\(\)](#) (*RNS.Transport static method*), 213
[hash\(\)](#) (*RNS.Destination static method*), 199
[hash_from_name_and_identity\(\)](#) (*RNS.Destination static method*), 199
[hops_to\(\)](#) (*RNS.Transport static method*), 213

I

[identify\(\)](#) (*RNS.Link method*), 204
[Identity](#) (*class in RNS*), 195
[inactive_for\(\)](#) (*RNS.Link method*), 206
[interface_discovery_sources\(\)](#) (*RNS.Reticulum static method*), 195
[is_compressed\(\)](#) (*RNS.Resource method*), 209
[is_ready_to_send\(\)](#) (*RNS.Channel.Channel method*), 210

K

[KEEPALIVE](#) (*RNS.Link attribute*), 204
[KEEPALIVE_TIMEOUT_FACTOR](#) (*RNS.Link attribute*), 204
[KEYSIZE](#) (*RNS.Identity attribute*), 195

L

[Link](#) (*class in RNS*), 204
[LINK_MTU_DISCOVERY](#) (*RNS.Reticulum attribute*), 193
[link_mtu_discovery\(\)](#) (*RNS.Reticulum static method*), 194
[load_private_key\(\)](#) (*RNS.Destination method*), 201
[load_private_key\(\)](#) (*RNS.Identity method*), 197
[load_public_key\(\)](#) (*RNS.Identity method*), 197

M

[mdu](#) (*RNS.Channel.Channel property*), 210
[MessageBase](#) (*class in RNS*), 210
[MINIMUM_BITRATE](#) (*RNS.Reticulum attribute*), 194
[MSGTYPE](#) (*RNS.MessageBase attribute*), 210
[MTU](#) (*RNS.Reticulum attribute*), 193

N

[next_hop\(\)](#) (*RNS.Transport static method*), 213
[next_hop_interface\(\)](#) (*RNS.Transport static method*), 213
[no_data_for\(\)](#) (*RNS.Link method*), 206
[no_inbound_for\(\)](#) (*RNS.Link method*), 206
[no_outbound_for\(\)](#) (*RNS.Link method*), 206

P

[pack\(\)](#) (*RNS.MessageBase method*), 210
[Packet](#) (*class in RNS*), 202
[PacketReceipt](#) (*class in RNS*), 203
[PATHFINDER_M](#) (*RNS.Transport attribute*), 213
[PLAIN_MDU](#) (*RNS.Packet attribute*), 202
[publish_blackhole_enabled\(\)](#) (*RNS.Reticulum static method*), 194

R

[RATCHET_COUNT](#) (*RNS.Destination attribute*), 198
[RATCHET_EXPIRY](#) (*RNS.Identity attribute*), 195
[RATCHET_INTERVAL](#) (*RNS.Destination attribute*), 199
[RATCHETSIZE](#) (*RNS.Identity attribute*), 195
[RawChannelReader](#) (*class in RNS*), 212
[RawChannelWriter](#) (*class in RNS*), 212
[recall\(\)](#) (*RNS.Identity static method*), 195
[recall_app_data\(\)](#) (*RNS.Identity static method*), 196
[register_announce_handler\(\)](#) (*RNS.Transport static method*), 213
[register_message_type\(\)](#) (*RNS.Channel.Channel method*), 209
[register_request_handler\(\)](#) (*RNS.Destination method*), 200
[remote_management_enabled\(\)](#) (*RNS.Reticulum static method*), 194
[remove_message_handler\(\)](#) (*RNS.Channel.Channel method*), 210
[remove_ready_callback\(\)](#) (*RNS.RawChannelReader method*), 212
[request\(\)](#) (*RNS.Link method*), 204
[request_path\(\)](#) (*RNS.Transport static method*), 214
[RequestReceipt](#) (*class in RNS*), 207
[required_discovery_value\(\)](#) (*RNS.Reticulum static method*), 194
[resend\(\)](#) (*RNS.Packet method*), 203
[Resource](#) (*class in RNS*), 208
[Reticulum](#) (*class in RNS*), 193

S

[send\(\)](#) (*RNS.Channel.Channel method*), 210
[send\(\)](#) (*RNS.Packet method*), 203
[set_default_app_data\(\)](#) (*RNS.Destination method*), 202
[set_delivery_callback\(\)](#) (*RNS.PacketReceipt method*), 203

set_link_closed_callback() (*RNS.Link method*),
 206
 set_link_established_callback()
 (*RNS.Destination method*), 199
 set_packet_callback() (*RNS.Destination method*),
 199
 set_packet_callback() (*RNS.Link method*), 207
 set_proof_requested_callback() (*RNS.Destination*
 method), 200
 set_proof_strategy() (*RNS.Destination method*),
 200
 set_ratchet_interval() (*RNS.Destination method*),
 201
 set_remote_identified_callback() (*RNS.Link*
 method), 207
 set_resource_callback() (*RNS.Link method*), 207
 set_resource_concluded_callback() (*RNS.Link*
 method), 207
 set_resource_started_callback() (*RNS.Link*
 method), 207
 set_resource_strategy() (*RNS.Link method*), 207
 set_retained_ratchets() (*RNS.Destination method*),
 201
 set_timeout() (*RNS.PacketReceipt method*), 203
 set_timeout_callback() (*RNS.PacketReceipt*
 method), 204
 should_use_implicit_proof() (*RNS.Reticulum*
 static method), 194
 sign() (*RNS.Destination method*), 202
 sign() (*RNS.Identity method*), 198
 STALE_GRACE (*RNS.Link attribute*), 204
 STALE_TIME (*RNS.Link attribute*), 204

T

teardown() (*RNS.Link method*), 206
 to_file() (*RNS.Identity method*), 197
 track_phy_stats() (*RNS.Link method*), 205
 Transport (*class in RNS*), 213
 transport_enabled() (*RNS.Reticulum static method*),
 194
 truncated_hash() (*RNS.Identity static method*), 196
 TRUNCATED_HASHLENGTH (*RNS.Identity attribute*), 195

U

unpack() (*RNS.MessageBase method*), 211

V

validate() (*RNS.Identity method*), 198